Credit Default Risk with Logistic Regression

Although many machine learning techniques deliver strong predictive performance, logistic regression remains the preferred choice in finance and risk management for its computational efficiency and interpretability.

In this project, I conduct data analysis, data cleaning and preparation, then build a logistic regression model to predict transaction defaults. Finally, I develop a credit scorecard from the model results.

The project employs industry-standard techniques like Weight of Evidence (WoE) for variable binning and the Youden Index (J statistic) for optimal threshold selection.

The dataset contains all available data for more than 800,000 consumer loans issued from 2007 to 2015 by Lending Club: a large US peer-to-peer lending company. There are several different versions of this dataset.

Preprocessing

In [1]:
```python
# Import the required libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, RepeatedStratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, roc_auc_score, confusion_matrix, precision_recall_curve, auc
from sklearn.feature_selection import f_classif
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin
from scipy.stats import chi2_contingency
```

In [2]:
```python
#Read the data
loan_data = pd.read_csv('/Users/camnguyen/Documents/Credit_Scorecard/loan_data_2007_2014.csv')
loan_data.head()
```

```
/var/folders/6h/8882zf396gl7nvkkt7wy23dh0000gn/T/ipykernel_41296/695972018.py:2: DtypeWarning: Columns (0: desc) have mixed types. Specify dtype option on import or set low_memor
y=False.
  loan_data = pd.read_csv('/Users/camnguyen/Documents/Credit_Scorecard/loan_data_2007_2014.csv')
```

Out[2]:

	Unnamed: 0	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	...	total_bal_il	il_util	open_rv_12m	open_rv_24m	max_bal_bc	all_util	total_re
0	0	1077501	1296599	5000	5000	4975.0	36 months	10.65	162.87	B	...	NaN	NaN	NaN	NaN	NaN	NaN	
1	1	1077430	1314167	2500	2500	2500.0	60 months	15.27	59.83	C	...	NaN	NaN	NaN	NaN	NaN	NaN	
2	2	1077175	1313524	2400	2400	2400.0	36 months	15.96	84.33	C	...	NaN	NaN	NaN	NaN	NaN	NaN	
3	3	1076863	1277178	10000	10000	10000.0	36 months	13.49	339.31	C	...	NaN	NaN	NaN	NaN	NaN	NaN	
4	4	1075358	1311748	3000	3000	3000.0	60 months	12.69	67.79	B	...	NaN	NaN	NaN	NaN	NaN	NaN	

5 rows × 75 columns

In [3]:
```python
pd.options.display.max_columns = None
loan_data
```

Out[3]:

	Unnamed: 0	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ownership	annual_inc	verifica
0	0	1077501	1296599	5000	5000	4975.0	36 months	10.65	162.87	B	B2	NaN	10+ years	RENT	24000.0	
1	1	1077430	1314167	2500	2500	2500.0	60 months	15.27	59.83	C	C4	Ryder	< 1 year	RENT	30000.0	So
2	2	1077175	1313524	2400	2400	2400.0	36 months	15.96	84.33	C	C5	NaN	10+ years	RENT	12252.0	
3	3	1076863	1277178	10000	10000	10000.0	36 months	13.49	339.31	C	C1	AIR RESOURCES BOARD	10+ years	RENT	49200.0	So
4	4	1075358	1311748	3000	3000	3000.0	60 months	12.69	67.79	B	B5	University Medical Group	1 year	RENT	80000.0	So
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
466280	466280	8598660	1440975	18400	18400	18400.0	60 months	14.47	432.64	C	C2	Financial Advisor	4 years	MORTGAGE	110000.0	So
466281	466281	9684700	11536848	22000	22000	22000.0	60 months	19.97	582.50	D	D5	Chief of Interpretation (Park Ranger)	10+ years	MORTGAGE	78000.0	
466282	466282	9584776	11436914	20700	20700	20700.0	60 months	16.99	514.34	D	D1	patrol	7 years	MORTGAGE	46000.0	
466283	466283	9604874	11457002	2000	2000	2000.0	36 months	7.90	62.59	A	A4	Server Engineer Lead	3 years	OWN	83000.0	
466284	466284	9199665	11061576	10000	10000	9975.0	36 months	19.20	367.58	D	D3	NaN	10+ years	MORTGAGE	46000.0	

466285 rows × 75 columns

LoanStatNew	Description
addr_state	The state provided by the borrower in the loan application
annual_inc	The self-reported annual income provided by the borrower during registration.
annual_inc_joint	The combined self-reported annual income provided by the co-borrowers during registration
application_type	Indicates whether the loan is an individual application or a joint application with two co-borrowers
collection_recovery_fee	Post charge off collection fee
collections_12_mths_ex_med	Number of collections in 12 months excluding medical collections
delinq_2yrs	The number of 30+ days past-due incidences of delinquency in the borrower's credit file for the past 2 years
desc	Loan description provided by the borrower
dti	A ratio calculated using the borrower's total monthly debt payments on the total debt obligations, excluding mortgage and the requested LC loan, divided by the borrower's self-reported monthly income.
dti_joint	A ratio calculated using the co-borrowers' total monthly payments on the total debt obligations, excluding mortgages and the requested LC loan, divided by the co-borrowers' combined self-reported monthly income
earliest_cr_line	The month the borrower's earliest reported credit line was opened
emp_length	Employment length in years. Possible values are between 0 and 10 where 0 means less than one year and 10 means ten or more years.
emp_title	The job title supplied by the borrower when applying for the loan.
fico_range_high	The upper boundary range the borrower's FICO at loan origination belongs to.
fico_range_low	The lower boundary range the borrower's FICO at loan origination belongs to.
funded_amnt	The total amount committed to that loan at that point in time.
funded_amnt_inv	The total amount committed by investors for that loan at that point in time.
grade	LC assigned loan grade
home_ownership	The home ownership status provided by the borrower during registration. Values: RENT, OWN, MORTGAGE, OTHER.
id	A unique LC assigned ID for the loan listing.
initial_list_status	The initial listing status of the loan. Possible values are – W, F
inq_last_6mths	The number of inquiries in past 6 months (excluding auto and mortgage inquiries)
installment	The monthly payment owed by the borrower if the loan originates.
int_rate	Interest rate on the loan
is_inc_v	Indicates if income was verified by LC, not verified, or if the income source was verified
issue_d	The month which the loan was funded
last_credit_pull_d	The most recent month LC pulled credit for this loan
last_fico_range_high	The upper boundary range the borrower's last FICO pulled belongs to.
last_fico_range_low	The lower boundary range the borrower's last FICO pulled belongs to.
last_pymnt_amnt	Last total payment amount received
last_pymnt_d	Last month payment was received
loan_amnt	The listed amount of the loan applied for by the borrower. If at some point in time, the credit department reduces the loan amount, then it will be reflected in this value.
loan_status	Current status of the loan
member_id	A unique LC assigned Id for the borrower member.
mths_since_last_delinq	The number of months since the borrower's last delinquency.
mths_since_last_major_derog	Months since most recent 90-day or worse rating
mths_since_last_record	The number of months since the last public record.
next_pymnt_d	Next scheduled payment date
open_acc	The number of open credit lines in the borrower's credit file.
out_prncp	Remaining outstanding principal for total amount funded
out_prncp_inv	Remaining outstanding principal for portion of total amount funded by investors
policy_code	Publicly available policy_code=1; new products not publicly available policy_code=2
pub_rec	Number of derogatory public records
purpose	A category provided by the borrower for the loan request.
pymnt_plan	Indicates if a payment plan has been put in place for the loan
recoveries	Post charge off gross recovery
revol_bal	Total credit revolving balance
revol_util	Revolving line utilization rate, or the amount of credit the borrower is using relative to all available revolving credit.
sub_grade	LC assigned loan subgrade
term	The number of payments on the loan. Values are in months and can be either 36 or 60.
title	The loan title provided by the borrower
total_acc	The total number of credit lines currently in the borrower's credit file
total_pymnt	Payments received to date for total amount funded
total_pymnt_inv	Payments received to date for portion of total amount funded by investors
total_rec_int	Interest received to date
total_rec_late_fee	Late fees received to date
total_rec_prncp	Principal received to date
url	URL for the LC page with listing data.
verified_status_joint	Indicates if the co-borrowers' joint income was verified by LC, not verified, or if the income source was verified
zip_code	The first 3 numbers of the zip code provided by the borrower in the loan application.
open_acc_6m	Number of open trades in last 6 months
open_il_6m	Number of currently active installment trades
open_il_12m	Number of installment accounts opened in past 12 months
open_il_24m	Number of installment accounts opened in past 24 months
mths_since_rcnt_il	Months since most recent installment accounts opened
total_bal_il	Total current balance of all installment accounts
il_util	Ratio of total current balance to high credit/credit limit on all install accounts
open_rv_12m	Number of revolving trades opened in past 12 months
open_rv_24m	Number of revolving trades opened in past 24 months
max_bal_bc	Maximum current balance owed on all revolving accounts
all_util	Balance to credit limit on all trades
total_rev_hi_lim	Total revolving high credit/credit limit
inq_fi	Number of personal finance inquiries
total_cu_tl	Number of finance trades
inq_last_12m	Number of credit inquiries in past 12 months
acc_now_delinq	The number of accounts on which the borrower is now delinquent.
tot_coll_amt	Total collection amounts ever owed
tot_cur_bal	Total current balance of all accounts

Variables we will need for further analysis: annual_inc, deling_2yrs, inq_last_6mths, open_acc, pub_rec, total_acc, acc_now_deling, total_rev_hi_lim, emp_length_int, mths_since_earliest_cr_line

In [4]:
```python
#please show all column in the dataframe
pd.set_option('display.max_columns', None)
```

In [5]:
```python
loan_data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 466285 entries, 0 to 466284
Data columns (total 75 columns):
 #   Column                       Non-Null Count   Dtype  
---  ------                       --------------   -----  
 0   Unnamed: 0                   466285 non-null  int64  
 1   id                           466285 non-null  int64  
 2   member_id                    466285 non-null  int64  
 3   loan_amnt                    466285 non-null  int64  
 4   funded_amnt                  466285 non-null  int64  
 5   funded_amnt_inv              466285 non-null  float64
 6   term                         466285 non-null  str    
 7   int_rate                     466285 non-null  float64
 8   installment                  466285 non-null  float64
 9   grade                        466285 non-null  str    
 10  sub_grade                    466285 non-null  str    
 11  emp_title                    438697 non-null  str    
 12  emp_length                   445277 non-null  str    
 13  home_ownership               466285 non-null  str    
 14  annual_inc                   466281 non-null  float64
 15  verification_status          466285 non-null  str    
 16  issue_d                      466285 non-null  str    
 17  loan_status                  466285 non-null  str    
 18  pymnt_plan                   466285 non-null  str    
 19  url                          466285 non-null  str    
 20  desc                         125981 non-null  str    
 21  purpose                      466285 non-null  str    
 22  title                        466264 non-null  str    
 23  zip_code                     466285 non-null  str    
 24  addr_state                   466285 non-null  str    
 25  dti                          466285 non-null  float64
 26  delinq_2yrs                  466256 non-null  float64
 27  earliest_cr_line             466256 non-null  str    
 28  inq_last_6mths               466256 non-null  float64
 29  mths_since_last_delinq       215934 non-null  float64
 30  mths_since_last_record       62638 non-null   float64
 31  open_acc                     466256 non-null  float64
 32  pub_rec                      466256 non-null  float64
 33  revol_bal                    466285 non-null  int64  
 34  revol_util                   465945 non-null  float64
 35  total_acc                    466256 non-null  float64
 36  initial_list_status          466285 non-null  str    
 37  out_prncp                    466285 non-null  float64
 38  out_prncp_inv                466285 non-null  float64
 39  total_pymnt                  466285 non-null  float64
 40  total_pymnt_inv              466285 non-null  float64
 41  total_rec_prncp              466285 non-null  float64
 42  total_rec_int                466285 non-null  float64
 43  total_rec_late_fee           466285 non-null  float64
 44  recoveries                   466285 non-null  float64
 45  collection_recovery_fee      466285 non-null  float64
 46  last_pymnt_d                 465909 non-null  str    
 47  last_pymnt_amnt              466285 non-null  float64
 48  next_pymnt_d                 239071 non-null  str    
 49  last_credit_pull_d           466243 non-null  str    
 50  collections_12_mths_ex_med   466140 non-null  float64
 51  mths_since_last_major_derog  98974 non-null   float64
 52  policy_code                  466285 non-null  int64  
 53  application_type             466285 non-null  str    
 54  annual_inc_joint             0 non-null       float64
 55  dti_joint                    0 non-null       float64
 56  verification_status_joint    0 non-null       float64
 57  acc_now_delinq               466256 non-null  float64
 58  tot_coll_amt                 396009 non-null  float64
 59  tot_cur_bal                  396009 non-null  float64
 60  open_acc_6m                  0 non-null       float64
 61  open_il_6m                   0 non-null       float64
 62  open_il_12m                  0 non-null       float64
 63  open_il_24m                  0 non-null       float64
 64  mths_since_rcnt_il           0 non-null       float64
 65  total_bal_il                 0 non-null       float64
 66  il_util                      0 non-null       float64
 67  open_rv_12m                  0 non-null       float64
 68  open_rv_24m                  0 non-null       float64
 69  max_bal_bc                   0 non-null       float64
 70  all_util                     0 non-null       float64
 71  total_rev_hi_lim             396009 non-null  float64
 72  inq_fi                       0 non-null       float64
 73  total_cu_tl                  0 non-null       float64
 74  inq_last_12m                 0 non-null       float64
dtypes: float64(46), int64(7), str(22)
memory usage: 266.8 MB
```

In [6]:
```
pip install openpyxl
```

```
Collecting openpyxl
  Using cached openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et-xmlfile (from openpyxl)
  Using cached et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Using cached openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Using cached et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2/2 [openpyxl]
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
Note: you may need to restart the kernel to use updated packages.
```

In [7]:
```python
#show all the fully paid column values
loan_data['loan_status'].value_counts()
```

Out[7]:
```
loan_status
Current                                                224226
Fully Paid                                             184739
Charged Off                                             42475
Late (31-120 days)                                       6900
In Grace Period                                          3146
Does not meet the credit policy. Status:Fully Paid       1988
Late (16-30 days)                                        1218
Default                                                   832
Does not meet the credit policy. Status:Charged Off       761
Name: count, dtype: int64
```

In [8]:
```python
loan_data['annual_inc'].fillna(loan_data['annual_inc'].mean(), inplace=True)
# We fill the missing values with the mean value of the non-missing values.
```

```
/var/folders/6h/8882zf396gl7nvkkt7wy23dh0000gn/T/ipykernel_41296/1423679003.py:1: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-o
n-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inplace on the original object,
or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html
  loan_data['annual_inc'].fillna(loan_data['annual_inc'].mean(), inplace=True)
```

Out[8]:
```
0          24000.0
1          30000.0
2          12252.0
3          49200.0
4          80000.0
            ...   
466280    110000.0
466281     78000.0
466282     46000.0
466283     83000.0
466284     46000.0
Name: annual_inc, Length: 466285, dtype: float64
```

In [9]:
```python
# get a list of columns that have more than 80% null values
na_values = loan_data.isnull().mean()
na_values[na_values>0.8]
```

Out[9]:
```
mths_since_last_record       0.865666
annual_inc_joint             1.000000
dti_joint                    1.000000
verification_status_joint    1.000000
open_acc_6m                  1.000000
open_il_6m                   1.000000
open_il_12m                  1.000000
open_il_24m                  1.000000
mths_since_rcnt_il           1.000000
total_bal_il                 1.000000
il_util                      1.000000
open_rv_12m                  1.000000
open_rv_24m                  1.000000
max_bal_bc                   1.000000
all_util                     1.000000
inq_fi                       1.000000
total_cu_tl                  1.000000
inq_last_12m                 1.000000
dtype: float64
```

In [10]:
```python
# drop columns with more than 80% null values
loan_data.dropna(thresh = loan_data.shape[0]*0.2,  axis = 1, inplace = True)

'''
drop redundant and forward-looking columns
redundant like id, member_id, title, etc.
forward-looking like recoveries, collection_recovery_fee, etc.
drop sub_grade as same information is captured in grade column
drop next_pymnt_d since, given that our data is historical and this column is supposed to have future dates, will not make sense for our model
'''
loan_data.drop(columns = ['id', 'member_id', 'sub_grade', 'emp_title', 'url', 'desc', 'title', 'zip_code', 'next_pymnt_d',
                          'recoveries', 'collection_recovery_fee', 'total_rec_prncp', 'total_rec_late_fee'], inplace = True)
```

In [11]:
```python
loan_data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 466285 entries, 0 to 466284
Data columns (total 44 columns):
 #   Column                       Non-Null Count   Dtype  
---  ------                       --------------   -----  
 0   Unnamed: 0                   466285 non-null  int64  
 1   loan_amnt                    466285 non-null  int64  
 2   funded_amnt                  466285 non-null  int64  
 3   funded_amnt_inv              466285 non-null  float64
 4   term                         466285 non-null  str    
 5   int_rate                     466285 non-null  float64
 6   installment                  466285 non-null  float64
 7   grade                        466285 non-null  str    
 8   emp_length                   445277 non-null  str    
 9   home_ownership               466285 non-null  str    
 10  annual_inc                   466281 non-null  float64
 11  verification_status          466285 non-null  str    
 12  issue_d                      466285 non-null  str    
 13  loan_status                  466285 non-null  str    
 14  pymnt_plan                   466285 non-null  str    
 15  purpose                      466285 non-null  str    
 16  addr_state                   466285 non-null  str    
 17  dti                          466285 non-null  float64
 18  delinq_2yrs                  466256 non-null  float64
 19  earliest_cr_line             466256 non-null  str    
 20  inq_last_6mths               466256 non-null  float64
 21  mths_since_last_delinq       215934 non-null  float64
 22  open_acc                     466256 non-null  float64
 23  pub_rec                      466256 non-null  float64
 24  revol_bal                    466285 non-null  int64  
 25  revol_util                   465945 non-null  float64
 26  total_acc                    466256 non-null  float64
 27  initial_list_status          466285 non-null  str    
 28  out_prncp                    466285 non-null  float64
 29  out_prncp_inv                466285 non-null  float64
 30  total_pymnt                  466285 non-null  float64
 31  total_pymnt_inv              466285 non-null  float64
 32  total_rec_int                466285 non-null  float64
 33  last_pymnt_d                 465909 non-null  str    
 34  last_pymnt_amnt              466285 non-null  float64
 35  last_credit_pull_d           466243 non-null  str    
 36  collections_12_mths_ex_med   466140 non-null  float64
 37  mths_since_last_major_derog  98974 non-null   float64
 38  policy_code                  466285 non-null  int64  
 39  application_type             466285 non-null  str    
 40  acc_now_delinq               466256 non-null  float64
 41  tot_coll_amt                 396009 non-null  float64
 42  tot_cur_bal                  396009 non-null  float64
 43  total_rev_hi_lim             396009 non-null  float64
dtypes: float64(24), int64(5), str(15)
memory usage: 156.5 MB
```

Preprocessing continuous variables

In [12]:
```python
loan_data['emp_length'].unique()
```

Out[12]:
```
<StringArray>
['10+ years',  '< 1 year',    '1 year',   '3 years',   '8 years',   '9 years',
   '4 years',   '5 years',   '6 years',   '2 years',   '7 years',         nan]
Length: 12, dtype: str
```

emp_length now is still object

In [13]:
```python
#remove notify
import warnings
warnings.filterwarnings('ignore')
```

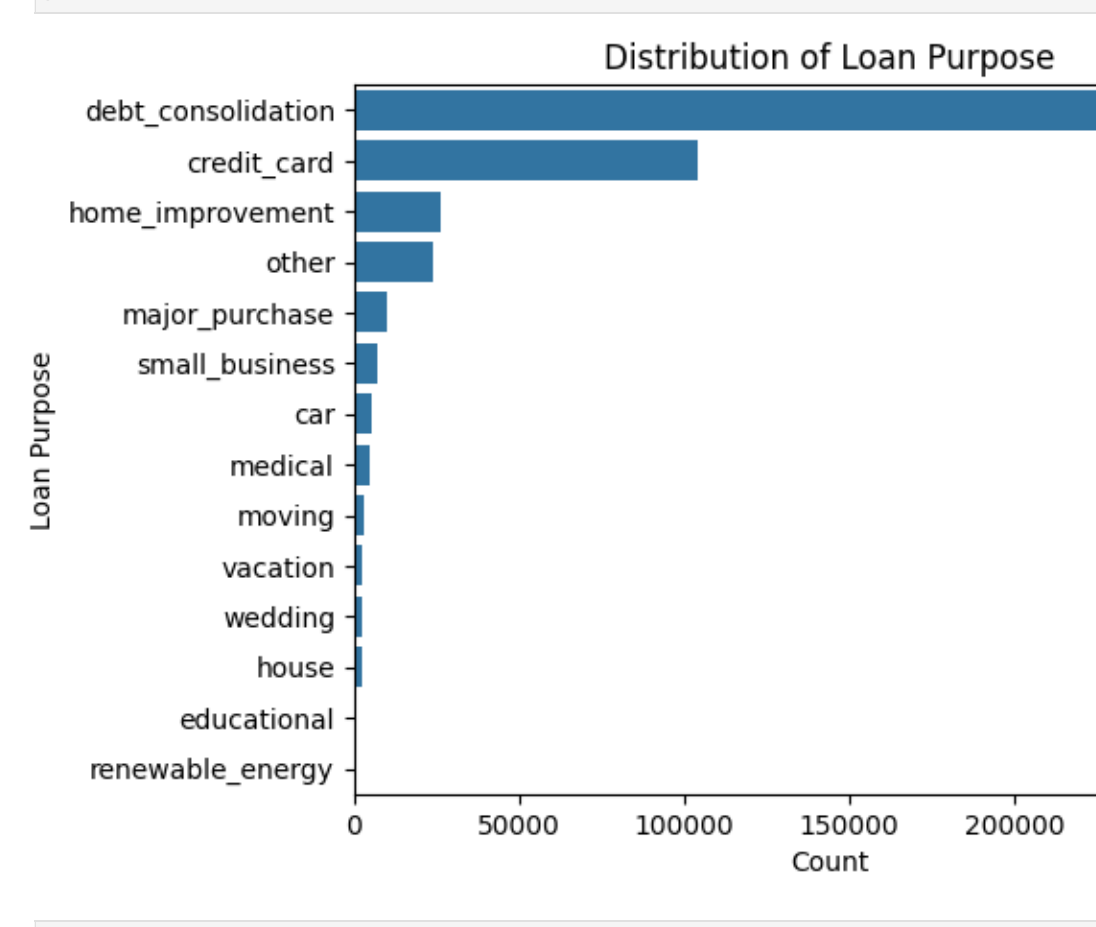
In [14]:
```python
loan_data['loan_status'].unique()
# Displays unique values of a column.
```

Out[14]:
```
<StringArray>
[                                          'Fully Paid',
                                         'Charged Off',
                                             'Current',
                                             'Default',
                                  'Late (31-120 days)',
                                     'In Grace Period',
                                   'Late (16-30 days)',
  'Does not meet the credit policy. Status:Fully Paid',
 'Does not meet the credit policy. Status:Charged Off']
Length: 9, dtype: str
```

In [15]:
```python
# create a new column based on the loan_status column that will be our target variable
loan_data['good_bad'] = np.where(loan_data.loc[:, 'loan_status'].isin(['Charged Off', 'Default', 'Late (31-120 days)',
                                                                     'Does not meet the credit policy. Status:Charged Off']), 0, 1)
# Drop the original 'loan_status' column
loan_data.drop(columns = ['loan_status'], inplace = True)
```

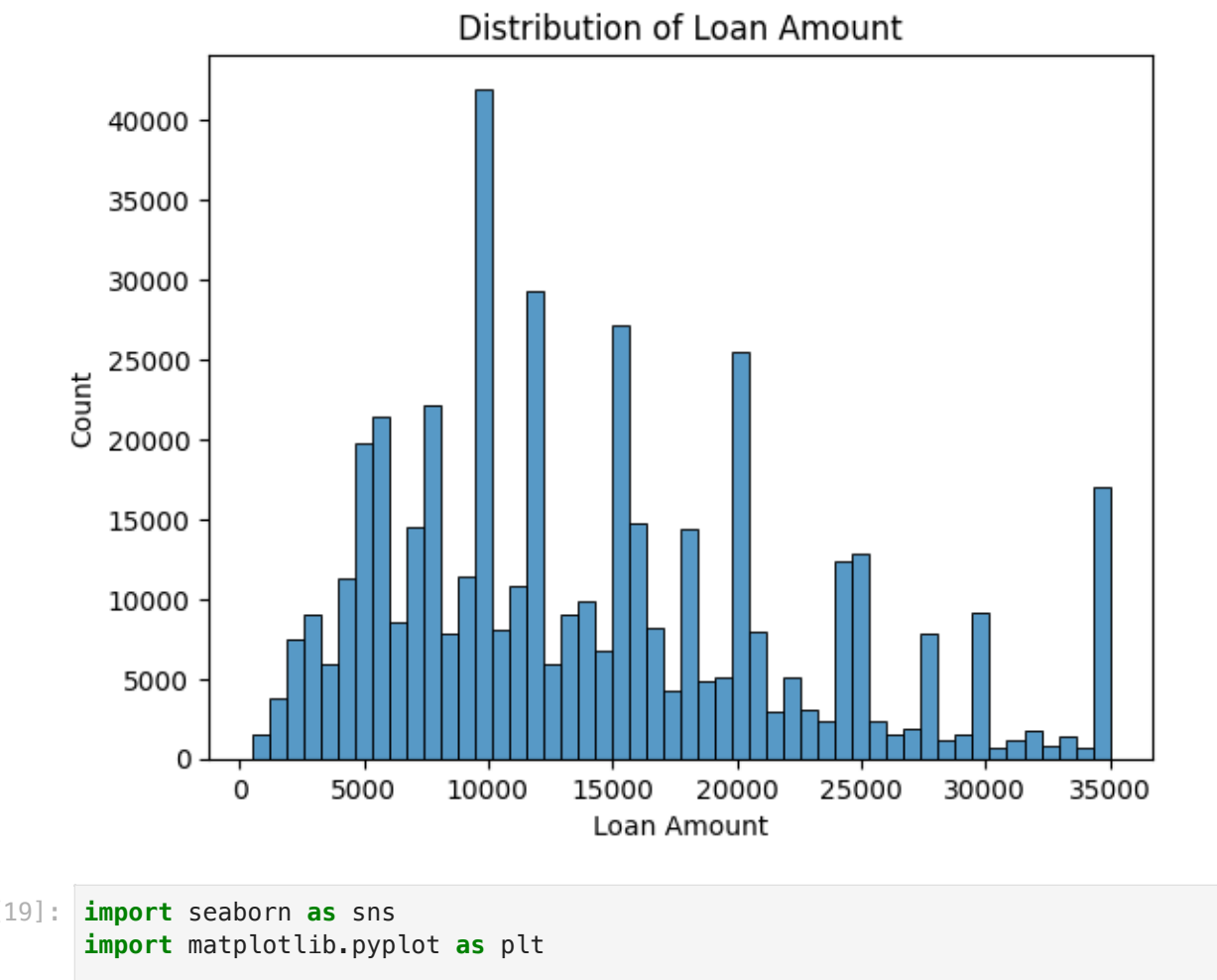
Exploratory Data Analysis

In [16]:
```python
#number of good and bad loans
loan_data['good_bad'].value_counts()
#plot the distribution of the target variable
sns.countplot(x = 'good_bad', data = loan_data)
plt.title('Distribution of Good and Bad Loans')
plt.xlabel('Loan Status (0 = Bad, 1 = Good)')
plt.ylabel('Count')
plt.show()
```

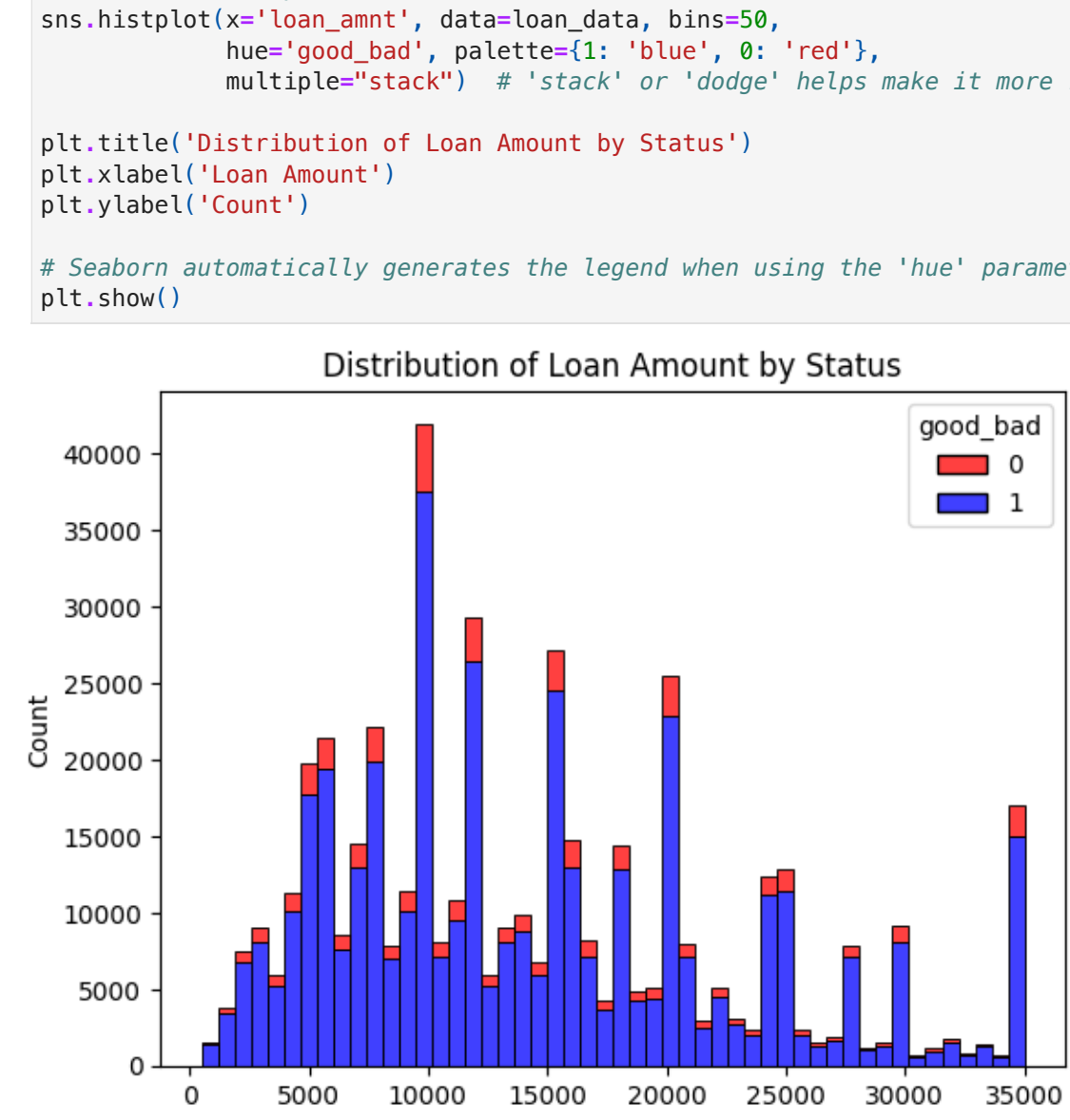


The distribution is completely imbalanced

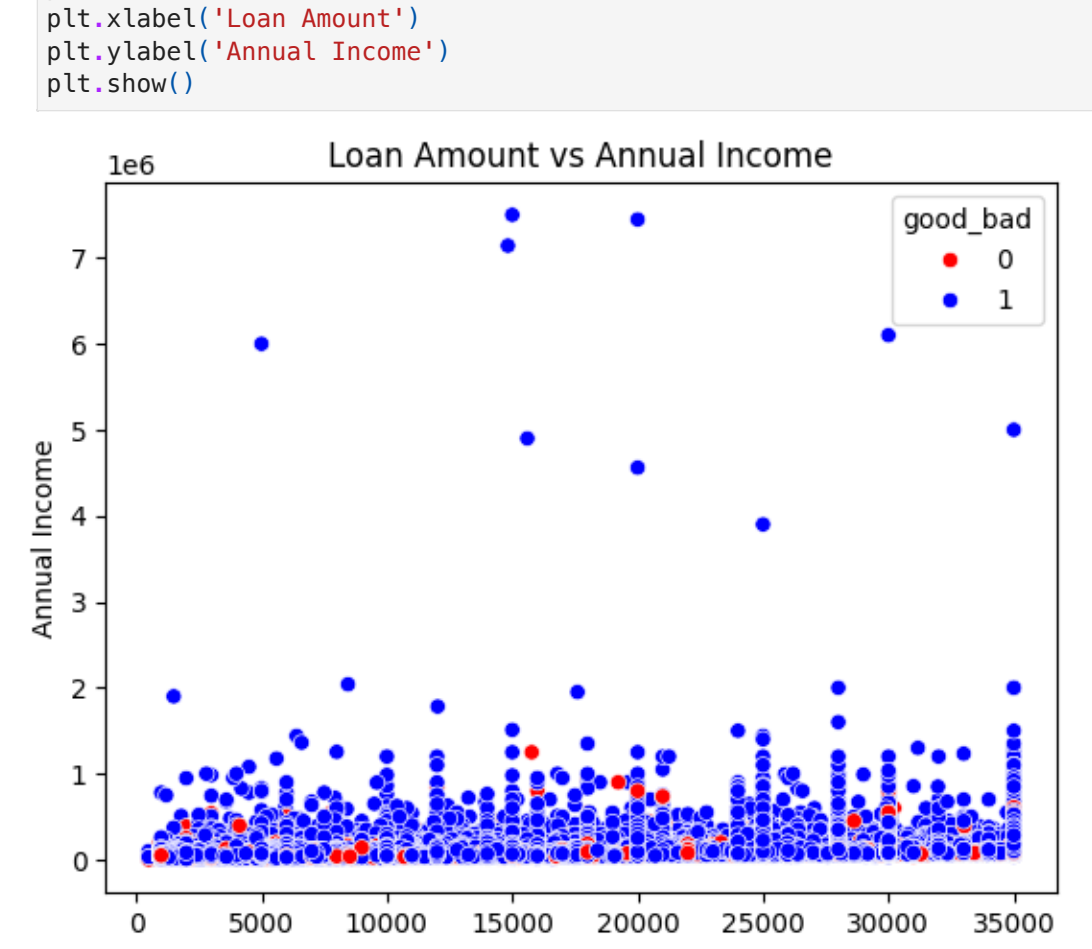
In [17]:
```python
# show the number of loan through purpose
loan_data['purpose'].value_counts()
# plot the distribution of loan purpose as a horizontal bar chart
sns.countplot(y = 'purpose', data = loan_data, order = loan_data['purpose'].value_counts().index)
plt.title('Distribution of Loan Purpose')
plt.xlabel('Count')
plt.ylabel('Loan Purpose')
plt.show()
```



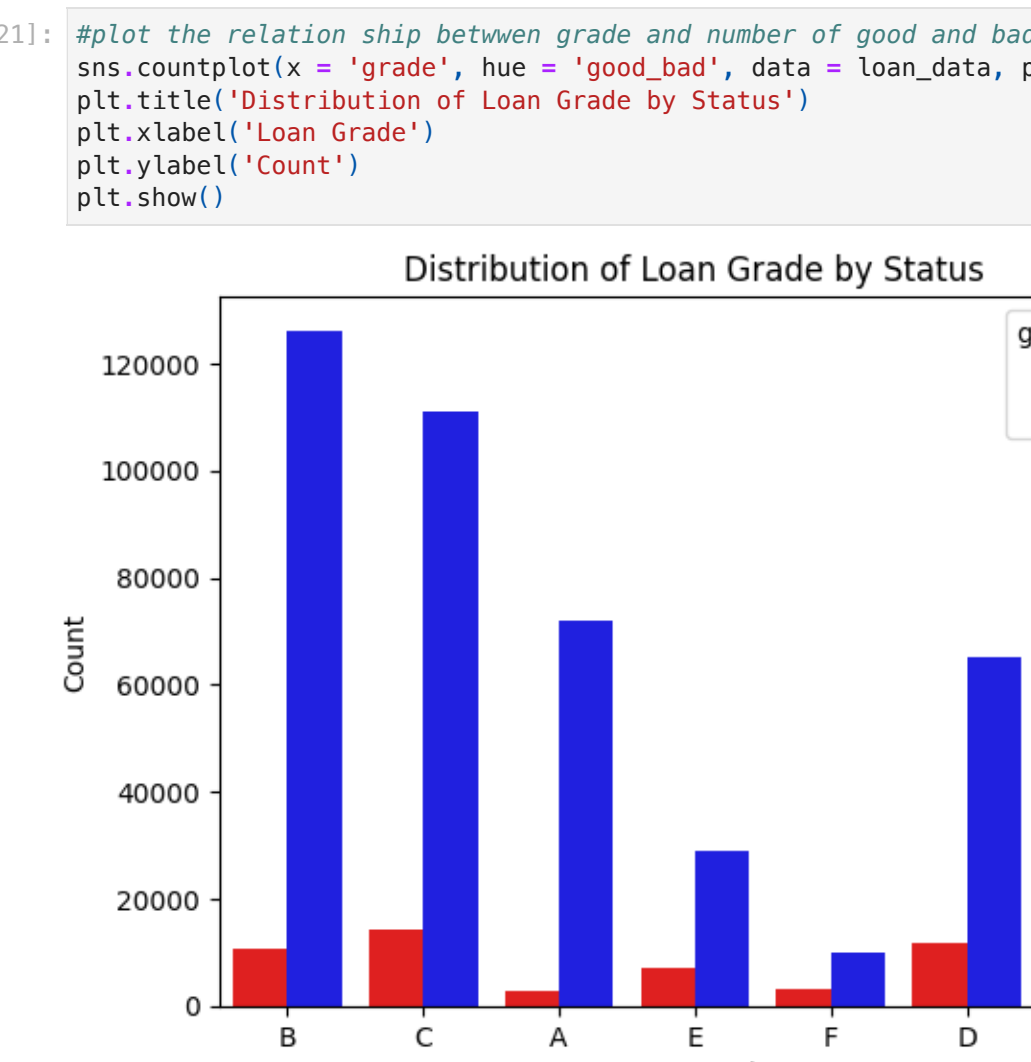
In [18]:
```python
#plot the highest loan amount
sns.histplot(x = 'loan_amnt', data = loan_data, bins = 50)
plt.title('Distribution of Loan Amount')
plt.xlabel('Loan Amount')
plt.ylabel('Count')
plt.show()
```



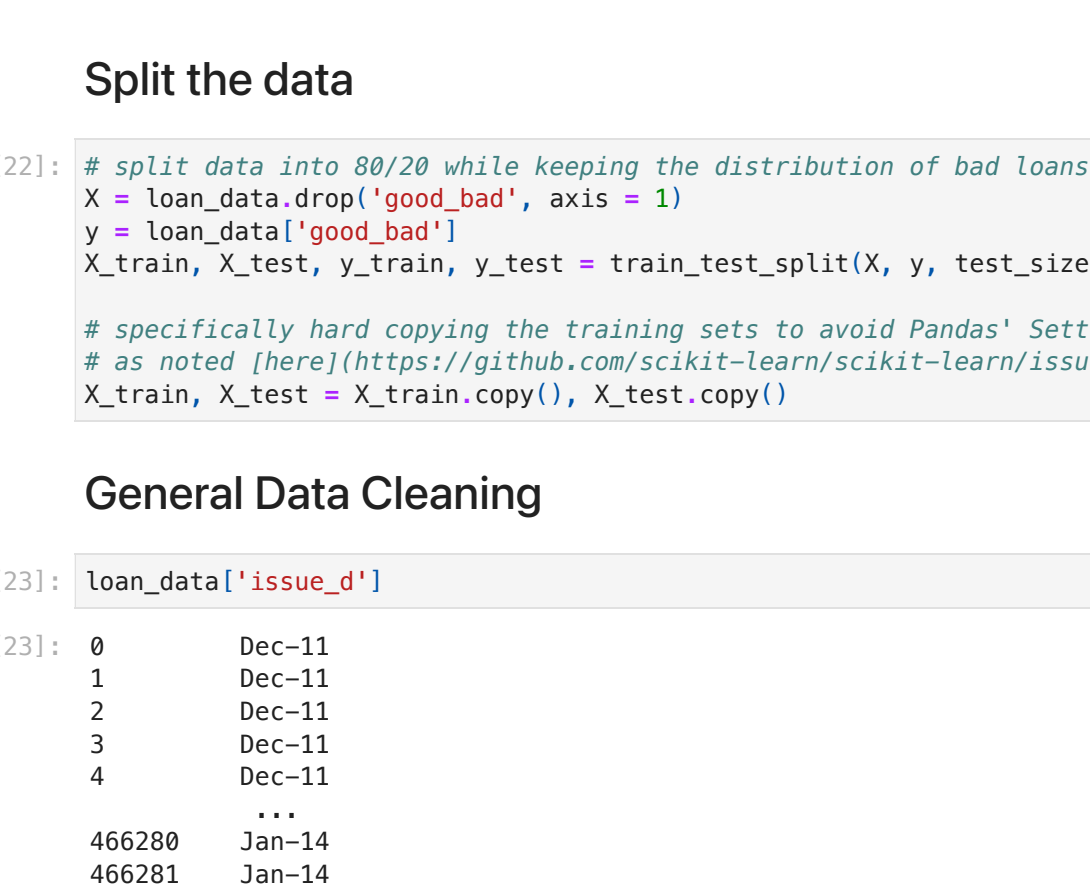
In [19]:
```python
import seaborn as sns
import matplotlib.pyplot as plt

# Using hue to differentiate groups and palette to assign specific colors
# 1 (Good) = Blue, 0 (Bad) = Red
sns.histplot(x='loan_amnt', data=loan_data, bins=50,
             hue='good_bad', palette={1: 'blue', 0: 'red'},
             multiple="stack")  # 'stack' or 'dodge' helps make it more readable

plt.title('Distribution of Loan Amount by Status')
plt.xlabel('Loan Amount')
plt.ylabel('Count')

# Seaborn automatically generates the legend when using the 'hue' parameter
plt.show()
```



In [20]:
```python
#plot the highest loan amount and also make the red color for the bad loans and blue color for the good loans
sns.scatterplot(x = 'loan_amnt', y = 'annual_inc', hue = 'good_bad', data = loan_data, palette = {0: 'red', 1: 'blue'})
plt.title('Loan Amount vs Annual Income')
plt.xlabel('Loan Amount')
plt.ylabel('Annual Income')
plt.show()
```



In [21]:
```python
#plot the relation ship betwwen grade and number of good and bad loans
sns.countplot(x = 'grade', hue = 'good_bad', data = loan_data, palette = {0: 'red', 1: 'blue'})
plt.title('Distribution of Loan Grade by Status')
plt.xlabel('Loan Grade')
plt.ylabel('Count')
plt.show()
```



Split the data

In [22]:
```python
# split data into 80/20 while keeping the distribution of bad loans in test set same as that in the pre-split dataset
X = loan_data.drop('good_bad', axis = 1)
y = loan_data['good_bad']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42, stratify = y)

# specifically hard copying the training sets to avoid Pandas' SetttingWithCopyWarning when we play around with this data later on.
# as noted [here](https://github.com/scikit-learn/scikit-learn/issues/8723), this is currently an open issue between Pandas and Scikit-Learn teams
X_train, X_test = X_train.copy(), X_test.copy()
```

General Data Cleaning

In [23]:
```python
loan_data['issue_d']
```

Out[23]:
```
0         Dec-11
1         Dec-11
2         Dec-11
3         Dec-11
4         Dec-11
           ...  
466280    Jan-14
466281    Jan-14
466282    Jan-14
466283    Jan-14
466284    Jan-14
Name: issue_d, Length: 466285, dtype: str
```

In [24]:
```python
import pandas as pd
import numpy as np
# function to convert date columns to month difference from a reference date (e.g. today)
def date_columns(df, column):
    today_date = pd.to_datetime('2017-12-01')
    # z.B. "Dec-15" -> 2015-12-01
    df[column] = pd.to_datetime(df[column], format="%b-%y", errors="coerce")

    # exact month difference: year * 12 + month
    m_col = 'mths_since_' + column
    df[m_col] = (
        (today_date.year - df[column].dt.year) * 12
        + (today_date.month - df[column].dt.month)
    )

    # replace negatives with max (ignoring NaNs)
    max_val = df[m_col].max()
    df[m_col] = df[m_col].apply(lambda x: max_val if pd.notna(x) and x < 0 else x)

    df.drop(columns=[column], inplace=True)
```

In [25]:
```python
# apply the function to the relevant date columns
date_columns(X_train, 'earliest_cr_line')
date_columns(X_train, 'issue_d')
date_columns(X_train, 'last_pymnt_d')
date_columns(X_train, 'last_credit_pull_d')
```

In [26]:
```python
# check the new columns
print(X_train['mths_since_earliest_cr_line'].describe())
print(X_train['mths_since_issue_d'].describe())
print(X_train['mths_since_last_pymnt_d'].describe())
print(X_train['mths_since_last_credit_pull_d'].describe())
```

```
count    373003.000000
mean        242.293609
std          86.372742
min          73.000000
25%         183.000000
50%         225.000000
75%         285.000000
max         587.000000
Name: mths_since_earliest_cr_line, dtype: float64
count    373028.000000
mean         51.252485
std          14.339074
min          36.000000
25%          41.000000
50%          47.000000
75%          57.000000
max         126.000000
Name: mths_since_issue_d, dtype: float64
count    372717.000000
```

```
mean          31.289989
std           12.803859
min           23.000000
25%           23.000000
50%           24.000000
75%           35.000000
max          120.000000
Name: mths_since_last_pymnt_d, dtype: float64
count    372998.000000
mean         27.041810
std           9.630887
min          23.000000
25%          23.000000
50%          23.000000
75%          25.000000
max         127.000000
Name: mths_since_last_credit_pull_d, dtype: float64
```

In [27]:
```python
loan_data['term']
```

Out[27]:
```
0          36 months
1          60 months
2          36 months
3          36 months
4          60 months
             ...
466280     60 months
466281     60 months
466282     60 months
466283     36 months
466284     36 months
Name: term, Length: 466285, dtype: str
```

In [28]:
```python
# function to remove 'months' string from the 'term' column and convert it to numeric
def loan_term_converter(df, column):
    df[column] = pd.to_numeric(df[column].str.replace(' months', ''))

loan_term_converter(X_train, 'term')
```

In [29]:
```python
X_train["term"].unique()
```

Out[29]:
```
array([36, 60])
```

In [30]:
```python
# first divide training data into categorical and numerical subsets
X_train_cat = X_train.select_dtypes(include = 'object').copy()
X_train_num = X_train.select_dtypes(include = 'number').copy()
```

In [31]:
```python
# define an empty dictionary to store chi-squared test results
chi2_check = {}

# loop over each column in the training set to calculate chi-statistic with the target variable
for column in X_train_cat:
    chi, p, dof, ex = chi2_contingency(pd.crosstab(y_train, X_train_cat[column]))
    chi2_check.setdefault('Feature',[]).append(column)
    chi2_check.setdefault('p-value',[]).append(round(p, 10))

# convert the dictionary to a DF
chi2_result = pd.DataFrame(data = chi2_check)
chi2_result.sort_values(by = ['p-value'], ascending = True, ignore_index = True, inplace = True)
chi2_result
```

Out[31]:

	Feature	p-value
0	grade	0.000000
1	emp_length	0.000000
2	home_ownership	0.000000
3	verification_status	0.000000
4	purpose	0.000000
5	addr_state	0.000000
6	initial_list_status	0.000000
7	pymnt_plan	0.000923
8	application_type	1.000000

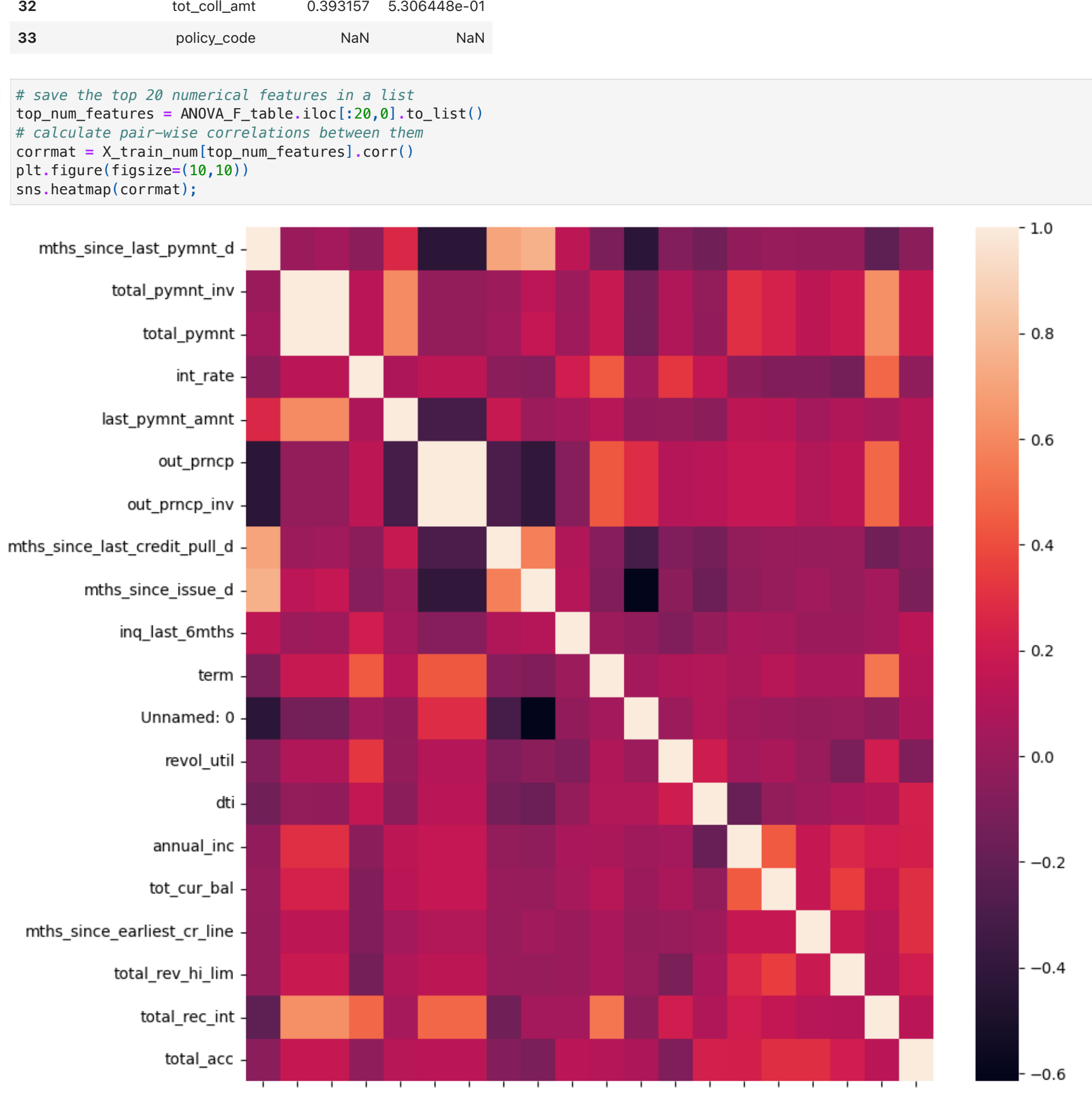
- The Chi-Squared test of independence is used here to see if a categorical feature and the target variable are independent or related.
- Low P-Value (typically < 0.05): This suggests a strong relationship. We should keep this features
- High P-Value (close to 1.0): This suggests the feature is "independent" of the target.

In [32]:
```python
# since f_class_if does not accept missing values, we will do a very crude imputation of missing values
X_train_num.fillna(X_train_num.mean(), inplace = True)
# Calculate F Statistic and corresponding p values
F_statistic, p_values = f_classif(X_train_num, y_train)
# convert to a DF
# anova f test is used to check if there is a significant difference in the means of the numerical features between the two classes of the target variable (good vs bad loans). A
ANOVA_F_table = pd.DataFrame(data = {'Numerical_Feature': X_train_num.columns.values, 'F-Score': F_statistic, 'p values': p_values.round(decimals=10)})
ANOVA_F_table.sort_values(by = ['F-Score'], ascending = False, ignore_index = True, inplace = True)
ANOVA_F_table
```

Out[32]:

	Numerical_Feature	F-Score	p values
0	mths_since_last_pymnt_d	23513.805570	0.000000e+00
1	total_pymnt_inv	14784.534040	0.000000e+00
2	total_pymnt	14686.729567	0.000000e+00
3	int_rate	11462.788313	0.000000e+00
4	last_pymnt_amnt	10826.949727	0.000000e+00
5	out_prncp	9635.116160	0.000000e+00
6	out_prncp_inv	9633.442129	0.000000e+00
7	mths_since_last_credit_pull_d	7020.218888	0.000000e+00
8	mths_since_issue_d	2816.028871	0.000000e+00
9	inq_last_6mths	2003.820465	0.000000e+00
10	term	1590.811890	0.000000e+00
11	Unnamed: 0	1143.897928	0.000000e+00
12	revol_util	931.880533	0.000000e+00
13	dti	863.811228	0.000000e+00
14	annual_inc	861.298449	0.000000e+00
15	tot_cur_bal	784.596617	0.000000e+00
16	mths_since_earliest_cr_line	447.541677	0.000000e+00
17	total_rev_hi_lim	377.070243	0.000000e+00
18	total_rec_int	273.765687	0.000000e+00
19	total_acc	208.751175	0.000000e+00
20	revol_bal	119.561249	0.000000e+00
21	installment	74.947851	0.000000e+00
22	loan_amnt	57.496753	0.000000e+00
23	funded_amnt	50.089615	0.000000e+00
24	pub_rec	39.596857	3.000000e-10
25	funded_amnt_inv	23.260798	1.415100e-06
26	open_acc	16.299684	5.408370e-05
27	collections_12_mths_ex_med	7.358687	6.674257e-03
28	mths_since_last_major_derog	3.116419	7.750777e-02
29	mths_since_last_delinq	3.079668	7.927751e-02
30	delinq_2yrs	1.066138	3.018201e-01
31	acc_now_delinq	0.690407	4.060264e-01
32	tot_coll_amt	0.393157	5.306448e-01
33	policy_code	NaN	NaN

In [33]:
```python
# save the top 20 numerical features in a list
top_num_features = ANOVA_F_table.iloc[:20,0].to_list()
# calculate pair-wise correlations between them
corrmat = X_train_num[top_num_features].corr()
plt.figure(figsize=(10,10))
sns.heatmap(corrmat);
```



Dummy Variables

Convert discrete variables to dummy variables

In [34]:
```python
X_train.info()
```

```
<class 'pandas.DataFrame'>
Index: 373028 entries, 456615 to 38736
Data columns (total 43 columns):
 #   Column                         Non-Null Count   Dtype
---  ------                         --------------   -----
 0   Unnamed: 0                     373028 non-null  int64
 1   loan_amnt                      373028 non-null  int64
 2   funded_amnt                    373028 non-null  int64
 3   funded_amnt_inv                373028 non-null  float64
 4   term                           373028 non-null  int64
 5   int_rate                       373028 non-null  float64
 6   installment                    373028 non-null  float64
 7   grade                          373028 non-null  str
 8   emp_length                     356235 non-null  str
 9   home_ownership                 373028 non-null  str
 10  annual_inc                     373024 non-null  float64
 11  verification_status            373028 non-null  str
 12  pymnt_plan                     373028 non-null  str
 13  purpose                        373028 non-null  str
 14  addr_state                     373028 non-null  str
 15  dti                            373028 non-null  float64
 16  delinq_2yrs                    373003 non-null  float64
 17  inq_last_6mths                 373003 non-null  float64
 18  mths_since_last_delinq         172651 non-null  float64
 19  open_acc                       373003 non-null  float64
 20  pub_rec                        373003 non-null  float64
 21  revol_bal                      373028 non-null  int64
 22  revol_util                     372764 non-null  float64
 23  total_acc                      373003 non-null  float64
 24  initial_list_status            373028 non-null  str
 25  out_prncp                      373028 non-null  float64
 26  out_prncp_inv                  373028 non-null  float64
 27  total_pymnt                    373028 non-null  float64
 28  total_pymnt_inv                373028 non-null  float64
 29  total_rec_int                  373028 non-null  float64
 30  last_pymnt_amnt                373028 non-null  float64
 31  collections_12_mths_ex_med     372906 non-null  float64
 32  mths_since_last_major_derog    79250 non-null   float64
 33  policy_code                    373028 non-null  int64
 34  application_type               373028 non-null  str
 35  acc_now_delinq                 373003 non-null  float64
 36  tot_coll_amt                   316872 non-null  float64
 37  tot_cur_bal                    316872 non-null  float64
 38  total_rev_hi_lim               316872 non-null  float64
 39  mths_since_earliest_cr_line    373003 non-null  float64
 40  mths_since_issue_d             373028 non-null  int64
 41  mths_since_last_pymnt_d        372717 non-null  float64
 42  mths_since_last_credit_pull_d  372998 non-null  float64
dtypes: float64(27), int64(7), str(9)
memory usage: 125.2 MB
```

In [35]:
```python
import pandas as pd
# function to convert employment length to numeric
def emp_length_converter(df, column):
    if column not in df.columns:
        return  # nothing to do

    s = df[column].fillna("0 years")

    df[column + "_int"] = (
        s.str.replace(r"\+ years", "", regex=True)   # "10+ years" -> "10"
         .str.replace("years", "", regex=False)
         .str.replace("year", "", regex=False)
         .str.replace("< 1", "0", regex=False)       # "< 1 year" -> "0"
         .str.replace("n/a", "0", regex=False)
         .str.strip()
    )

    df[column + "_int"] = (
        pd.to_numeric(df[column + "_int"], errors="coerce")
          .astype("Int64")
    )
    df["emp_length"] = df["emp_length_int"]
# Anwendung
emp_length_converter(X_train, "emp_length")
```

In [36]:
```python
# function to create dummy variables
def dummy_creation(df, columns_list):
    # create an empty list to store the dummy variable dataframes
    df_dummies = []
    # loop through the list of columns and create dummy variables for each column, then concatenate them together
    for col in columns_list:
        #get_dummies() is a function in pandas that converts categorical variables into dummy/indicator variables. The prefix and prefix_sep arguments are used to create more in
        df_dummies.append(pd.get_dummies(df[col], prefix = col, prefix_sep = ':'))
    # concatenate the dummy variables together axis = 1 means we want to concatenate horizontally (i.e. add more columns)
    df_dummies = pd.concat(df_dummies, axis = 1)
    # concatenate the dummy variables with the original dataframe
    df = pd.concat([df, df_dummies], axis = 1)
    return df

# apply to our final four categorical variables
X_train = dummy_creation(X_train, ['grade', 'home_ownership', 'verification_status', 'purpose'])
```

In [37]:
```python
X_train
```

Out[37]:

	Unnamed: 0	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	emp_length	home_ownership	annual_inc	verification_status	pymnt_plan	purpose	addr_
456615	456615	15000	15000	15000.0	36	8.90	476.30	A	7	MORTGAGE	80000.0	Source Verified	n	credit_card	
451541	451541	8000	8000	8000.0	60	18.25	204.24	D	10	OWN	44000.0	Verified	n	other	
394474	394474	12150	12150	12100.0	60	18.92	314.65	D	3	OWN	27000.0	Source Verified	n	credit_card	
110294	110294	10000	10000	10000.0	36	6.03	304.36	A	3	MORTGAGE	33000.0	Not Verified	n	debt_consolidation	
139343	139343	15825	15825	15825.0	36	12.12	526.53	B	10	MORTGAGE	59000.0	Verified	n	debt_consolidation	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
163633	163633	22800	22800	22800.0	60	16.29	557.98	C	10	MORTGAGE	85000.0	Verified	n	debt_consolidation	
399306	399306	20275	20275	20175.0	36	12.49	678.18	B	7	MORTGAGE	48000.0	Verified	n	credit_card	
57787	57787	25000	25000	24975.0	60	14.30	585.61	C	0	MORTGAGE	65530.0	Verified	n	debt_consolidation	
81641	81641	10000	10000	10000.0	36	15.10	347.15	C	4	MORTGAGE	50000.0	Not Verified	n	debt_consolidation	
38736	38736	7500	7500	0.0	36	10.08	242.29	B	7	MORTGAGE	50000.0	Not Verified	n	debt_consolidation	

373028 rows × 73 columns

Update the test data set with all data cleaning procedures performed so far

In [38]:
```python
X_test
```

Out[38]:

	Unnamed: 0	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	emp_length	home_ownership	annual_inc	verification_status	issue_d	pymnt_plan	pur
395346	395346	1800	1800	1800.0	36 months	14.64	62.09	C	10+ years	OWN	50000.0	Source Verified	May-14	n	home_improve
376583	376583	6000	6000	6000.0	36 months	7.12	185.60	A	1 year	RENT	63000.0	Not Verified	Jun-14	n	debt_consolic
297790	297790	10700	10700	10650.0	36 months	14.99	370.87	C	< 1 year	RENT	82000.0	Source Verified	Oct-14	n	debt_consolic
47347	47347	15000	15000	15000.0	36 months	9.67	481.69	B	2 years	RENT	60000.0	Verified	Dec-13	n	debt_consolic
446772	446772	14000	14000	13925.0	60 months	18.25	357.42	D	10+ years	MORTGAGE	86000.0	Source Verified	Feb-14	n	debt_consolic
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
32147	32147	2500	2500	2500.0	36 months	6.39	76.51	A	10+ years	OWN	33600.0	Not Verified	Feb-10	n	me
78936	78936	10400	10400	10400.0	36 months	19.52	383.97	D	6 years	RENT	42000.0	Not Verified	Oct-13	n	debt_consolic
459582	459582	3000	3000	3000.0	36 months	23.40	116.76	E	8 years	MORTGAGE	30000.0	Not Verified	Jan-14	n	
454915	454915	24000	24000	24000.0	60 months	15.61	578.68	C	5 years	MORTGAGE	80000.0	Verified	Jan-14	n	debt_consolic
223621	223621	23325	23325	23300.0	36 months	13.11	787.15	B	7 years	RENT	47000.0	Verified	Mar-12	n	credit_

93257 rows × 43 columns

In [39]:
```python
print(X_test.columns.tolist())
'earliest_cr_line' in X_test.columns  # should be False
```

```
['Unnamed: 0', 'loan_amnt', 'funded_amnt', 'funded_amnt_inv', 'term', 'int_rate', 'installment', 'grade', 'emp_length', 'home_ownership', 'annual_inc', 'verification_status', 'is
sue_d', 'pymnt_plan', 'purpose', 'addr_state', 'dti', 'delinq_2yrs', 'earliest_cr_line', 'inq_last_6mths', 'mths_since_last_delinq', 'open_acc', 'pub_rec', 'revol_bal', 'revol_ut
il', 'total_acc', 'initial_list_status', 'out_prncp', 'out_prncp_inv', 'total_pymnt', 'total_pymnt_inv', 'total_rec_int', 'last_pymnt_d', 'last_pymnt_amnt', 'last_credit_pull_d',
'collections_12_mths_ex_med', 'mths_since_last_major_derog', 'policy_code', 'application_type', 'acc_now_delinq', 'tot_coll_amt', 'tot_cur_bal', 'total_rev_hi_lim']
```

Out[39]:
```
True
```

In [40]:
```python
X_train
```

Out[40]:

	Unnamed: 0	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	emp_length	home_ownership	annual_inc	verification_status	pymnt_plan	purpose	addr_
456615	456615	15000	15000	15000.0	36	8.90	476.30	A	7	MORTGAGE	80000.0	Source Verified	n	credit_card	
451541	451541	8000	8000	8000.0	60	18.25	204.24	D	10	OWN	44000.0	Verified	n	other	
394474	394474	12150	12150	12100.0	60	18.92	314.65	D	3	OWN	27000.0	Source Verified	n	credit_card	
110294	110294	10000	10000	10000.0	36	6.03	304.36	A	3	MORTGAGE	33000.0	Not Verified	n	debt_consolidation	
139343	139343	15825	15825	15825.0	36	12.12	526.53	B	10	MORTGAGE	59000.0	Verified	n	debt_consolidation	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
163633	163633	22800	22800	22800.0	60	16.29	557.98	C	10	MORTGAGE	85000.0	Verified	n	debt_consolidation	
399306	399306	20275	20275	20175.0	36	12.49	678.18	B	7	MORTGAGE	48000.0	Verified	n	credit_card	
57787	57787	25000	25000	24975.0	60	14.30	585.61	C	0	MORTGAGE	65530.0	Verified	n	debt_consolidation	
81641	81641	10000	10000	10000.0	36	15.10	347.15	C	4	MORTGAGE	50000.0	Not Verified	n	debt_consolidation	
38736	38736	7500	7500	0.0	36	10.08	242.29	B	7	MORTGAGE	50000.0	Not Verified	n	debt_consolidation	

373028 rows × 73 columns

In [41]:
```python
X_test
```

Out[41]:

	Unnamed: 0	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	emp_length	home_ownership	annual_inc	verification_status	issue_d	pymnt_plan	pur
395346	395346	1800	1800	1800.0	36 months	14.64	62.09	C	10+ years	OWN	50000.0	Source Verified	May-14	n	home_improve
376583	376583	6000	6000	6000.0	36 months	7.12	185.60	A	1 year	RENT	63000.0	Not Verified	Jun-14	n	debt_consolic
297790	297790	10700	10700	10650.0	36 months	14.99	370.87	C	< 1 year	RENT	82000.0	Source Verified	Oct-14	n	debt_consolic
47347	47347	15000	15000	15000.0	36 months	9.67	481.69	B	2 years	RENT	60000.0	Verified	Dec-13	n	debt_consolic
446772	446772	14000	14000	13925.0	60 months	18.25	357.42	D	10+ years	MORTGAGE	86000.0	Source Verified	Feb-14	n	debt_consolic
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
32147	32147	2500	2500	2500.0	36 months	6.39	76.51	A	10+ years	OWN	33600.0	Not Verified	Feb-10	n	me
78936	78936	10400	10400	10400.0	36 months	19.52	383.97	D	6 years	RENT	42000.0	Not Verified	Oct-13	n	debt_consolic
459582	459582	3000	3000	3000.0	36 months	23.40	116.76	E	8 years	MORTGAGE	30000.0	Not Verified	Jan-14	n	
454915	454915	24000	24000	24000.0	60 months	15.61	578.68	C	5 years	MORTGAGE	80000.0	Verified	Jan-14	n	debt_consolic
223621	223621	23325	23325	23300.0	36 months	13.11	787.15	B	7 years	RENT	47000.0	Verified	Mar-12	n	credit_

93257 rows × 43 columns

In [42]:
```python
# assuming all converter functions modify df in-place

emp_length_converter(X_test, 'emp_length')


date_columns(X_test, 'earliest_cr_line')
date_columns(X_test, 'issue_d')
date_columns(X_test, 'last_pymnt_d')
date_columns(X_test, 'last_credit_pull_d')


loan_term_converter(X_test, 'term')



X_test = dummy_creation(
    X_test,
    ['grade', 'home_ownership', 'verification_status', 'purpose']
)

# align test to train feature set
X_test = X_test.reindex(labels=X_train.columns, axis=1, fill_value=0)
```

WoE Binning/Feature Engineering

Weight of Evidence (WoE) binning is a powerful feature engineering technique, primarily used in credit scoring and financial modeling, to transform continuous or categorical variables into a format that has a linear relationship with the log-odds of a target variable.

In simpler terms, it helps you understand how much "evidence" a specific category or range of values provides in predicting an outcome (like whether someone will default on a loan).

In [ ]:

In [43]:
```python
# Create copies of the 4 training sets to be preprocessed using WoE
X_train_prepr = X_train.copy()
y_train_prepr = y_train.copy()
X_test_prepr = X_test.copy()
y_test_prepr = y_test.copy()
```

In [44]:
```python
# The function takes 3 arguments: a dataframe (X_train_prepr), a string (column name), and a dataframe (y_train_prepr).
# The function returns a dataframe as a result.
# The function first concatenates the specified column from the input dataframe with the target variable dataframe along the columns axis.
# Then, it calculates the count and mean of the target variable for each category in the specified column and concatenates these two results together.
# Next, it renames the columns of the resulting dataframe and calculates several new columns based on the count and mean values. Finally, it sorts the dataframe by the WoE colum

def woe_discrete(df, cat_variabe_name, y_df):
    df = pd.concat([df[cat_variabe_name], y_df], axis = 1)
    df = pd.concat([df.groupby(df.columns.values[0], as_index = False)[df.columns.values[1]].count(),
                    df.groupby(df.columns.values[0], as_index = False)[df.columns.values[1]].mean()], axis = 1)
    df = df.iloc[:, [0, 1, 3]]
    df.columns = [df.columns.values[0], 'n_obs', 'prop_good']
    df['prop_n_obs'] = df['n_obs'] / df['n_obs'].sum()
    df['n_good'] = df['prop_good'] * df['n_obs']
    df['n_bad'] = (1 - df['prop_good']) * df['n_obs']
    df['prop_n_good'] = df['n_good'] / df['n_good'].sum()
    df['prop_n_bad'] = df['n_bad'] / df['n_bad'].sum()
    df['WoE'] = np.log(df['prop_n_good'] / df['prop_n_bad'])
    df = df.sort_values(['WoE'])
    df = df.reset_index(drop = True)
    df['diff_prop_good'] = df['prop_good'].diff().abs()
    df['diff_WoE'] = df['WoE'].diff().abs()
    df['IV'] = (df['prop_n_good'] - df['prop_n_bad']) * df['WoE']
    df['IV'] = df['IV'].sum()
    return df
```

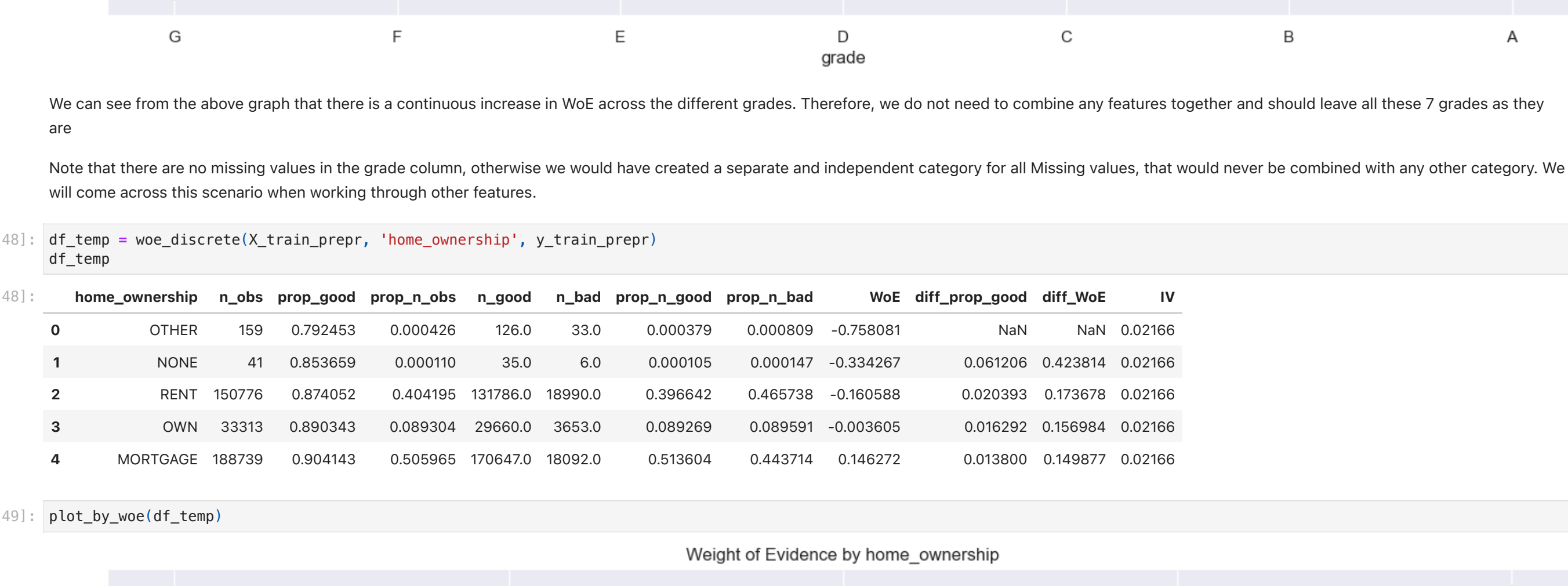
In [45]:
```python
# We set the default style of the graphs to the seaborn style.
sns.set()
# Below we define a function for plotting WoE across categories that takes 2 arguments: a dataframe and a number.
def plot_by_woe(df_WoE, rotation_of_x_axis_labels = 0):
    x = np.array(df_WoE.iloc[:, 0].apply(str))
    y = df_WoE['WoE']
    plt.figure(figsize=(18, 6))
    plt.plot(x, y, marker = 'o', linestyle = '--', color = 'k')
    plt.xlabel(df_WoE.columns[0])
    plt.ylabel('Weight of Evidence')
    plt.title(str('Weight of Evidence by ' + df_WoE.columns[0]))
    plt.xticks(rotation = rotation_of_x_axis_labels)
```

In [46]:
```python
df_temp = woe_discrete(X_train_prepr, 'grade', y_train_prepr)
df_temp
```

Out[46]:

	grade	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	G	2623	0.721693	0.007032	1893.0	730.0	0.005697	0.017904	-1.144981	NaN	NaN	0.292145
1	F	10606	0.758061	0.028432	8040.0	2566.0	0.024198	0.062932	-0.955774	0.036369	0.189207	0.292145
2	E	28590	0.804477	0.076643	23000.0	5590.0	0.069224	0.137097	-0.683340	0.046416	0.272434	0.292145
3	D	61713	0.845527	0.165438	52180.0	9533.0	0.157049	0.233801	-0.397915	0.041050	0.285425	0.292145
4	C	100342	0.885870	0.268993	88890.0	11452.0	0.267536	0.280865	-0.048620	0.040343	0.349295	0.292145
5	B	109344	0.921422	0.293125	100752.0	8592.0	0.303238	0.210723	0.363975	0.035552	0.412595	0.292145
6	A	59810	0.961361	0.160336	57499.0	2311.0	0.173057	0.056678	1.116232	0.039939	0.752257	0.292145

In [47]:
```python
plot_by_woe(df_temp)
```



We can see from the above graph that there is a continuous increase in WoE across the different grades. Therefore, we do not need to combine any features together and should leave all these 7 grades as they are

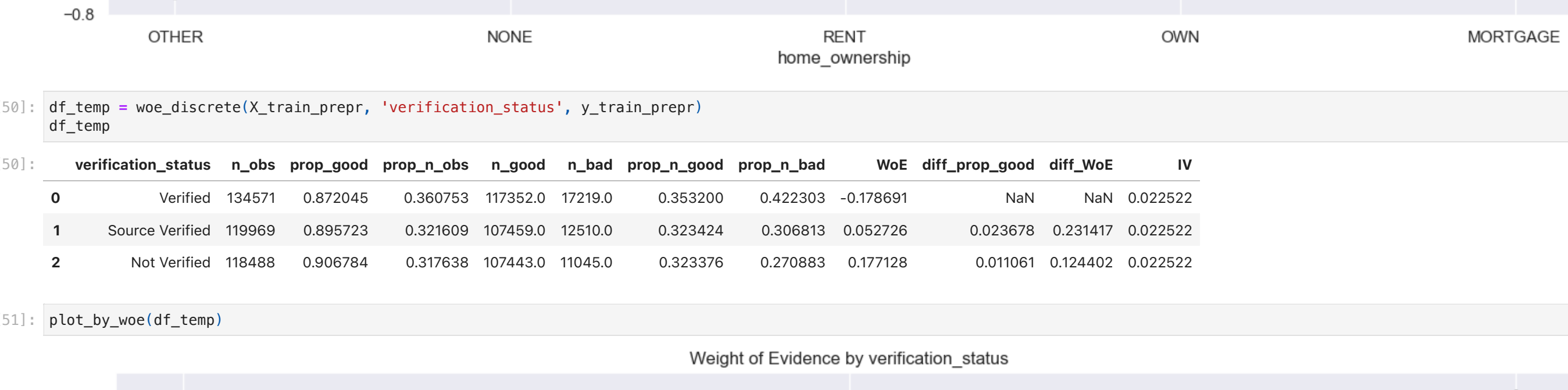
Note that there are no missing values in the grade column, otherwise we would have created a separate and independent category for all Missing values, that would never be combined with any other category. We will come across this scenario when working through other features.

In [48]:
```python
df_temp = woe_discrete(X_train_prepr, 'home_ownership', y_train_prepr)
df_temp
```

Out[48]:

	home_ownership	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	OTHER	159	0.792453	0.000426	126.0	33.0	0.000379	0.000809	-0.758081	NaN	NaN	0.02166
1	NONE	41	0.853659	0.000110	35.0	6.0	0.000105	0.000147	-0.334267	0.061206	0.423814	0.02166
2	RENT	150776	0.874052	0.404195	131786.0	18990.0	0.396642	0.465738	-0.160588	0.020393	0.173678	0.02166
3	OWN	33313	0.890343	0.089304	29660.0	3653.0	0.089269	0.089591	-0.003605	0.016292	0.156984	0.02166
4	MORTGAGE	188739	0.904143	0.505965	170647.0	18092.0	0.513604	0.443714	0.146272	0.013800	0.149877	0.02166

In [49]:
```python
plot_by_woe(df_temp)
```

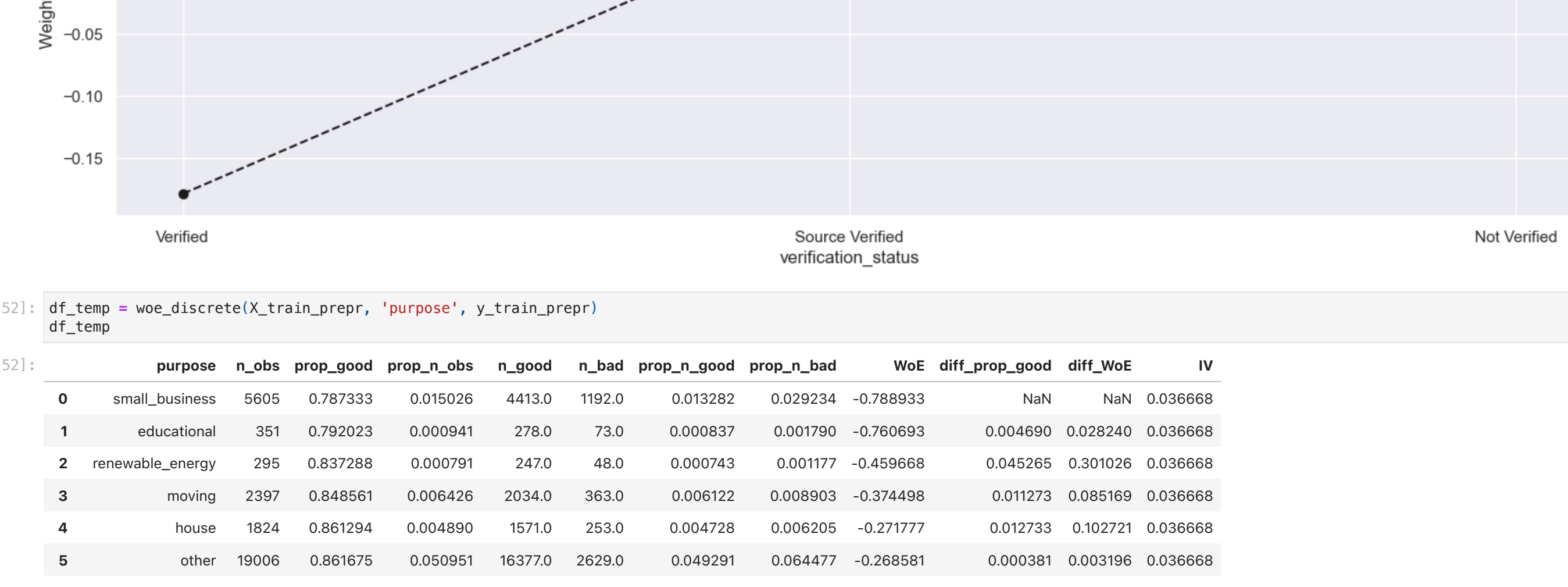


In [50]:
```python
df_temp = woe_discrete(X_train_prepr, 'verification_status', y_train_prepr)
df_temp
```

Out[50]:

	verification_status	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	Verified	134571	0.872045	0.360753	117352.0	17219.0	0.353200	0.422303	-0.178691	NaN	NaN	0.022522
1	Source Verified	119969	0.895723	0.321609	107459.0	12510.0	0.323424	0.306813	0.052726	0.023678	0.231417	0.022522
2	Not Verified	118488	0.906784	0.317638	107443.0	11045.0	0.323376	0.270883	0.177128	0.011061	0.124402	0.022522

In [51]:
```python
plot_by_woe(df_temp)
```

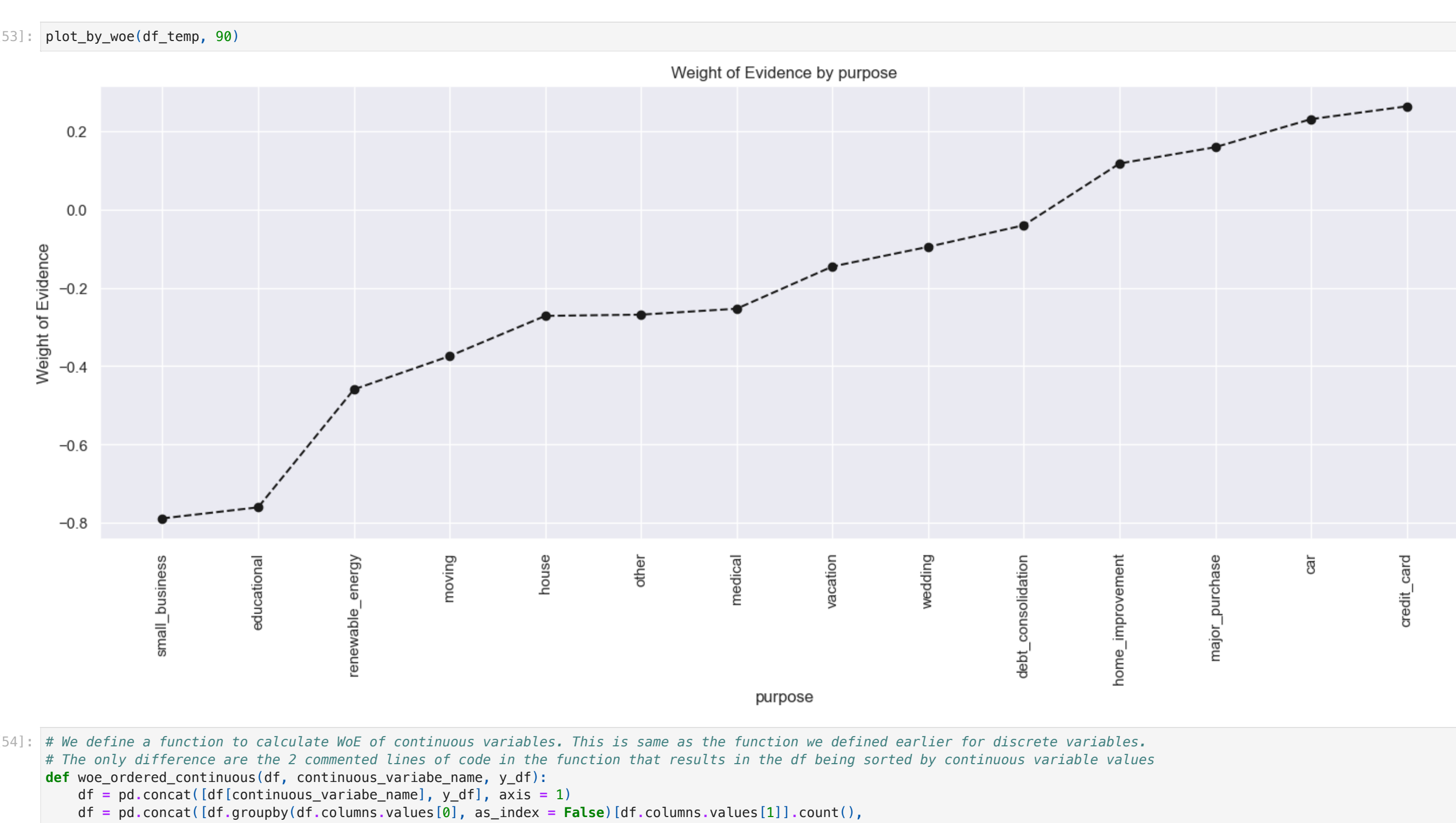


In [52]:
```python
df_temp = woe_discrete(X_train_prepr, 'purpose', y_train_prepr)
df_temp
```

Out[52]:

	purpose	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	small_business	5605	0.787333	0.015026	4413.0	1192.0	0.013282	0.029234	-0.788933	NaN	NaN	0.036668
1	educational	351	0.792023	0.000941	278.0	73.0	0.000837	0.001790	-0.760693	0.004690	0.028240	0.036668
2	renewable_energy	295	0.837288	0.000791	247.0	48.0	0.000743	0.001177	-0.459668	0.045265	0.301026	0.036668
3	moving	2397	0.848561	0.006426	2034.0	363.0	0.006122	0.008903	-0.374498	0.011273	0.085169	0.036668
4	house	1824	0.861294	0.004890	1571.0	253.0	0.004728	0.006205	-0.271777	0.012733	0.102721	0.036668
5	other	19006	0.861675	0.050951	16377.0	2629.0	0.049291	0.064477	-0.268581	0.000381	0.003196	0.036668
6	medical	3750	0.863467	0.010053	3238.0	512.0	0.009746	0.012557	-0.253469	0.001791	0.015112	0.036668
7	vacation	2003	0.875686	0.005370	1754.0	249.0	0.005279	0.006107	-0.145654	0.012220	0.107815	0.036668
8	wedding	1900	0.881053	0.005093	1674.0	226.0	0.005038	0.005543	-0.095419	0.005366	0.050235	0.036668
9	debt_consolidation	219196	0.886704	0.587613	194362.0	24834.0	0.584980	0.609065	-0.040346	0.005652	0.055072	0.036668
10	home_improvement	21253	0.901614	0.056974	19162.0	2091.0	0.057673	0.051283	0.117432	0.014910	0.157778	0.036668
11	major_purchase	7873	0.905246	0.021106	7127.0	746.0	0.021450	0.018296	0.159065	0.003632	0.041633	0.036668
12	car	4246	0.911211	0.011383	3869.0	377.0	0.011645	0.009246	0.230651	0.005965	0.071586	0.036668
13	credit_card	83329	0.913824	0.223385	76148.0	7181.0	0.229186	0.176117	0.263385	0.002613	0.032734	0.036668

In [53]:
```python
plot_by_woe(df_temp, 90)
```



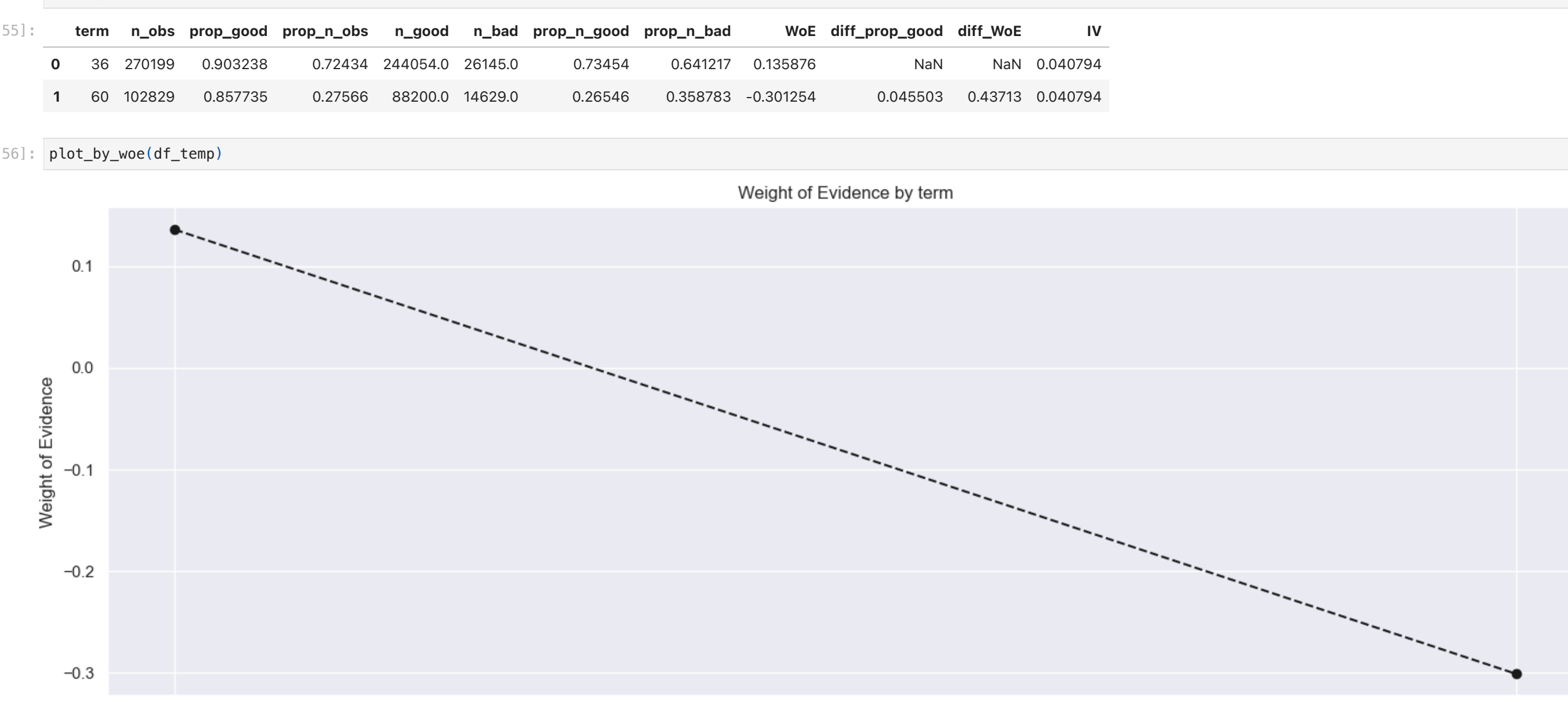
In [54]:
```python
# We define a function to calculate WoE of continuous variables. This is same as the function we defined earlier for discrete variables.
# The only difference are the 2 commented lines of code in the function that results in the df being sorted by continuous variable values
def woe_ordered_continuous(df, continuous_variabe_name, y_df):
    df = pd.concat([df[continuous_variabe_name], y_df], axis = 1)
    df = pd.concat([df.groupby(df.columns.values[0], as_index = False)[df.columns.values[1]].count(),
                    df.groupby(df.columns.values[0], as_index = False)[df.columns.values[1]].mean()], axis = 1)
    df = df.iloc[:, [0, 1, 3]]
    df.columns = [df.columns.values[0], 'n_obs', 'prop_good']
    df['prop_n_obs'] = df['n_obs'] / df['n_obs'].sum()
    df['n_good'] = df['prop_good'] * df['n_obs']
    df['n_bad'] = (1 - df['prop_good']) * df['n_obs']
    df['prop_n_good'] = df['n_good'] / df['n_good'].sum()
    df['prop_n_bad'] = df['n_bad'] / df['n_bad'].sum()
    df['WoE'] = np.log(df['prop_n_good'] / df['prop_n_bad'])
    #df = df.sort_values(['WoE'])
    #df = df.reset_index(drop = True)
    df['diff_prop_good'] = df['prop_good'].diff().abs()
    df['diff_WoE'] = df['WoE'].diff().abs()
    df['IV'] = (df['prop_n_good'] - df['prop_n_bad']) * df['WoE']
    df['IV'] = df['IV'].sum()
    return df
```

In [55]:
```python
df_temp = woe_ordered_continuous(X_train_prepr, 'term', y_train_prepr)
df_temp
```

Out[55]:

	term	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	36	270199	0.903238	0.72434	244054.0	26145.0	0.73454	0.641217	0.135876	NaN	NaN	0.040794
1	60	102829	0.857735	0.27566	88200.0	14629.0	0.26546	0.358783	-0.301254	0.045503	0.43713	0.040794

In [56]:
```python
plot_by_woe(df_temp)
```



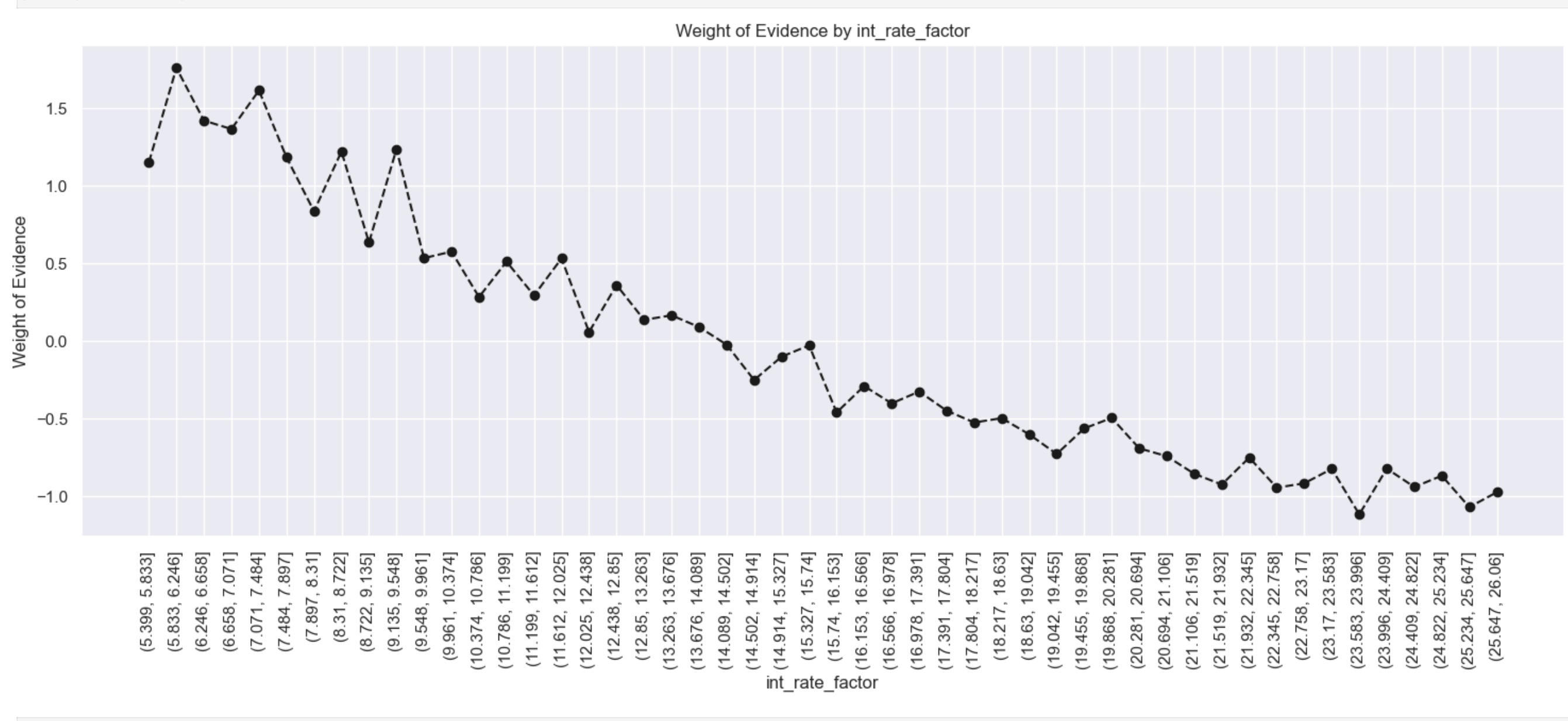
In [57]:
```python
# fine-classing using the 'cut' method, given the large number of unique values
X_train_prepr['int_rate_factor'] = pd.cut(X_train_prepr['int_rate'], 50)
# Process 'int_rate_factor' column through woe_ordered_continuous and plot_by_woe functions
df_temp = woe_ordered_continuous(X_train_prepr, 'int_rate_factor', y_train_prepr)
df_temp
```

Out[57]:

	int_rate_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(5.399, 5.833]	776	0.962629	0.002080	747.0	29.0	0.002248	0.000711	1.150914	NaN	NaN	0.346603
1	(5.833, 6.246]	8408	0.979305	0.022540	8234.0	174.0	0.024782	0.004267	1.759117	0.016677	0.608203	0.346603
2	(6.246, 6.658]	7842	0.971181	0.021023	7616.0	226.0	0.022922	0.005543	1.419616	0.008125	0.339500	0.346603
3	(6.658, 7.071]	1777	0.969612	0.004764	1723.0	54.0	0.005186	0.001324	1.364983	0.001569	0.054633	0.346603
4	(7.071, 7.484]	3186	0.976146	0.008541	3110.0	76.0	0.009360	0.001864	1.613790	0.006534	0.248806	0.346603

5	(7.484, 7.897]	12590	0.963781	0.033751	12134.0	456.0	0.036520	0.011184	1.183419	0.012365	0.430371	0.346603
6	(7.897, 8.31]	9680	0.949483	0.025950	9191.0	489.0	0.027663	0.011993	0.835762	0.014297	0.347656	0.346603
7	(8.31, 8.722]	6666	0.965047	0.017870	6433.0	233.0	0.019362	0.005714	1.220303	0.015563	0.384540	0.346603
8	(8.722, 9.135]	9578	0.938923	0.025676	8993.0	585.0	0.027067	0.014347	0.634735	0.026124	0.585568	0.346603
9	(9.135, 9.548]	6563	0.965565	0.017594	6337.0	226.0	0.019073	0.005543	1.235771	0.026642	0.601036	0.346603
10	(9.548, 9.961]	7597	0.932868	0.020366	7087.0	510.0	0.021330	0.012508	0.533752	0.032696	0.702019	0.346603
11	(9.961, 10.374]	10937	0.935448	0.029320	10231.0	706.0	0.030793	0.017315	0.575707	0.002580	0.041956	0.346603
12	(10.374, 10.786]	5777	0.915354	0.015487	5288.0	489.0	0.015916	0.011993	0.282978	0.020094	0.292729	0.346603
13	(10.786, 11.199]	17733	0.931427	0.047538	16517.0	1216.0	0.049712	0.029823	0.510968	0.016073	0.227990	0.346603
14	(11.199, 11.612]	4819	0.916165	0.012919	4415.0	404.0	0.013288	0.009908	0.293493	0.015262	0.217475	0.346603
15	(11.612, 12.025]	15494	0.932748	0.041536	14452.0	1042.0	0.043497	0.025556	0.531836	0.016583	0.238343	0.346603
16	(12.025, 12.438]	12356	0.896164	0.033124	11073.0	1283.0	0.033327	0.031466	0.057454	0.036584	0.474382	0.346603
17	(12.438, 12.85]	11008	0.921148	0.029510	10140.0	868.0	0.030519	0.021288	0.360196	0.024984	0.302743	0.346603
18	(12.85, 13.263]	21451	0.903268	0.057505	19376.0	2075.0	0.058317	0.050890	0.136219	0.017880	0.223978	0.346603
19	(13.263, 13.676]	15377	0.905768	0.041222	13928.0	1449.0	0.041920	0.035537	0.165172	0.002500	0.028954	0.346603
20	(13.676, 14.089]	10984	0.899126	0.029446	9876.0	1108.0	0.029724	0.027174	0.089696	0.006642	0.075477	0.346603
21	(14.089, 14.502]	23313	0.888388	0.062497	20711.0	2602.0	0.062335	0.063815	-0.023471	0.010738	0.113166	0.346603
22	(14.502, 14.914]	4431	0.863688	0.011878	3827.0	604.0	0.011518	0.014813	-0.251593	0.024701	0.228122	0.346603
23	(14.914, 15.327]	19020	0.880442	0.050988	16746.0	2274.0	0.050401	0.055771	-0.101236	0.016754	0.150357	0.346603
24	(15.327, 15.74]	12381	0.887893	0.033191	10993.0	1388.0	0.033086	0.034041	-0.028460	0.007451	0.072776	0.346603
25	(15.74, 16.153]	7475	0.837191	0.020039	6258.0	1217.0	0.018835	0.029847	-0.460383	0.050702	0.431923	0.346603
26	(16.153, 16.566]	13068	0.859045	0.035032	11226.0	1842.0	0.033787	0.045176	-0.290475	0.021854	0.169909	0.346603
27	(16.566, 16.978]	4257	0.844961	0.011412	3597.0	660.0	0.010826	0.016187	-0.402240	0.014084	0.111765	0.346603
28	(16.978, 17.391]	11418	0.854703	0.030609	9759.0	1659.0	0.029372	0.040688	-0.325880	0.009742	0.076359	0.346603
29	(17.391, 17.804]	12339	0.838723	0.033078	10349.0	1990.0	0.031148	0.048806	-0.449100	0.015980	0.123220	0.346603
30	(17.804, 18.217]	1257	0.828162	0.003370	1041.0	216.0	0.003133	0.005297	-0.525196	0.010560	0.076097	0.346603
31	(18.217, 18.63]	10959	0.832101	0.029378	9119.0	1840.0	0.027446	0.045127	-0.497261	0.003939	0.027936	0.346603
32	(18.63, 19.042]	6930	0.817027	0.018578	5662.0	1268.0	0.017041	0.031098	-0.601519	0.015074	0.104258	0.346603
33	(19.042, 19.455]	4470	0.797315	0.011983	3564.0	906.0	0.010727	0.022220	-0.728256	0.019712	0.126737	0.346603
34	(19.455, 19.868]	6744	0.822657	0.018079	5548.0	1196.0	0.016698	0.029332	-0.563400	0.025342	0.164855	0.346603
35	(19.868, 20.281]	4914	0.832316	0.013173	4090.0	824.0	0.012310	0.020209	-0.495725	0.009659	0.067675	0.346603
36	(20.281, 20.694]	3236	0.803152	0.008675	2599.0	637.0	0.007822	0.015623	-0.691743	0.029164	0.196017	0.346603
37	(20.694, 21.106]	3972	0.795065	0.010648	3158.0	814.0	0.009505	0.019964	-0.742121	0.008087	0.050379	0.346603
38	(21.106, 21.519]	2763	0.775968	0.007407	2144.0	619.0	0.006453	0.015181	-0.855532	0.019097	0.113411	0.346603
39	(21.519, 21.932]	969	0.763674	0.002598	740.0	229.0	0.002227	0.005616	-0.924927	0.012294	0.069395	0.346603
40	(21.932, 22.345]	3444	0.793554	0.009233	2733.0	711.0	0.008226	0.017438	-0.751372	0.029880	0.173555	0.346603
41	(22.345, 22.758]	2292	0.760035	0.006144	1742.0	550.0	0.005243	0.013489	-0.944984	0.033519	0.193612	0.346603
42	(22.758, 23.17]	1490	0.765101	0.003994	1140.0	350.0	0.003431	0.008584	-0.917005	0.005066	0.027980	0.346603
43	(23.17, 23.583]	2681	0.781425	0.007187	2095.0	586.0	0.006305	0.014372	-0.823866	0.016324	0.093139	0.346603
44	(23.583, 23.996]	1595	0.727273	0.004276	1160.0	435.0	0.003491	0.010669	-1.117026	0.054152	0.293160	0.346603
45	(23.996, 24.409]	1452	0.781680	0.003892	1135.0	317.0	0.003416	0.007775	-0.822369	0.054408	0.294657	0.346603
46	(24.409, 24.822]	1660	0.760843	0.004450	1263.0	397.0	0.003801	0.009737	-0.940546	0.020837	0.118177	0.346603
47	(24.822, 25.234]	1344	0.773810	0.003603	1040.0	304.0	0.003130	0.007456	-0.867907	0.012966	0.072639	0.346603
48	(25.234, 25.647]	774	0.736434	0.002075	570.0	204.0	0.001716	0.005003	-1.070339	0.037375	0.202432	0.346603
49	(25.647, 26.06]	1781	0.754632	0.004774	1344.0	437.0	0.004045	0.010718	-0.974383	0.018198	0.095956	0.346603

In [58]:
```
plot_by_woe(df_temp, 90)
```

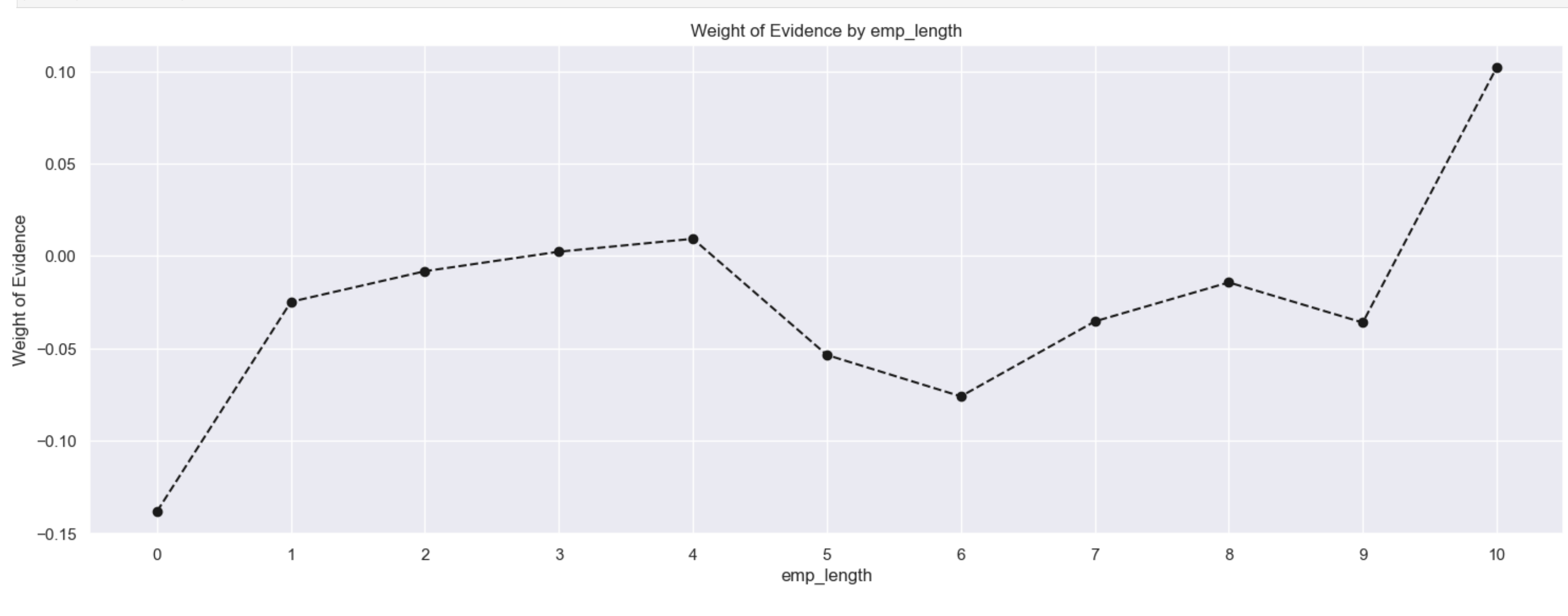

Weight of Evidence by int_rate_factor

In [59]:
```
df_temp = woe_ordered_continuous(X_train_prepr, 'emp_length', y_train_prepr)
df_temp
```

Out[59]:

	emp_length	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	0	45764	0.876497	0.122682	40112.0	5652.0	0.120727	0.138618	-0.138189	NaN	NaN	0.006408
1	1	23582	0.888262	0.063218	20947.0	2635.0	0.063045	0.064625	-0.024743	0.011765	0.113446	0.006408
2	2	33123	0.889895	0.088795	29476.0	3647.0	0.088715	0.089444	-0.008184	0.001633	0.016559	0.006408
3	3	29301	0.890925	0.078549	26105.0	3196.0	0.078569	0.078383	0.002372	0.001030	0.010555	0.006408
4	4	22482	0.891602	0.060269	20045.0	2437.0	0.060330	0.059768	0.009357	0.000677	0.006985	0.006408
5	5	24654	0.885374	0.066092	21828.0	2826.0	0.065697	0.069309	-0.053524	0.006229	0.062881	0.006408
6	6	21057	0.883079	0.056449	18595.0	2462.0	0.055966	0.060382	-0.075936	0.002294	0.022413	0.006408
7	7	20834	0.887204	0.055851	18484.0	2350.0	0.055632	0.057635	-0.035365	0.004124	0.040571	0.006408
8	8	17859	0.889300	0.047876	15882.0	1977.0	0.047801	0.048487	-0.014249	0.002096	0.021116	0.006408
9	9	14372	0.887142	0.038528	12750.0	1622.0	0.038374	0.039780	-0.035984	0.002158	0.021735	0.006408
10	10	120000	0.900250	0.321692	108030.0	11970.0	0.325143	0.293569	0.102150	0.013108	0.138134	0.006408

In [60]:
```
plot_by_woe(df_temp)
```


Weight of Evidence by emp_length

We will not use this feature for the following reasons:

very low IV WoE ranges between a very small range, implying low power of differentiating between good and bad loans

In [61]:
```
# fine-classing
X_train_prepr['annual_inc_factor'] = pd.cut(X_train_prepr['annual_inc'], 50)
# Process 'annual_inc_factor' column through woe_ordered_continuous and plot_by_woe functions
df_temp = woe_ordered_continuous(X_train_prepr, 'annual_inc_factor', y_train_prepr)
df_temp
```

Out[61]:

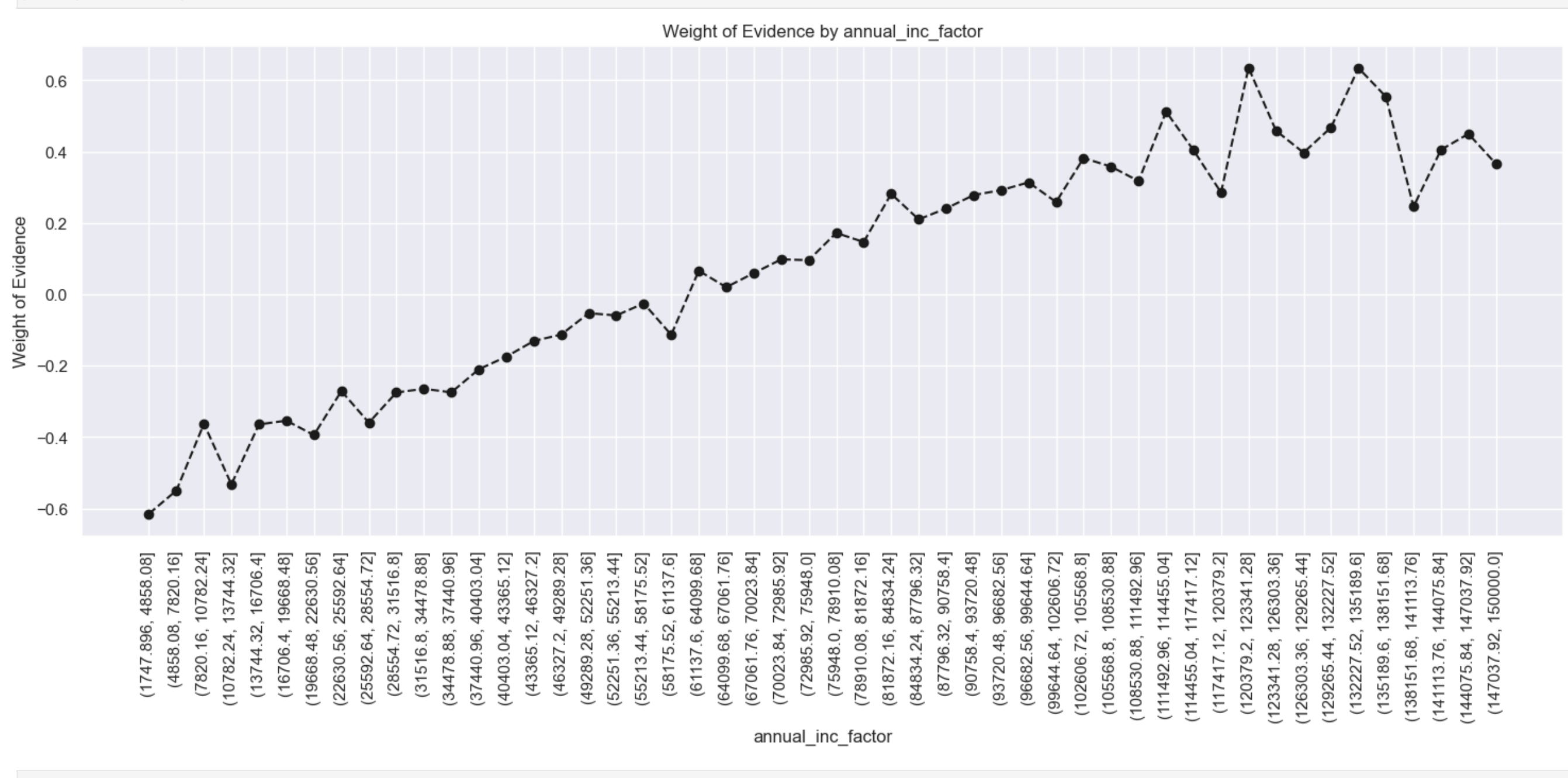
	annual_inc_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-5602.104, 151858.08]	356629	0.889078	0.956048	317071.0	39558.0	0.954315	0.970177	-0.016485	NaN	NaN	inf
1	(151858.08, 301820.16]	14796	0.925047	0.039665	13687.0	1109.0	0.041195	0.027199	0.415145	0.035969	0.431630	inf
2	(301820.16, 451782.24]	1076	0.930297	0.002885	1001.0	75.0	0.003013	0.001839	0.493424	0.005250	0.078279	inf
3	(451782.24, 601744.32]	291	0.941581	0.000780	274.0	17.0	0.000825	0.000417	0.682072	0.011283	0.188648	inf
4	(601744.32, 751706.4]	96	0.906250	0.000257	87.0	9.0	0.000262	0.000221	0.170840	0.035331	0.511231	inf
5	(751706.4, 901668.48]	59	0.932203	0.000158	55.0	4.0	0.000166	0.000098	0.523196	0.025953	0.352355	inf
6	(901668.48, 1051630.56]	30	0.966667	0.000080	29.0	1.0	0.000087	0.000025	1.269453	0.034463	0.746257	inf
7	(1051630.56, 1201592.64]	19	1.000000	0.000051	19.0	0.0	0.000057	0.000000	inf	0.033333	inf	inf
8	(1201592.64, 1351554.72]	7	0.857143	0.000019	6.0	1.0	0.000018	0.000025	-0.306084	0.142857	inf	inf
9	(1351554.72, 1501516.8]	5	1.000000	0.000013	5.0	0.0	0.000015	0.000000	inf	0.142857	inf	inf
10	(1501516.8, 1651478.88]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
11	(1651478.88, 1801440.96]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
12	(1801440.96, 1951403.04]	2	1.000000	0.000005	2.0	0.0	0.000006	0.000000	inf	0.000000	NaN	inf
13	(1951403.04, 2101365.12]	4	1.000000	0.000011	4.0	0.0	0.000012	0.000000	inf	0.000000	NaN	inf
14	(3750948.0, 3900910.08]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
15	(4800682.56, 4950644.64]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
16	(4950644.64, 5100606.72]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
17	(5850417.12, 6000379.2]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
18	(6000379.2, 6150341.28]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
19	(7050113.76, 7200075.84]	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
20	(7350037.92, 7500000.0]	2	1.000000	0.000005	2.0	0.0	0.000006	0.000000	inf	0.000000	NaN	inf

In [62]:
```
# Initial examination shows that there are too few individuals with large income (up to $75m!!) and too many with small income.
# Since 95.6% of observations have income < 151,858, we will have one category for more than 150K, and we are going to apply our approach to determine
# the categories of everyone with 150k or less.
X_train_prepr_temp = X_train_prepr[X_train_prepr['annual_inc'] <= 150000].copy()
# fine-classing again
X_train_prepr_temp['annual_inc_factor'] = pd.cut(X_train_prepr_temp['annual_inc'], 50)
# make sure to select only the relevant indexes in the target column
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'annual_inc_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[62]:

	annual_inc_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(1747.896, 4858.08]	16	0.812500	0.000045	13.0	3.0	0.000041	0.000076	-0.614889	NaN	NaN	0.04906
1	(4858.08, 7820.16]	45	0.822222	0.000126	37.0	8.0	0.000117	0.000202	-0.549749	0.009722	0.065139	0.04906
2	(7820.16, 10782.24]	342	0.847953	0.000959	290.0	52.0	0.000915	0.001315	-0.362589	0.025731	0.187161	0.04906
3	(10782.24, 13744.32]	696	0.824713	0.001952	574.0	122.0	0.001811	0.003085	-0.532617	0.023241	0.170029	0.04906
4	(13744.32, 16706.4]	1392	0.847701	0.003905	1180.0	212.0	0.003723	0.005361	-0.364542	0.022989	0.168075	0.04906
5	(16706.4, 19668.48]	1908	0.849057	0.005352	1620.0	288.0	0.005111	0.007282	-0.354005	0.001355	0.010538	0.04906
6	(19668.48, 22630.56]	3592	0.843820	0.010076	3031.0	561.0	0.009563	0.014185	-0.394299	0.005237	0.040294	0.04906
7	(22630.56, 25592.64]	6450	0.859380	0.018093	5543.0	907.0	0.017489	0.022934	-0.271077	0.015560	0.123222	0.04906
8	(25592.64, 28554.72]	5729	0.848316	0.016070	4860.0	869.0	0.015334	0.021973	-0.359775	0.011064	0.088698	0.04906
9	(28554.72, 31516.8]	10353	0.858881	0.029041	8892.0	1461.0	0.028055	0.036942	-0.275195	0.010566	0.084580	0.04906
10	(31516.8, 34478.88]	8645	0.860150	0.024250	7436.0	1209.0	0.023461	0.030570	-0.264686	0.001269	0.010509	0.04906
11	(34478.88, 37440.96]	14199	0.858934	0.039829	12196.0	2003.0	0.038479	0.050647	-0.274764	0.001217	0.010078	0.04906
12	(37440.96, 40403.04]	18216	0.866601	0.051097	15786.0	2430.0	0.049806	0.061444	-0.209994	0.007667	0.064770	0.04906
13	(40403.04, 43365.12]	12637	0.870618	0.035448	11002.0	1635.0	0.034712	0.041342	-0.174791	0.004017	0.035202	0.04906
14	(43365.12, 46327.2]	16547	0.875506	0.046416	14487.0	2060.0	0.045708	0.052089	-0.130680	0.004888	0.044111	0.04906
15	(46327.2, 49289.28]	11670	0.877464	0.032735	10240.0	1430.0	0.032308	0.036159	-0.112599	0.001957	0.018081	0.04906
16	(49289.28, 52251.36]	22582	0.883757	0.063344	19957.0	2625.0	0.062966	0.066375	-0.052727	0.006293	0.059872	0.04906
17	(52251.36, 55213.44]	16885	0.883092	0.047364	14911.0	1974.0	0.047045	0.049914	-0.059188	0.000665	0.006462	0.04906
18	(55213.44, 58175.52]	10355	0.886528	0.029047	9180.0	1175.0	0.028964	0.029711	-0.025467	0.003437	0.033722	0.04906
19	(58175.52, 61137.6]	18974	0.877358	0.053223	16647.0	2327.0	0.052523	0.058840	-0.113576	0.009170	0.088109	0.04906
20	(61137.6, 64099.68]	9342	0.895419	0.026205	8365.0	977.0	0.026392	0.024704	0.066099	0.018060	0.179675	0.04906
21	(64099.68, 67061.76]	15930	0.891023	0.044685	14194.0	1736.0	0.044783	0.043896	0.020010	0.004395	0.046089	0.04906
22	(67061.76, 70023.84]	16119	0.894783	0.045215	14423.0	1696.0	0.045506	0.042885	0.059326	0.003759	0.039316	0.04906
23	(70023.84, 72985.92]	7035	0.898365	0.019734	6320.0	715.0	0.019940	0.018079	0.097966	0.003583	0.038640	0.04906
24	(72985.92, 75948.0]	13561	0.898164	0.038040	12180.0	1381.0	0.038429	0.034920	0.095762	0.000201	0.002205	0.04906
25	(75948.0, 78910.08]	6217	0.904938	0.017439	5626.0	591.0	0.017750	0.014944	0.172112	0.006774	0.076351	0.04906
26	(78910.08, 81872.16]	12387	0.902721	0.034746	11182.0	1205.0	0.035280	0.030469	0.146600	0.002217	0.025512	0.04906
27	(81872.16, 84834.24]	6089	0.913943	0.017080	5565.0	524.0	0.017558	0.013250	0.281535	0.011223	0.134935	0.04906
28	(84834.24, 87796.32]	10038	0.908149	0.028157	9116.0	922.0	0.028762	0.023313	0.210015	0.005794	0.071519	0.04906
29	(87796.32, 90758.4]	10367	0.910678	0.029080	9441.0	926.0	0.029787	0.023415	0.240717	0.002529	0.030702	0.04906
30	(90758.4, 93720.48]	4541	0.913675	0.012738	4149.0	392.0	0.013090	0.009912	0.278135	0.002997	0.037418	0.04906
31	(93720.48, 96682.56]	7016	0.914766	0.019680	6418.0	598.0	0.020249	0.015121	0.292045	0.001091	0.013910	0.04906
32	(96682.56, 99644.64]	3734	0.916443	0.010474	3422.0	312.0	0.010797	0.007889	0.313752	0.001677	0.021706	0.04906
33	(99644.64, 102606.72]	9332	0.912130	0.026177	8512.0	820.0	0.026856	0.020734	0.258702	0.004313	0.055049	0.04906
34	(102606.72, 105568.8]	4753	0.921523	0.013333	4380.0	373.0	0.013819	0.009432	0.382000	0.009393	0.123298	0.04906
35	(105568.8, 108530.88]	2667	0.919760	0.007481	2453.0	214.0	0.007739	0.005411	0.357865	0.001763	0.024135	0.04906
36	(108530.88, 111492.96]	5596	0.916726	0.015697	5130.0	466.0	0.016186	0.011783	0.317450	0.003034	0.040416	0.04906
37	(111492.96, 114455.04]	1725	0.930435	0.004839	1605.0	120.0	0.005064	0.003034	0.512162	0.013709	0.194712	0.04906
38	(114455.04, 117417.12]	3224	0.923077	0.009044	2976.0	248.0	0.009390	0.006271	0.403681	0.007358	0.108481	0.04906
39	(117417.12, 120379.2]	6296	0.914231	0.017661	5756.0	540.0	0.018161	0.013654	0.285203	0.008846	0.118478	0.04906
40	(120379.2, 123341.28]	1048	0.937977	0.002940	983.0	65.0	0.003101	0.001644	0.634996	0.023746	0.349793	0.04906
41	(123341.28, 126303.36]	3556	0.926884	0.009975	3296.0	260.0	0.010399	0.006574	0.458558	0.011093	0.176439	0.04906
42	(126303.36, 129265.44]	969	0.922601	0.002718	894.0	75.0	0.002821	0.001896	0.396992	0.004284	0.061566	0.04906
43	(129265.44, 132227.52]	3200	0.927500	0.008976	2968.0	232.0	0.009364	0.005866	0.467680	0.004899	0.070689	0.04906
44	(132227.52, 135189.6]	1869	0.937935	0.005243	1753.0	116.0	0.005531	0.002933	0.634268	0.010435	0.166587	0.04906
45	(135189.6, 138151.68]	747	0.933066	0.002095	697.0	50.0	0.002199	0.001264	0.553537	0.004869	0.080731	0.04906
46	(138151.68, 141113.76]	2440	0.911066	0.006844	2223.0	217.0	0.007014	0.005487	0.245490	0.022000	0.308047	0.04906
47	(141113.76, 144075.84]	716	0.923184	0.002008	661.0	55.0	0.002086	0.001391	0.405195	0.012119	0.159705	0.04906
48	(144075.84, 147037.92]	1248	0.926282	0.003501	1156.0	92.0	0.003647	0.002326	0.449707	0.003098	0.044512	0.04906
49	(147037.92, 150000.0]	3502	0.920331	0.009823	3223.0	279.0	0.010169	0.007055	0.365630	0.005951	0.084076	0.04906

In [63]:
```
plot_by_woe(df_temp, 90)
```

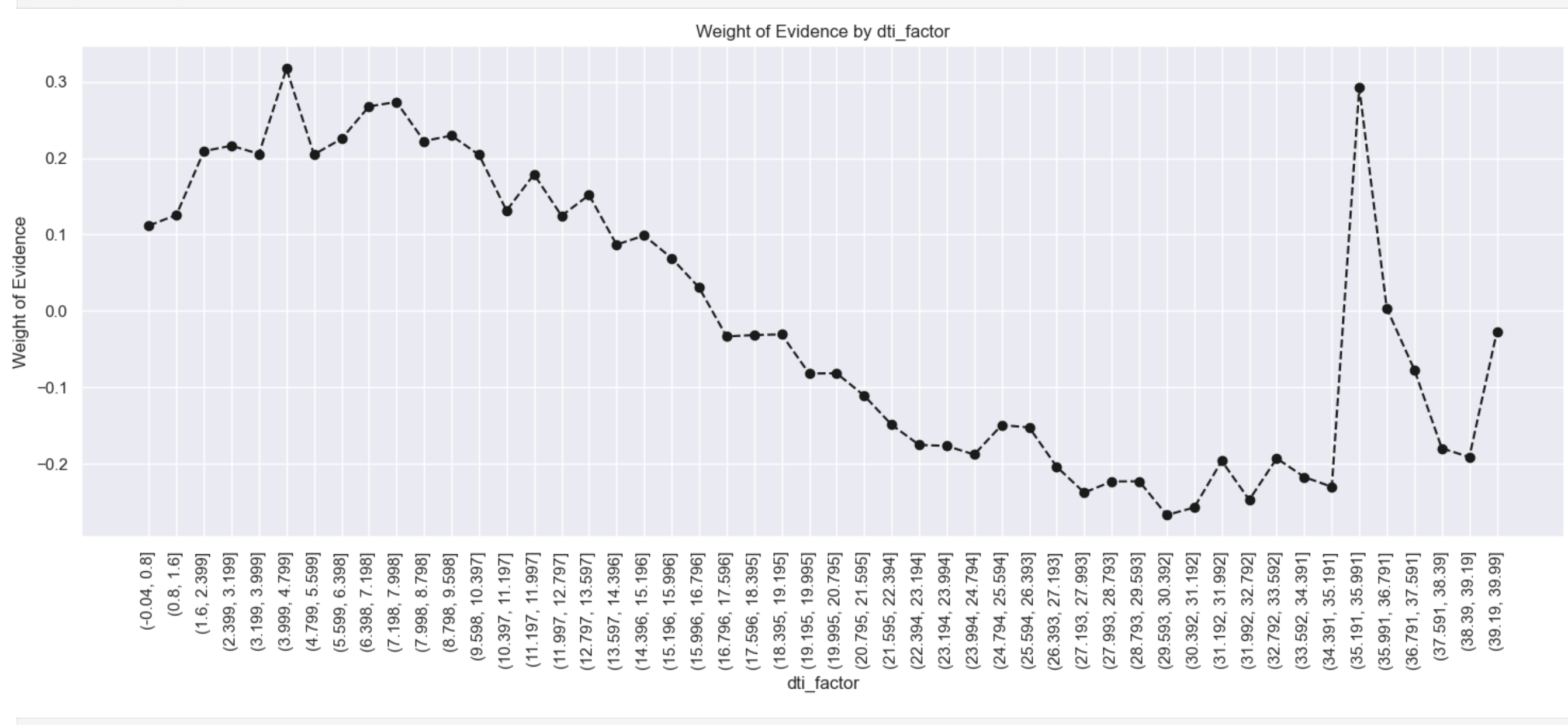

Weight of Evidence by annual_inc_factor

In [64]:
```
# fine-classing
X_train_prepr['dti_factor'] = pd.cut(X_train_prepr['dti'], 50)
# Process 'annual_inc_factor' column through woe_ordered_continuous and plot_by_woe functions
df_temp = woe_ordered_continuous(X_train_prepr, 'dti_factor', y_train_prepr)
df_temp
```

Out[64]:

	dti_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-0.04, 0.8]	1537	0.901106	0.004120	1385.0	152.0	0.004168	0.003728	0.111720	NaN	NaN	0.027353
1	(0.8, 1.6]	1966	0.902340	0.005270	1774.0	192.0	0.005339	0.004709	0.125642	0.001234	0.013922	0.027353
2	(1.6, 2.399]	2507	0.909454	0.006721	2280.0	227.0	0.006862	0.005567	0.209126	0.007114	0.083484	0.027353
3	(2.399, 3.199]	3257	0.910040	0.008731	2964.0	293.0	0.008921	0.007186	0.216267	0.000586	0.007142	0.027353
4	(3.199, 3.999]	4095	0.909158	0.010978	3723.0	372.0	0.011205	0.009123	0.205536	0.000882	0.010731	0.027353
5	(3.999, 4.799]	5059	0.917968	0.013562	4644.0	415.0	0.013977	0.010178	0.317198	0.008810	0.111662	0.027353
6	(4.799, 5.599]	6017	0.909091	0.016130	5470.0	547.0	0.016463	0.013415	0.204730	0.008877	0.112468	0.027353
7	(5.599, 6.398]	6825	0.910769	0.018296	6216.0	609.0	0.018709	0.014936	0.225209	0.001678	0.020479	0.027353
8	(6.398, 7.198]	7712	0.914160	0.020674	7050.0	662.0	0.021219	0.016236	0.267662	0.003391	0.042454	0.027353
9	(7.198, 7.998]	8679	0.914622	0.023266	7938.0	741.0	0.023891	0.018173	0.273561	0.000462	0.005899	0.027353
10	(7.998, 8.798]	9501	0.910536	0.025470	8651.0	850.0	0.026037	0.020847	0.222339	0.004086	0.051222	0.027353
11	(8.798, 9.598]	10373	0.911115	0.027808	9451.0	922.0	0.028445	0.022612	0.229475	0.000580	0.007137	0.027353
12	(9.598, 10.397]	11150	0.909058	0.029891	10136.0	1014.0	0.030507	0.024869	0.204335	0.002057	0.025140	0.027353
13	(10.397, 11.197]	11959	0.902835	0.032059	10797.0	1162.0	0.032496	0.028499	0.131271	0.006224	0.073065	0.027353
14	(11.197, 11.997]	12495	0.906923	0.033496	11332.0	1163.0	0.034106	0.028523	0.178773	0.004088	0.047502	0.027353
15	(11.997, 12.797]	13216	0.902240	0.035429	11924.0	1292.0	0.035888	0.031687	0.124507	0.004683	0.054266	0.027353
16	(12.797, 13.597]	13408	0.904609	0.035944	12129.0	1279.0	0.036505	0.031368	0.151666	0.002369	0.027159	0.027353
17	(13.597, 14.396]	13693	0.898853	0.036708	12308.0	1385.0	0.037044	0.033968	0.086694	0.005756	0.064971	0.027353
18	(14.396, 15.196]	14031	0.899936	0.037614	12627.0	1404.0	0.038004	0.034434	0.098657	0.001082	0.011963	0.027353
19	(15.196, 15.996]	13898	0.897251	0.037257	12470.0	1428.0	0.037532	0.035022	0.069196	0.002684	0.029461	0.027353
20	(15.996, 16.796]	13687	0.893622	0.036692	12231.0	1456.0	0.036812	0.035709	0.030426	0.003630	0.038770	0.027353
21	(16.796, 17.596]	13773	0.887388	0.036922	12222.0	1551.0	0.036785	0.038039	-0.033517	0.006233	0.063943	0.027353
22	(17.596, 18.395]	13636	0.887577	0.036555	12103.0	1533.0	0.036427	0.037597	-0.031628	0.000189	0.001889	0.027353
23	(18.395, 19.195]	13175	0.887666	0.035319	11695.0	1480.0	0.035199	0.036298	-0.030736	0.000089	0.000893	0.027353
24	(19.195, 19.995]	13092	0.882447	0.035097	11553.0	1539.0	0.034772	0.037745	-0.082043	0.005219	0.051307	0.027353
25	(19.995, 20.795]	12492	0.882485	0.033488	11024.0	1468.0	0.033179	0.036003	-0.081681	0.000037	0.000361	0.027353
26	(20.795, 21.595]	11901	0.879422	0.031904	10466.0	1435.0	0.031500	0.035194	-0.110888	0.003063	0.029207	0.027353
27	(21.595, 22.394]	11276	0.875310	0.030228	9870.0	1406.0	0.029706	0.034483	-0.149104	0.004112	0.038216	0.027353
28	(22.394, 23.194]	10886	0.872405	0.029183	9497.0	1389.0	0.028584	0.034066	-0.175463	0.002905	0.026359	0.027353
29	(23.194, 23.994]	10216	0.872259	0.027387	8911.0	1305.0	0.026820	0.032006	-0.176772	0.000146	0.001308	0.027353
30	(23.994, 24.794]	9798	0.870994	0.026266	8534.0	1264.0	0.025685	0.031000	-0.188078	0.001265	0.011307	0.027353
31	(24.794, 25.594]	8546	0.875263	0.022910	7480.0	1066.0	0.022513	0.026144	-0.149536	0.004269	0.038543	0.027353
32	(25.594, 26.393]	7828	0.874936	0.020985	6849.0	979.0	0.020614	0.024010	-0.152529	0.000327	0.002993	0.027353
33	(26.393, 27.193]	7293	0.869190	0.019551	6339.0	954.0	0.019079	0.023397	-0.204042	0.005746	0.051514	0.027353
34	(27.193, 27.993]	6733	0.865290	0.018050	5826.0	907.0	0.017535	0.022245	-0.237912	0.003899	0.033869	0.027353
35	(27.993, 28.793]	6330	0.866983	0.016969	5488.0	842.0	0.016517	0.020650	-0.223316	0.001692	0.014596	0.027353
36	(28.793, 29.593]	5835	0.867009	0.015642	5059.0	776.0	0.015226	0.019032	-0.223084	0.000027	0.000232	0.027353
37	(29.593, 30.392]	4719	0.861835	0.012651	4067.0	652.0	0.012241	0.015991	-0.267239	0.005174	0.044155	0.027353
38	(30.392, 31.192]	3803	0.863003	0.010195	3282.0	521.0	0.009878	0.012778	-0.257397	0.001168	0.009842	0.027353
39	(31.192, 31.992]	3486	0.870052	0.009345	3033.0	453.0	0.009129	0.011110	-0.196440	0.007049	0.060957	0.027353
40	(31.992, 32.792]	3085	0.864182	0.008270	2666.0	419.0	0.008024	0.010276	-0.247392	0.005870	0.050952	0.027353
41	(32.792, 33.592]	2817	0.870430	0.007552	2452.0	365.0	0.007380	0.008952	-0.193093	0.006248	0.054298	0.027353
42	(33.592, 34.391]	2637	0.867653	0.007069	2288.0	349.0	0.006886	0.008559	-0.217494	0.002777	0.024401	0.027353
43	(34.391, 35.191]	1883	0.866171	0.005048	1631.0	252.0	0.004909	0.006180	-0.230336	0.001482	0.012842	0.027353
44	(35.191, 35.991]	548	0.916058	0.001469	502.0	46.0	0.001511	0.001128	0.292104	0.049887	0.522439	0.027353
45	(35.991, 36.791]	523	0.891013	0.001402	466.0	57.0	0.001403	0.001398	0.003279	0.025045	0.288824	0.027353
46	(36.791, 37.591]	470	0.882979	0.001260	415.0	55.0	0.001249	0.001349	-0.076910	0.008035	0.080189	0.027353
47	(37.591, 38.39]	445	0.871910	0.001193	388.0	57.0	0.001168	0.001398	-0.179901	0.011069	0.102991	0.027353
48	(38.39, 39.19]	371	0.870620	0.000995	323.0	48.0	0.000972	0.001177	-0.191404	0.001290	0.011503	0.027353
49	(39.19, 39.99]	366	0.887978	0.000981	325.0	41.0	0.000978	0.001006	-0.027602	0.017358	0.163802	0.027353

In [65]:
```
plot_by_woe(df_temp, 90)
```

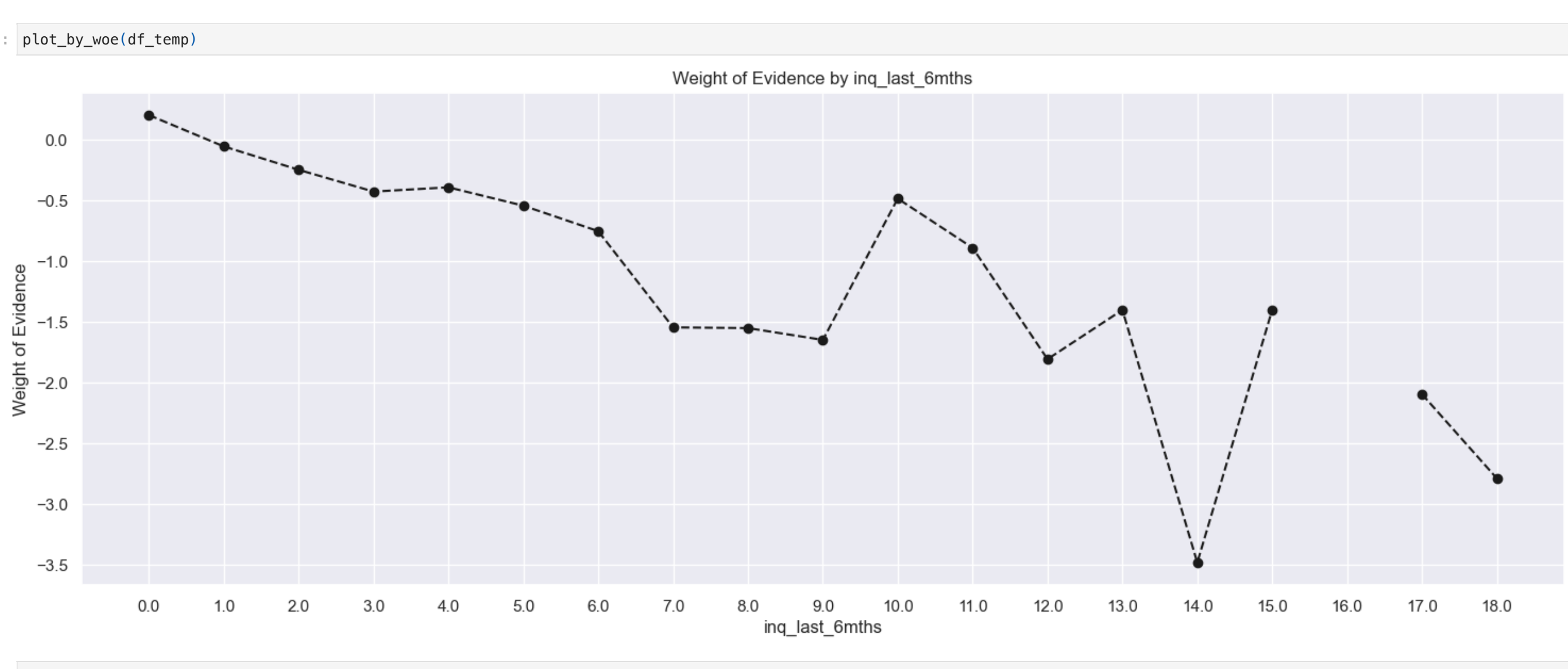

Weight of Evidence by dti_factor

In [66]:
```
df_temp = woe_ordered_continuous(X_train_prepr, 'inq_last_6mths', y_train_prepr)
df_temp
```

Out[66]:

	inq_last_6mths	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	0.0	193162	0.908574	0.517856	175502.0	17660.0	0.528251	0.433151	0.198486	NaN	NaN	inf
1	1.0	104049	0.884814	0.278949	92064.0	11985.0	0.277108	0.293959	-0.059034	0.023760	0.257520	inf
2	2.0	46361	0.863679	0.124291	40041.0	6320.0	0.120521	0.155012	-0.251678	0.021135	0.192643	inf
3	3.0	20110	0.841174	0.053914	16916.0	3194.0	0.050916	0.078340	-0.430877	0.022505	0.179199	inf
4	4.0	5772	0.845807	0.015474	4882.0	890.0	0.014695	0.021829	-0.395774	0.004634	0.035103	inf
5	5.0	2218	0.825068	0.005946	1830.0	388.0	0.005508	0.009517	-0.546797	0.020740	0.151023	inf
6	6.0	960	0.792708	0.002574	761.0	199.0	0.002291	0.004881	-0.756534	0.032359	0.209737	inf
7	7.0	153	0.633987	0.000410	97.0	56.0	0.000292	0.001374	-1.548503	0.158721	0.791969	inf
8	8.0	98	0.632653	0.000263	62.0	36.0	0.000187	0.000883	-1.554247	0.001334	0.005744	inf
9	9.0	41	0.609756	0.000110	25.0	16.0	0.000075	0.000392	-1.651575	0.022897	0.097328	inf
10	10.0	18	0.833333	0.000048	15.0	3.0	0.000045	0.000074	-0.488425	0.223577	1.163151	inf
11	11.0	13	0.769231	0.000035	10.0	3.0	0.000030	0.000074	-0.893890	0.064103	0.405465	inf
12	12.0	14	0.571429	0.000038	8.0	6.0	0.000024	0.000147	-1.810180	0.197802	0.916291	inf
13	13.0	6	0.666667	0.000016	4.0	2.0	0.000012	0.000049	-1.404715	0.095238	0.405465	inf
14	14.0	5	0.200000	0.000013	1.0	4.0	0.000003	0.000098	-3.484157	0.466667	2.079442	inf
15	15.0	6	0.666667	0.000016	4.0	2.0	0.000012	0.000049	-1.404715	0.466667	2.079442	inf
16	16.0	2	1.000000	0.000005	2.0	0.0	0.000006	0.000000	inf	0.333333	inf	inf
17	17.0	2	0.500000	0.000005	1.0	1.0	0.000003	0.000025	-2.097862	0.500000	inf	inf
18	18.0	3	0.333333	0.000008	1.0	2.0	0.000003	0.000049	-2.791010	0.166667	0.693147	inf
19	19.0	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.666667	inf	inf
20	20.0	1	0.000000	0.000003	0.0	1.0	0.000000	0.000025	-inf	1.000000	inf	inf
21	24.0	2	1.000000	0.000005	2.0	0.0	0.000006	0.000000	inf	1.000000	inf	inf
22	25.0	1	0.000000	0.000003	0.0	1.0	0.000000	0.000025	-inf	1.000000	inf	inf
23	27.0	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	1.000000	inf	inf
24	28.0	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
25	31.0	1	1.000000	0.000003	1.0	0.0	0.000003	0.000000	inf	0.000000	NaN	inf
26	32.0	1	0.000000	0.000003	0.0	1.0	0.000000	0.000025	-inf	1.000000	inf	inf
27	33.0	1	0.000000	0.000003	0.0	1.0	0.000000	0.000025	-inf	0.000000	NaN	inf

In [67]:
```
plot_by_woe(df_temp)
```

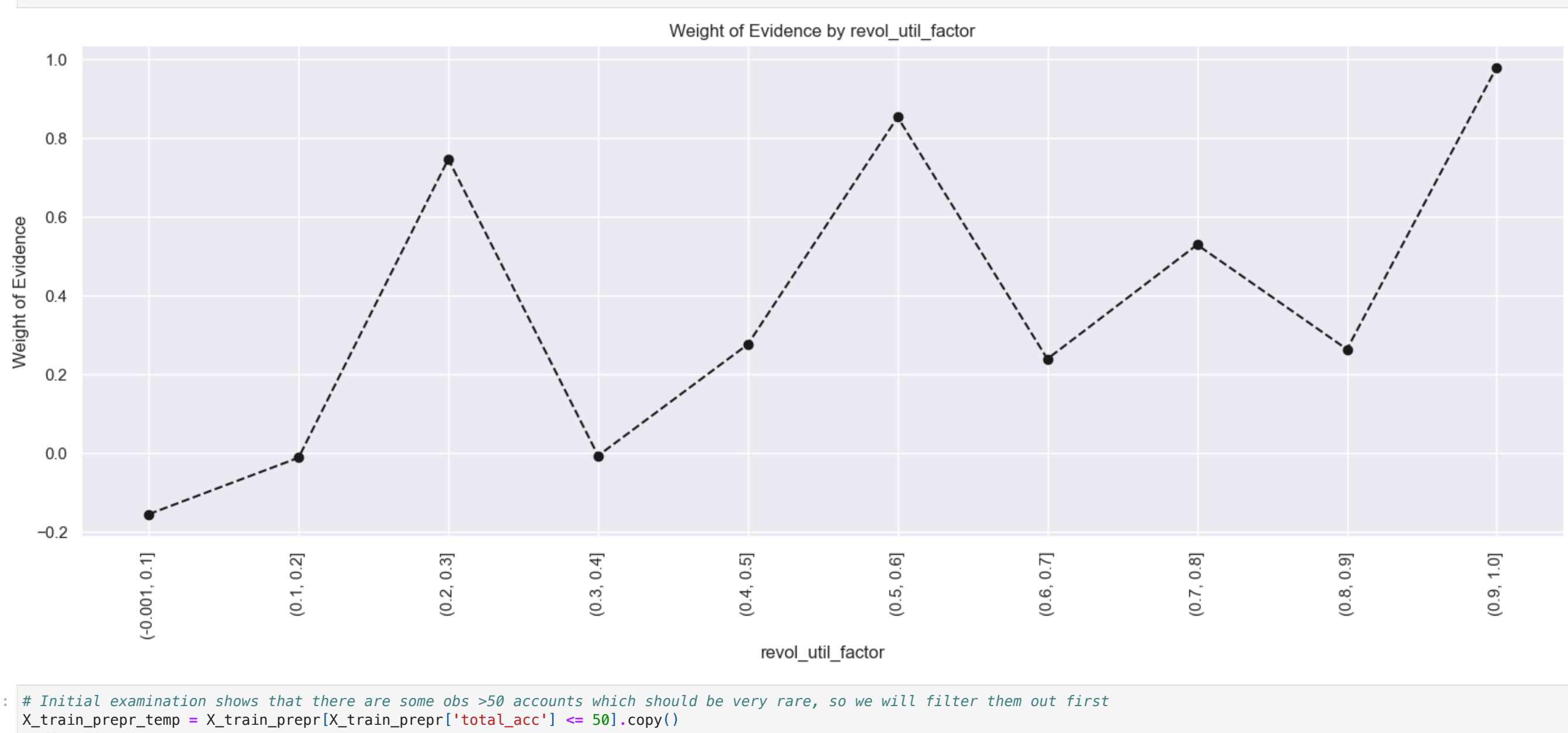

Weight of Evidence by inq_last_6mths

In [68]:
```
# Initial examination shows that there are some obs with utilization of >1 times which should be very rare, so we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['revol_util'] <= 1].copy()
# fine-classing
X_train_prepr_temp['revol_util_factor'] = pd.cut(X_train_prepr_temp['revol_util'], 10)
# preprocess 'revol_util'
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'revol_util_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[68]:

	revol_util_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-0.001, 0.1]	1957	0.866633	0.656271	1696.0	261.0	0.643643	0.752161	-0.155806	NaN	NaN	0.090671
1	(0.1, 0.2]	153	0.882353	0.051308	135.0	18.0	0.051233	0.051873	-0.012411	0.015720	0.143396	0.090671
2	(0.2, 0.3]	136	0.941176	0.045607	128.0	8.0	0.048577	0.023055	0.745275	0.058824	0.757686	0.090671
3	(0.3, 0.4]	111	0.882883	0.037223	98.0	13.0	0.037192	0.037464	-0.007296	0.058294	0.752571	0.090671
4	(0.4, 0.5]	110	0.909091	0.036888	100.0	10.0	0.037951	0.028818	0.275271	0.026208	0.282567	0.090671
5	(0.5, 0.6]	94	0.946809	0.031522	89.0	5.0	0.033776	0.014409	0.851885	0.037718	0.576613	0.090671
6	(0.6, 0.7]	117	0.905983	0.039235	106.0	11.0	0.040228	0.031700	0.238230	0.040826	0.613655	0.090671
7	(0.7, 0.8]	111	0.927928	0.037223	103.0	8.0	0.039089	0.023055	0.527974	0.021945	0.289744	0.090671
8	(0.8, 0.9]	87	0.908046	0.029175	79.0	8.0	0.029981	0.023055	0.262693	0.019882	0.265281	0.090671
9	(0.9, 1.0]	106	0.952830	0.035547	101.0	5.0	0.038330	0.014409	0.978369	0.044784	0.715676	0.090671

In [69]:
```
plot_by_woe(df_temp, 90)
```

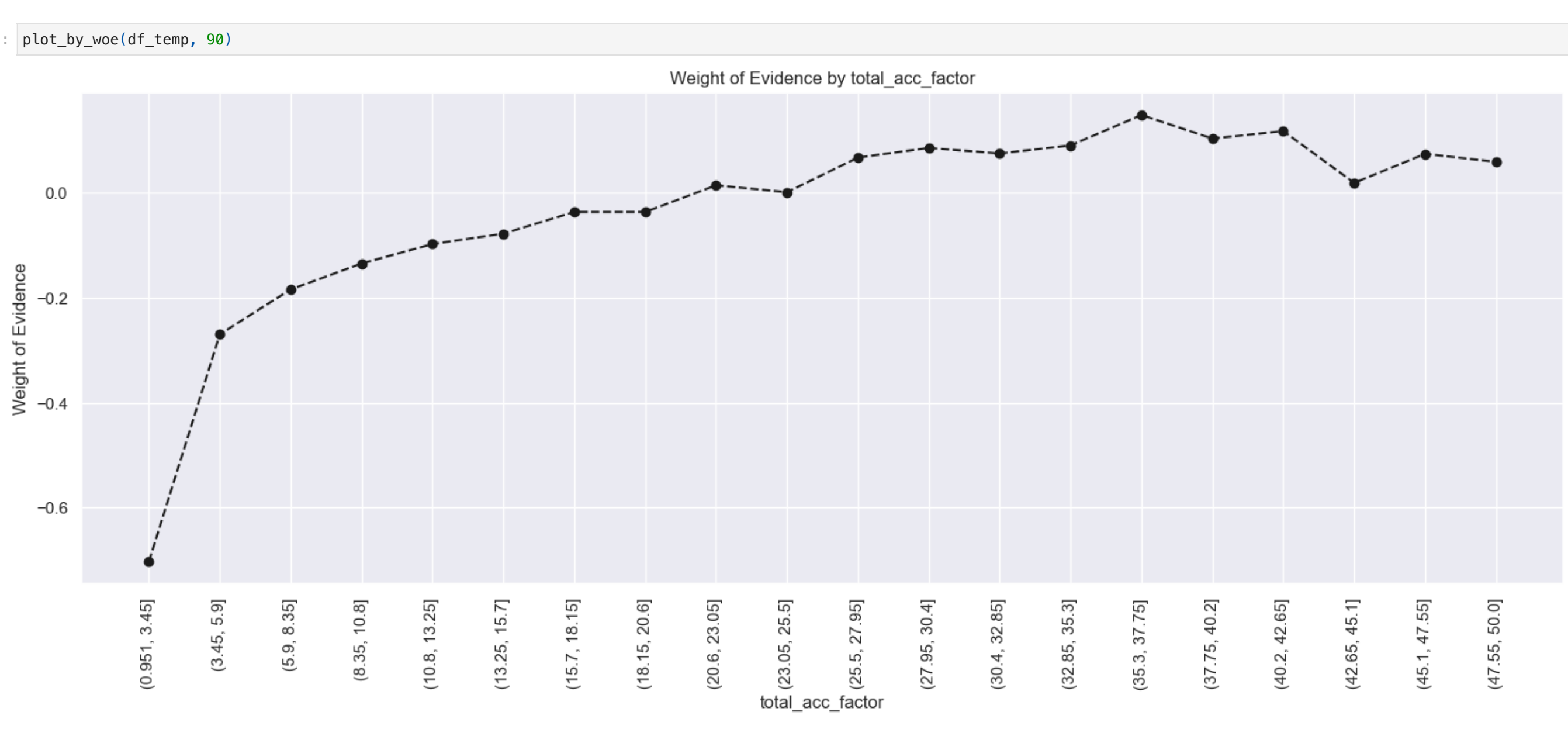

Weight of Evidence by revol_util_factor

In [70]:
```
# Initial examination shows that there are some obs >50 accounts which should be very rare, so we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['total_acc'] <= 50].copy()
# fine-classing
X_train_prepr_temp['total_acc_factor'] = pd.cut(X_train_prepr_temp['total_acc'], 20)
# preprocess 'total_acc'
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'total_acc_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[70]:

	total_acc_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(0.951, 3.45]	462	0.800866	0.001278	370.0	92.0	0.001149	0.002320	-0.702454	NaN	NaN	0.008031
1	(3.45, 5.9]	3117	0.861084	0.008621	2684.0	433.0	0.008338	0.010921	-0.269843	6.021858e-02	0.432611	0.008031
2	(5.9, 8.35]	11689	0.870990	0.032330	10181.0	1508.0	0.031627	0.038033	-0.184430	9.905443e-03	0.085413	0.008031
3	(8.35, 10.8]	13426	0.876434	0.037134	11767.0	1659.0	0.036554	0.041841	-0.135085	5.443966e-03	0.049345	0.008031
4	(10.8, 13.25]	28228	0.880402	0.078074	24852.0	3376.0	0.077203	0.085145	-0.097922	3.968652e-03	0.037163	0.008031
5	(13.25, 15.7]	23108	0.882465	0.063913	20392.0	2716.0	0.063348	0.068499	-0.078186	2.062510e-03	0.019736	0.008031
6	(15.7, 18.15]	38785	0.886683	0.107272	34390.0	4395.0	0.106832	0.110845	-0.036870	4.218049e-03	0.041316	0.008031
7	(18.15, 20.6]	27039	0.886682	0.074785	23975.0	3064.0	0.074478	0.077276	-0.036878	7.962181e-07	0.000008	0.008031
8	(20.6, 23.05]	40701	0.891673	0.112572	36292.0	4409.0	0.112741	0.111198	0.013781	4.991223e-03	0.050659	0.008031
9	(23.05, 25.5]	25768	0.890407	0.071270	22944.0	2824.0	0.071275	0.071223	0.000734	1.266717e-03	0.013047	0.008031
10	(25.5, 27.95]	23857	0.896676	0.065984	21392.0	2465.0	0.066454	0.062169	0.066657	6.269322e-03	0.065923	0.008031
11	(27.95, 30.4]	31583	0.898363	0.087353	28373.0	3210.0	0.088141	0.080958	0.084998	1.687015e-03	0.018342	0.008031
12	(30.4, 32.85]	17933	0.897396	0.049600	16093.0	1840.0	0.049993	0.046406	0.074450	9.671810e-04	0.010548	0.008031
13	(32.85, 35.3]	22324	0.898808	0.061744	20065.0	2259.0	0.062332	0.056974	0.089886	1.412595e-03	0.015436	0.008031
14	(35.3, 37.75]	12265	0.903954	0.033923	11087.0	1178.0	0.034442	0.029710	0.147787	5.145884e-03	0.057900	0.008031
15	(37.75, 40.2]	14628	0.899986	0.040458	13165.0	1463.0	0.040897	0.036898	0.102904	3.968014e-03	0.044882	0.008031
16	(40.2, 42.65]	7869	0.901258	0.021764	7092.0	777.0	0.022031	0.019596	0.117114	1.271774e-03	0.014210	0.008031
17	(42.65, 45.1]	9057	0.892128	0.025050	8080.0	977.0	0.025100	0.024641	0.018492	9.130465e-03	0.098622	0.008031
18	(45.1, 47.55]	4519	0.897322	0.012499	4055.0	464.0	0.012597	0.011702	0.073653	5.194780e-03	0.055161	0.008031
19	(47.55, 50.0]	5198	0.895922	0.014377	4657.0	541.0	0.014467	0.013644	0.058539	1.400908e-03	0.015114	0.008031

In [71]:
```
plot_by_woe(df_temp, 90)
```


Weight of Evidence by total_acc_factor

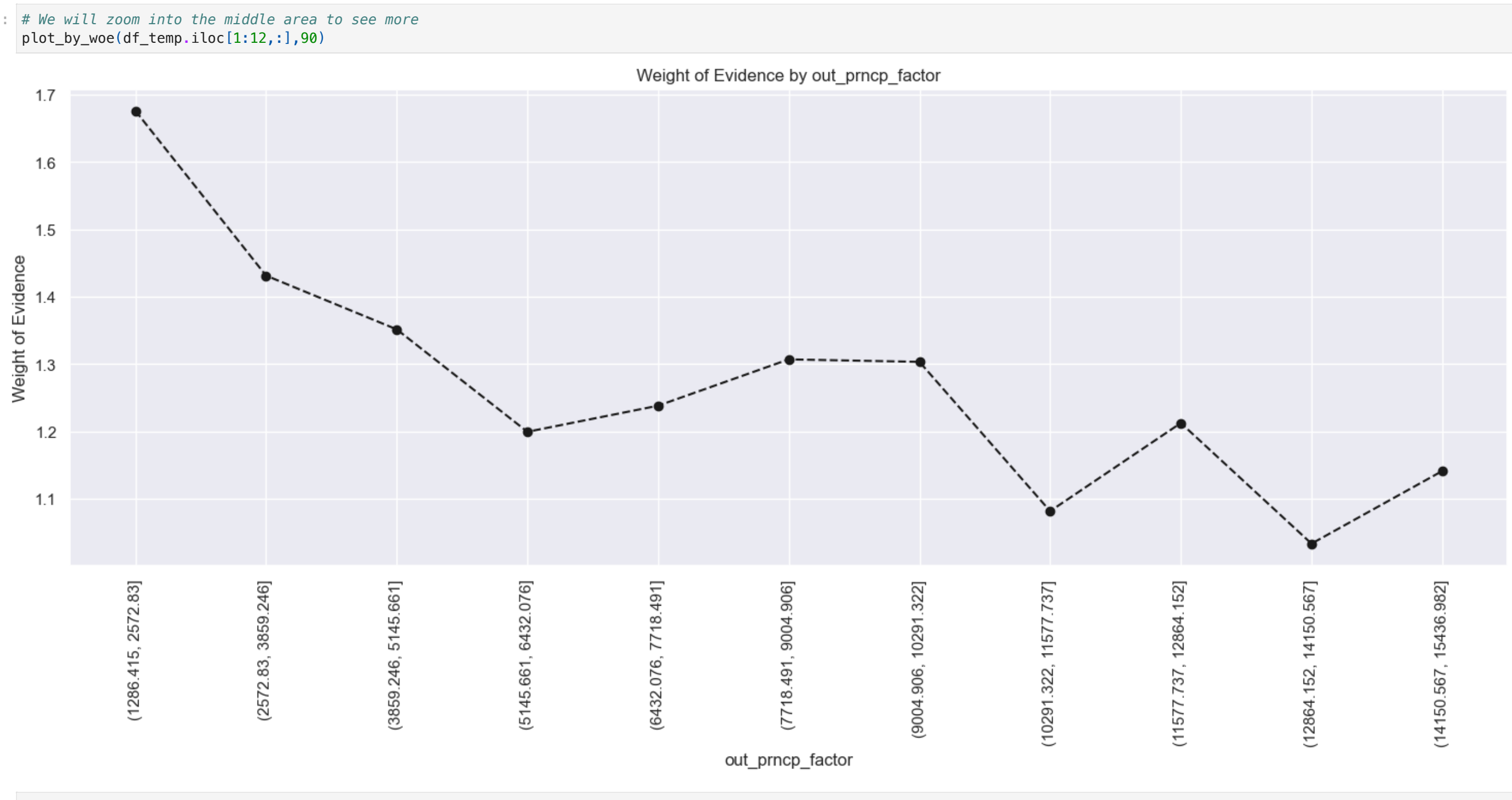
We will discard this feature given its low IV

In [72]:
```
# Here we do fine-classing: using the 'cut' method, we split the variable into 25 categories by its values.
X_train_prepr['out_prncp_factor'] = pd.cut(X_train_prepr['out_prncp'], 25)
# We calculate weight of evidence.
df_temp = woe_ordered_continuous(X_train_prepr, 'out_prncp_factor', y_train_prepr)
df_temp
```

Out[72]:

	out_prncp_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-32.16, 1286.415]	196686	0.823078	0.527269	161888.0	34798.0	0.487242	0.853436	-0.560510	NaN	NaN	0.681688
1	(1286.415, 2572.83]	19695	0.977558	0.052798	19253.0	442.0	0.057947	0.010840	1.676257	0.154479	2.236767	0.681688
2	(2572.83, 3859.246]	21412	0.971511	0.057401	20802.0	610.0	0.062609	0.014961	1.431490	0.006046	0.244767	0.681688
3	(3859.246, 5145.661]	17867	0.969217	0.047897	17317.0	550.0	0.052120	0.013489	1.351671	0.002294	0.079820	0.681688
4	(5145.661, 6432.076]	14835	0.964341	0.039769	14306.0	529.0	0.043057	0.012974	1.199591	0.004876	0.152080	0.681688
5	(6432.076, 7718.491]	13042	0.965649	0.034963	12594.0	448.0	0.037905	0.010987	1.238327	0.001308	0.038737	0.681688
6	(7718.491, 9004.906]	13099	0.967860	0.035115	12678.0	421.0	0.038158	0.010325	1.307136	0.002211	0.068808	0.681688
7	(9004.906, 10291.322]	12866	0.967744	0.034491	12451.0	415.0	0.037474	0.010178	1.303423	0.000116	0.003713	0.681688
8	(10291.322, 11577.737]	9890	0.960061	0.026513	9495.0	395.0	0.028578	0.009688	1.081780	0.007684	0.221643	0.681688
9	(11577.737, 12864.152]	9513	0.964785	0.025502	9178.0	335.0	0.027623	0.008216	1.212579	0.004724	0.130799	0.681688
10	(12864.152, 14150.567]	7363	0.958169	0.019738	7055.0	308.0	0.021234	0.007554	1.033537	0.006616	0.179042	0.681688
11	(14150.567, 15436.982]	6363	0.962282	0.017058	6123.0	240.0	0.018429	0.005886	1.141313	0.004113	0.107776	0.681688
12	(15436.982, 16723.398]	6095	0.965381	0.016339	5884.0	211.0	0.017709	0.005175	1.230279	0.003100	0.088965	0.681688
13	(16723.398, 18009.813]	4557	0.960500	0.012216	4377.0	180.0	0.013174	0.004415	1.093307	0.004881	0.136972	0.681688
14	(18009.813, 19296.228]	3834	0.960094	0.010278	3681.0	153.0	0.011079	0.003752	1.082647	0.000406	0.010660	0.681688
15	(19296.228, 20582.643]	3832	0.965292	0.010273	3699.0	133.0	0.011133	0.003262	1.227614	0.005198	0.144967	0.681688
16	(20582.643, 21869.058]	2577	0.949942	0.006908	2448.0	129.0	0.007368	0.003164	0.845359	0.015350	0.382254	0.681688
17	(21869.058, 23155.474]	2437	0.960607	0.006533	2341.0	96.0	0.007046	0.002354	1.096130	0.010666	0.250771	0.681688
18	(23155.474, 24441.889]	1915	0.950914	0.005134	1821.0	94.0	0.005481	0.002305	0.865991	0.009693	0.230139	0.681688
19	(24441.889, 25728.304]	1509	0.951624	0.004045	1436.0	73.0	0.004322	0.001790	0.881302	0.000710	0.015311	0.681688
20	(25728.304, 27014.719]	1101	0.947321	0.002952	1043.0	58.0	0.003139	0.001422	0.791558	0.004303	0.089744	0.681688
21	(27014.719, 28301.134]	1113	0.962264	0.002984	1071.0	42.0	0.003223	0.001030	1.140823	0.014944	0.349265	0.681688
22	(28301.134, 29587.55]	1098	0.954463	0.002943	1048.0	50.0	0.003154	0.001226	0.944761	0.007801	0.196063	0.681688
23	(29587.55, 30873.965]	302	0.847682	0.000810	256.0	46.0	0.000770	0.001128	-0.381319	0.106781	1.326080	0.681688
24	(30873.965, 32160.38]	27	0.333333	0.000072	9.0	18.0	0.000027	0.000441	-2.791002	0.514349	2.409683	0.681688

In [73]:
```
# We will zoom into the middle area to see more
plot_by_woe(df_temp.iloc[1:12,:],90)
```


Weight of Evidence by out_prncp_factor

In [74]:
```
# Initial examination shows that there are vey few obs with >25,000 which should be very rare, so we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['total_pymnt'] <= 25000].copy()
# fine-classing
X_train_prepr_temp['total_pymnt_factor'] = pd.cut(X_train_prepr_temp['total_pymnt'], 20)
# preprocess 'total_acc'
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'total_pymnt_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

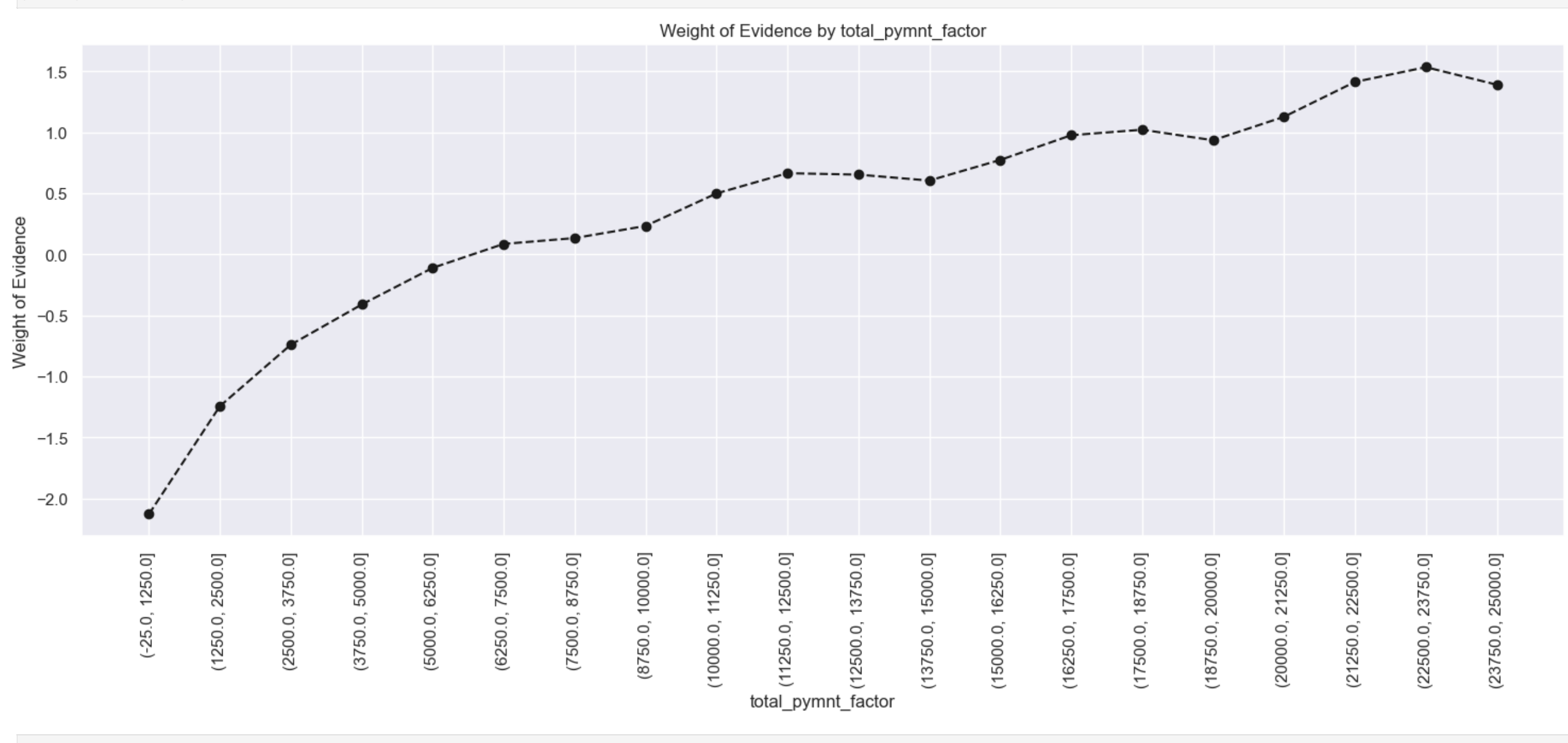
Out[74]:

	total_pymnt_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-25.0, 1250.0]	5609	0.473881	0.016279	2658.0	2951.0	0.008734	0.073386	-2.128553	NaN	NaN	0.548419
1	(1250.0, 2500.0]	16242	0.685753	0.047139	11138.0	5104.0	0.036597	0.126927	-1.243645	0.211872	0.884908	0.548419
2	(2500.0, 3750.0]	25630	0.783145	0.074386	20072.0	5558.0	0.065952	0.138217	-0.739896	0.097392	0.503749	0.548419
3	(3750.0, 5000.0]	30880	0.833873	0.089623	25750.0	5130.0	0.084609	0.127574	-0.410654	0.050728	0.329241	0.548419

4	(5000.0, 6250.0]	33943	0.871432	0.098513	29579.0	4364.0	0.097190	0.108525	-0.110308	0.037558	0.300347	0.548419
5	(6250.0, 7500.0]	32268	0.891843	0.093652	28778.0	3490.0	0.094558	0.086790	0.085726	0.020412	0.196034	0.548419
6	(7500.0, 8750.0]	28052	0.896371	0.081416	25145.0	2907.0	0.082621	0.072292	0.133554	0.004528	0.047828	0.548419
7	(8750.0, 10000.0]	24748	0.905245	0.071826	22403.0	2345.0	0.073612	0.058316	0.232926	0.008874	0.099372	0.548419
8	(10000.0, 11250.0]	24596	0.925760	0.071385	22770.0	1826.0	0.074817	0.045409	0.499333	0.020515	0.266407	0.548419
9	(11250.0, 12500.0]	21711	0.936392	0.063012	20330.0	1381.0	0.066800	0.034343	0.665306	0.010631	0.165974	0.548419
10	(12500.0, 13750.0]	18174	0.935677	0.052747	17005.0	1169.0	0.055875	0.029071	0.653375	0.000714	0.011931	0.548419
11	(13750.0, 15000.0]	14622	0.932704	0.042438	13638.0	984.0	0.044812	0.024470	0.605006	0.002973	0.048369	0.548419
12	(15000.0, 16250.0]	12950	0.942548	0.037585	12206.0	744.0	0.040106	0.018502	0.773659	0.009844	0.168652	0.548419
13	(16250.0, 17500.0]	12423	0.952588	0.036055	11834.0	589.0	0.038884	0.014647	0.976323	0.010040	0.202664	0.548419
14	(17500.0, 18750.0]	9601	0.954588	0.027865	9165.0	436.0	0.030114	0.010843	1.021522	0.002000	0.045199	0.548419
15	(18750.0, 20000.0]	8112	0.950690	0.023544	7712.0	400.0	0.025340	0.009947	0.935085	0.003898	0.086437	0.548419
16	(20000.0, 21250.0]	7360	0.958967	0.021361	7058.0	302.0	0.023191	0.007510	1.127507	0.008277	0.192422	0.548419
17	(21250.0, 22500.0]	6932	0.968840	0.020119	6716.0	216.0	0.022067	0.005372	1.412986	0.009873	0.285480	0.548419
18	(22500.0, 23750.0]	5991	0.972292	0.017388	5825.0	166.0	0.019140	0.004128	1.533943	0.003452	0.120957	0.548419
19	(23750.0, 25000.0]	4709	0.968146	0.013667	4559.0	150.0	0.014980	0.003730	1.390240	0.004146	0.143703	0.548419

In [75]:

```
plot_by_woe(df_temp,90)
```



In [76]:

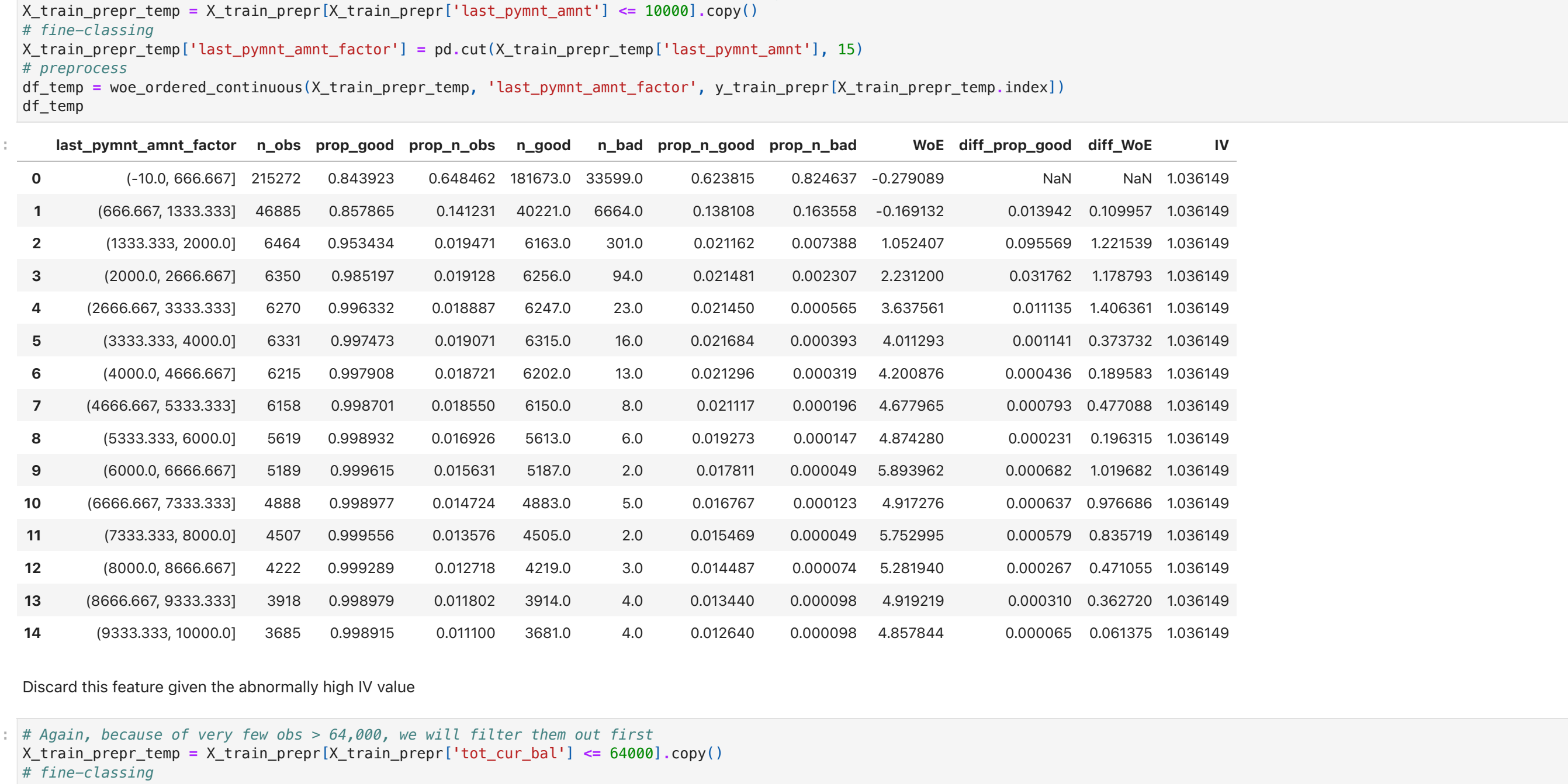
```
# Initial examination shows that there are vey few obs with >7,261 which should be very rare, so we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['total_rec_int'] <= 7261].copy()
# fine-classing
X_train_prepr_temp['total_rec_int_factor'] = pd.cut(X_train_prepr_temp['total_rec_int'], 20)
# preprocess 'total_rec_int'
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'total_rec_int_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[76]:

	total_rec_int_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-7.261, 363.03]	22845	0.836594	0.064964	19112.0	3733.0	0.061078	0.096345	-0.455790	NaN	NaN	0.024917
1	(363.03, 726.061]	41058	0.872546	0.116755	35825.0	5233.0	0.114489	0.135059	-0.165233	0.035952	0.290557	0.024917
2	(726.061, 1089.092]	45720	0.890332	0.130013	40706.0	5014.0	0.130088	0.129407	0.005247	0.017786	0.170480	0.024917
3	(1089.092, 1452.122]	41081	0.892797	0.116821	36677.0	4404.0	0.117212	0.113663	0.030742	0.002465	0.025495	0.024917
4	(1452.122, 1815.152]	35526	0.896977	0.101024	31866.0	3660.0	0.101837	0.094461	0.075182	0.004180	0.044440	0.024917
5	(1815.152, 2178.183]	30227	0.898601	0.085956	27162.0	3065.0	0.086804	0.079105	0.092877	0.001624	0.017695	0.024917
6	(2178.183, 2541.214]	25037	0.898790	0.071197	22503.0	2534.0	0.071915	0.065400	0.094955	0.000189	0.002078	0.024917
7	(2541.214, 2904.244]	20417	0.902532	0.058059	18427.0	1990.0	0.058889	0.051360	0.136788	0.003742	0.041833	0.024917
8	(2904.244, 3267.274]	16480	0.899636	0.046864	14826.0	1654.0	0.047381	0.042688	0.104291	0.002896	0.032497	0.024917
9	(3267.274, 3630.305]	13114	0.897819	0.037292	11774.0	1340.0	0.037627	0.034584	0.084330	0.001817	0.019962	0.024917
10	(3630.305, 3993.335]	10758	0.893661	0.030592	9614.0	1144.0	0.030724	0.029526	0.039795	0.004159	0.044535	0.024917
11	(3993.335, 4356.366]	9187	0.899205	0.026125	8261.0	926.0	0.026400	0.023899	0.099532	0.005545	0.059737	0.024917
12	(4356.366, 4719.396]	7976	0.898320	0.022681	7165.0	811.0	0.022898	0.020931	0.089801	0.000885	0.009731	0.024917
13	(4719.396, 5082.427]	6684	0.907241	0.019007	6064.0	620.0	0.019379	0.016002	0.191511	0.008921	0.101710	0.024917
14	(5082.427, 5445.457]	5646	0.896741	0.016055	5063.0	583.0	0.016180	0.015047	0.072633	0.010500	0.118878	0.024917
15	(5445.457, 5808.488]	5117	0.896228	0.014551	4586.0	531.0	0.014656	0.013705	0.067107	0.000513	0.005526	0.024917
16	(5808.488, 6171.518]	4461	0.898902	0.012686	4010.0	451.0	0.012815	0.011640	0.096185	0.002673	0.029078	0.024917
17	(6171.518, 6534.549]	3861	0.900285	0.010979	3476.0	385.0	0.011109	0.009937	0.111500	0.001383	0.015315	0.024917
18	(6534.549, 6897.579]	3483	0.891473	0.009905	3105.0	378.0	0.009923	0.009756	0.016980	0.008812	0.094519	0.024917
19	(6897.579, 7260.61]	2980	0.902685	0.008474	2690.0	290.0	0.008597	0.007485	0.138521	0.011212	0.121541	0.024917

In [77]:

```
plot_by_woe(df_temp, 90)
```



In [78]:

```
# Initial examination shows that there are vey few obs with 10,000 which should be very rare, so we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['last_pymnt_amnt'] <= 10000].copy()
# fine-classing
X_train_prepr_temp['last_pymnt_amnt_factor'] = pd.cut(X_train_prepr_temp['last_pymnt_amnt'], 15)
# preprocess
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'last_pymnt_amnt_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[78]:

	last_pymnt_amnt_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-10.0, 666.667]	215272	0.843923	0.648462	181673.0	33599.0	0.623815	0.824637	-0.279089	NaN	NaN	1.036149
1	(666.667, 1333.333]	46885	0.857865	0.141231	40221.0	6664.0	0.138108	0.163558	-0.169132	0.013942	0.109957	1.036149
2	(1333.333, 2000.0]	6464	0.953434	0.019471	6163.0	301.0	0.021162	0.007388	1.052407	0.095569	1.221539	1.036149
3	(2000.0, 2666.667]	6350	0.985197	0.019128	6256.0	94.0	0.021481	0.002307	2.231200	0.031762	1.178793	1.036149
4	(2666.667, 3333.333]	6270	0.996332	0.018887	6247.0	23.0	0.021450	0.000565	3.637561	0.011135	1.406361	1.036149
5	(3333.333, 4000.0]	6331	0.997473	0.019071	6315.0	16.0	0.021684	0.000393	4.011293	0.001141	0.373732	1.036149
6	(4000.0, 4666.667]	6215	0.997908	0.018721	6202.0	13.0	0.021296	0.000319	4.200876	0.000436	0.189583	1.036149
7	(4666.667, 5333.333]	6158	0.998701	0.018550	6150.0	8.0	0.021117	0.000196	4.677965	0.000793	0.477088	1.036149
8	(5333.333, 6000.0]	5619	0.998932	0.016926	5613.0	6.0	0.019273	0.000147	4.874280	0.000231	0.196315	1.036149
9	(6000.0, 6666.667]	5189	0.999615	0.015631	5187.0	2.0	0.017811	0.000049	5.893962	0.000682	1.019682	1.036149
10	(6666.667, 7333.333]	4888	0.998977	0.014724	4883.0	5.0	0.016767	0.000123	4.917276	0.000637	0.976686	1.036149
11	(7333.333, 8000.0]	4507	0.999556	0.013576	4505.0	2.0	0.015469	0.000049	5.752995	0.000579	0.835719	1.036149
12	(8000.0, 8666.667]	4222	0.999289	0.012718	4219.0	3.0	0.014487	0.000074	5.281940	0.000267	0.471055	1.036149
13	(8666.667, 9333.333]	3918	0.998979	0.011802	3914.0	4.0	0.013440	0.000098	4.919219	0.000310	0.362720	1.036149
14	(9333.333, 10000.0]	3685	0.998915	0.011100	3681.0	4.0	0.012640	0.000098	4.857844	0.000065	0.061375	1.036149

Discard this feature given the abnormally high IV value

In [79]:

```
# Again, because of very few obs > 64,000, we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['tot_cur_bal'] <= 64000].copy()
# fine-classing
X_train_prepr_temp['tot_cur_bal_factor'] = pd.cut(X_train_prepr_temp['tot_cur_bal'], 25)
# preprocess
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'tot_cur_bal_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[79]:

	tot_cur_bal_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-64.0, 2560.0]	1778	0.902700	0.012373	1605.0	173.0	0.012617	0.010492	0.184381	NaN	NaN	0.00311
1	(2560.0, 5120.0]	4621	0.897425	0.032158	4147.0	474.0	0.032600	0.028748	0.125727	0.005275	0.058654	0.00311
2	(5120.0, 7680.0]	6453	0.897567	0.044907	5792.0	661.0	0.045531	0.040090	0.127273	0.000142	0.001546	0.00311
3	(7680.0, 10240.0]	7500	0.887867	0.052193	6659.0	841.0	0.052347	0.051007	0.025927	0.009700	0.101346	0.00311
4	(10240.0, 12800.0]	7949	0.883759	0.055317	7025.0	924.0	0.055224	0.056041	-0.014688	0.004108	0.040615	0.00311
5	(12800.0, 15360.0]	8321	0.882706	0.057906	7345.0	976.0	0.057739	0.059195	-0.024894	0.001053	0.010206	0.00311
6	(15360.0, 17920.0]	8342	0.888036	0.058052	7408.0	934.0	0.058234	0.056647	0.027633	0.005330	0.052527	0.00311
7	(17920.0, 20480.0]	8406	0.878420	0.058498	7384.0	1022.0	0.058046	0.061984	-0.065652	0.009616	0.093285	0.00311
8	(20480.0, 23040.0]	8246	0.880669	0.057384	7262.0	984.0	0.057087	0.059680	-0.044422	0.002249	0.021231	0.00311
9	(23040.0, 25600.0]	8260	0.880993	0.057482	7277.0	983.0	0.057205	0.059619	-0.041342	0.000323	0.003080	0.00311
10	(25600.0, 28160.0]	7839	0.878684	0.054552	6888.0	951.0	0.054147	0.057678	-0.063184	0.002309	0.021843	0.00311
11	(28160.0, 30720.0]	7563	0.887479	0.052631	6712.0	851.0	0.052763	0.051613	0.022034	0.008795	0.085218	0.00311
12	(30720.0, 33280.0]	6977	0.886054	0.048553	6182.0	795.0	0.048597	0.048217	0.007849	0.001424	0.014185	0.00311
13	(33280.0, 35840.0]	6457	0.881524	0.044935	5692.0	765.0	0.044745	0.046397	-0.036265	0.004530	0.044114	0.00311
14	(35840.0, 38400.0]	6137	0.888545	0.042708	5453.0	684.0	0.042866	0.041485	0.032757	0.007021	0.069022	0.00311
15	(38400.0, 40960.0]	5567	0.889168	0.038741	4950.0	617.0	0.038912	0.037421	0.039067	0.000623	0.006311	0.00311
16	(40960.0, 43520.0]	5017	0.880407	0.034913	4417.0	600.0	0.034722	0.036390	-0.046920	0.008762	0.085987	0.00311
17	(43520.0, 46080.0]	4637	0.879664	0.032269	4079.0	558.0	0.032065	0.033843	-0.053958	0.000743	0.007038	0.00311
18	(46080.0, 48640.0]	4277	0.883797	0.029764	3780.0	497.0	0.029715	0.030143	-0.014317	0.004133	0.039641	0.00311
19	(48640.0, 51200.0]	3952	0.883603	0.027502	3492.0	460.0	0.027451	0.027899	-0.016203	0.000194	0.001886	0.00311
20	(51200.0, 53760.0]	3556	0.884421	0.024746	3145.0	411.0	0.024723	0.024927	-0.008230	0.000817	0.007973	0.00311
21	(53760.0, 56320.0]	3348	0.887993	0.023299	2973.0	375.0	0.023371	0.022744	0.027194	0.003572	0.035425	0.00311
22	(56320.0, 58880.0]	3031	0.897064	0.021093	2719.0	312.0	0.021374	0.018923	0.121810	0.009071	0.094615	0.00311
23	(58880.0, 61440.0]	2795	0.883005	0.019451	2468.0	327.0	0.019401	0.019833	-0.022003	0.014058	0.143813	0.00311
24	(61440.0, 64000.0]	2669	0.882728	0.018574	2356.0	313.0	0.018521	0.018984	-0.024689	0.000278	0.002686	0.00311

In [80]:

```
# Again, because of very few obs > 64,000, we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['tot_cur_bal'] <= 64000].copy()
# fine-classing
X_train_prepr_temp['tot_cur_bal_factor'] = pd.cut(X_train_prepr_temp['tot_cur_bal'], 25)
# preprocess
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'tot_cur_bal_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[80]:

	tot_cur_bal_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-64.0, 2560.0]	1778	0.902700	0.012373	1605.0	173.0	0.012617	0.010492	0.184381	NaN	NaN	0.00311
1	(2560.0, 5120.0]	4621	0.897425	0.032158	4147.0	474.0	0.032600	0.028748	0.125727	0.005275	0.058654	0.00311
2	(5120.0, 7680.0]	6453	0.897567	0.044907	5792.0	661.0	0.045531	0.040090	0.127273	0.000142	0.001546	0.00311
3	(7680.0, 10240.0]	7500	0.887867	0.052193	6659.0	841.0	0.052347	0.051007	0.025927	0.009700	0.101346	0.00311
4	(10240.0, 12800.0]	7949	0.883759	0.055317	7025.0	924.0	0.055224	0.056041	-0.014688	0.004108	0.040615	0.00311
5	(12800.0, 15360.0]	8321	0.882706	0.057906	7345.0	976.0	0.057739	0.059195	-0.024894	0.001053	0.010206	0.00311
6	(15360.0, 17920.0]	8342	0.888036	0.058052	7408.0	934.0	0.058234	0.056647	0.027633	0.005330	0.052527	0.00311
7	(17920.0, 20480.0]	8406	0.878420	0.058498	7384.0	1022.0	0.058046	0.061984	-0.065652	0.009616	0.093285	0.00311
8	(20480.0, 23040.0]	8246	0.880669	0.057384	7262.0	984.0	0.057087	0.059680	-0.044422	0.002249	0.021231	0.00311
9	(23040.0, 25600.0]	8260	0.880993	0.057482	7277.0	983.0	0.057205	0.059619	-0.041342	0.000323	0.003080	0.00311
10	(25600.0, 28160.0]	7839	0.878684	0.054552	6888.0	951.0	0.054147	0.057678	-0.063184	0.002309	0.021843	0.00311
11	(28160.0, 30720.0]	7563	0.887479	0.052631	6712.0	851.0	0.052763	0.051613	0.022034	0.008795	0.085218	0.00311
12	(30720.0, 33280.0]	6977	0.886054	0.048553	6182.0	795.0	0.048597	0.048217	0.007849	0.001424	0.014185	0.00311
13	(33280.0, 35840.0]	6457	0.881524	0.044935	5692.0	765.0	0.044745	0.046397	-0.036265	0.004530	0.044114	0.00311
14	(35840.0, 38400.0]	6137	0.888545	0.042708	5453.0	684.0	0.042866	0.041485	0.032757	0.007021	0.069022	0.00311
15	(38400.0, 40960.0]	5567	0.889168	0.038741	4950.0	617.0	0.038912	0.037421	0.039067	0.000623	0.006311	0.00311
16	(40960.0, 43520.0]	5017	0.880407	0.034913	4417.0	600.0	0.034722	0.036390	-0.046920	0.008762	0.085987	0.00311
17	(43520.0, 46080.0]	4637	0.879664	0.032269	4079.0	558.0	0.032065	0.033843	-0.053958	0.000743	0.007038	0.00311
18	(46080.0, 48640.0]	4277	0.883797	0.029764	3780.0	497.0	0.029715	0.030143	-0.014317	0.004133	0.039641	0.00311
19	(48640.0, 51200.0]	3952	0.883603	0.027502	3492.0	460.0	0.027451	0.027899	-0.016203	0.000194	0.001886	0.00311
20	(51200.0, 53760.0]	3556	0.884421	0.024746	3145.0	411.0	0.024723	0.024927	-0.008230	0.000817	0.007973	0.00311
21	(53760.0, 56320.0]	3348	0.887993	0.023299	2973.0	375.0	0.023371	0.022744	0.027194	0.003572	0.035425	0.00311
22	(56320.0, 58880.0]	3031	0.897064	0.021093	2719.0	312.0	0.021374	0.018923	0.121810	0.009071	0.094615	0.00311
23	(58880.0, 61440.0]	2795	0.883005	0.019451	2468.0	327.0	0.019401	0.019833	-0.022003	0.014058	0.143813	0.00311
24	(61440.0, 64000.0]	2669	0.882728	0.018574	2356.0	313.0	0.018521	0.018984	-0.024689	0.000278	0.002686	0.00311

Ignore this feature given the low IV value

In [81]:

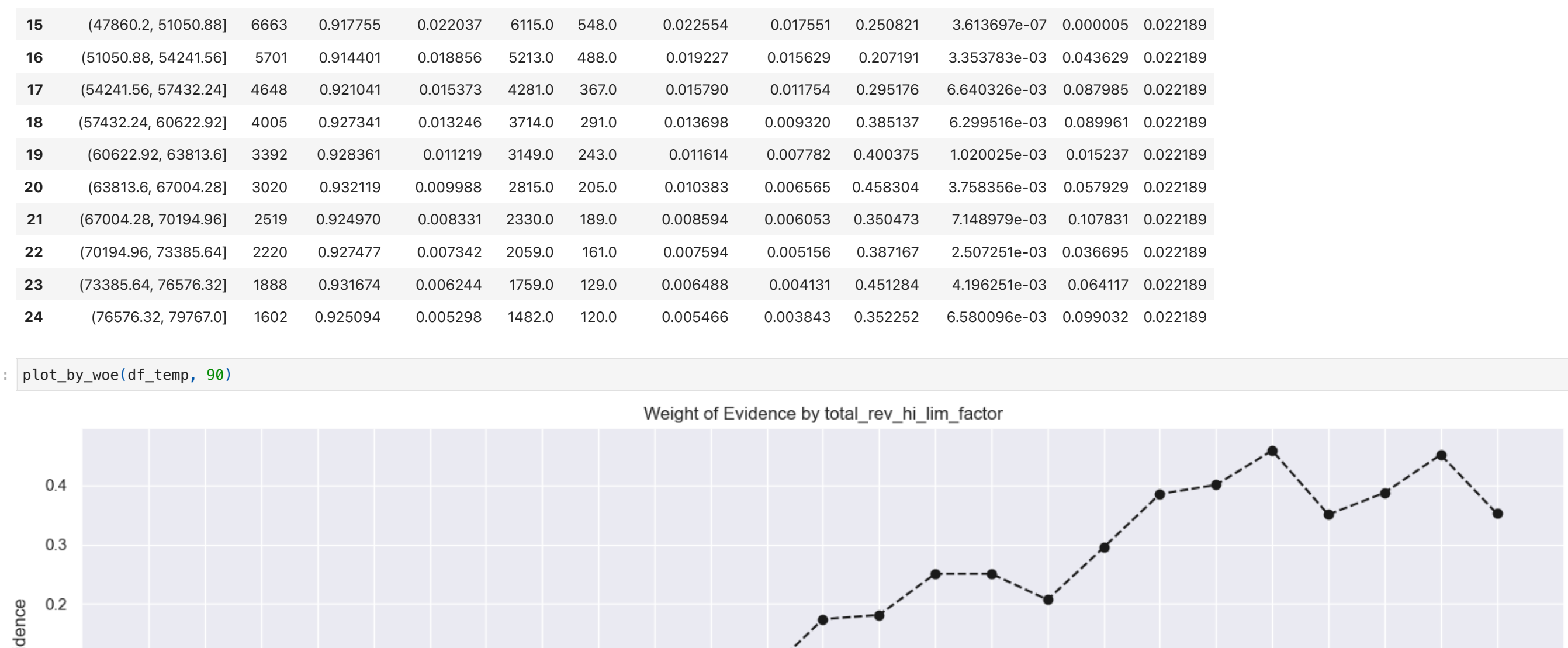
```
# initial examination reveals very few obs > 79,780, we will filter them out first
X_train_prepr_temp = X_train_prepr[X_train_prepr['total_rev_hi_lim'] <= 79780].copy()
# fine-classing
X_train_prepr_temp['total_rev_hi_lim_factor'] = pd.cut(X_train_prepr_temp['total_rev_hi_lim'], 25)
# preprocess
df_temp = woe_ordered_continuous(X_train_prepr_temp, 'total_rev_hi_lim_factor', y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[81]:

	total_rev_hi_lim_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(-79.767, 3190.68]	3574	0.866256	0.011821	3096.0	478.0	0.011419	0.015309	-0.293149	NaN	NaN	0.022189
1	(3190.68, 6381.36]	14632	0.880331	0.048394	12881.0	1751.0	0.047509	0.056079	-0.165838	1.407449e-02	0.127311	0.022189
2	(6381.36, 9572.04]	24622	0.885306	0.081435	21798.0	2824.0	0.080397	0.090443	-0.117740	4.975042e-03	0.048098	0.022189
3	(9572.04, 12762.72]	29008	0.885790	0.095941	25695.0	3313.0	0.094771	0.106104	-0.112962	4.843028e-04	0.004778	0.022189
4	(12762.72, 15953.4]	29826	0.887481	0.098647	26470.0	3356.0	0.097629	0.107481	-0.096142	1.690595e-03	0.016820	0.022189
5	(15953.4, 19144.08]	28069	0.888703	0.092836	24945.0	3124.0	0.092005	0.100051	-0.083845	1.222118e-03	0.012297	0.022189
6	(19144.08, 22334.76]	25057	0.893762	0.082874	22395.0	2662.0	0.082599	0.085255	-0.031644	5.059383e-03	0.052201	0.022189
7	(22334.76, 25525.44]	22187	0.893767	0.073381	19830.0	2357.0	0.073139	0.075487	-0.031598	4.397959e-06	0.000046	0.022189
8	(25525.44, 28716.12]	19265	0.897898	0.063717	17298.0	1967.0	0.063800	0.062996	0.012677	4.131122e-03	0.044275	0.022189
9	(28716.12, 31906.8]	16446	0.898760	0.054394	14781.0	1665.0	0.054517	0.053324	0.022113	8.618348e-04	0.009436	0.022189
10	(31906.8, 35097.48]	14019	0.898852	0.046366	12601.0	1418.0	0.046476	0.045414	0.023125	9.198180e-05	0.001011	0.022189
11	(35097.48, 38288.16]	12208	0.904161	0.040377	11038.0	1170.0	0.040711	0.037471	0.082936	5.309647e-03	0.059811	0.022189
12	(38288.16, 41478.84]	10772	0.911808	0.035627	9822.0	950.0	0.036226	0.030425	0.174514	7.647186e-03	0.091578	0.022189
13	(41478.84, 44669.52]	9118	0.912371	0.030157	8319.0	799.0	0.030683	0.025589	0.181532	5.627419e-04	0.007018	0.022189
14	(44669.52, 47860.2]	7891	0.917754	0.026099	7242.0	649.0	0.026711	0.020785	0.250816	5.383270e-03	0.069284	0.022189
15	(47860.2, 51050.88]	6663	0.917755	0.022037	6115.0	548.0	0.022554	0.017551	0.250821	3.613697e-07	0.000005	0.022189
16	(51050.88, 54241.56]	5701	0.914401	0.018856	5213.0	488.0	0.019227	0.015629	0.207191	3.353783e-03	0.043629	0.022189
17	(54241.56, 57432.24]	4648	0.921041	0.015373	4281.0	367.0	0.015790	0.011754	0.295176	6.640326e-03	0.087985	0.022189
18	(57432.24, 60622.92]	4005	0.927341	0.013246	3714.0	291.0	0.013698	0.009320	0.385137	6.299516e-03	0.089961	0.022189
19	(60622.92, 63813.6]	3392	0.928361	0.011219	3149.0	243.0	0.011614	0.007782	0.400375	1.020025e-03	0.015237	0.022189
20	(63813.6, 67004.28]	3020	0.932119	0.009988	2815.0	205.0	0.010383	0.006565	0.458304	3.758356e-03	0.057929	0.022189
21	(67004.28, 70194.96]	2519	0.924970	0.008331	2330.0	189.0	0.008594	0.006053	0.350473	7.148979e-03	0.107831	0.022189
22	(70194.96, 73385.64]	2220	0.927477	0.007342	2059.0	161.0	0.007594	0.005156	0.387167	2.507251e-03	0.036695	0.022189
23	(73385.64, 76576.32]	1888	0.931674	0.006244	1759.0	129.0	0.006488	0.004131	0.451284	4.196251e-03	0.064117	0.022189
24	(76576.32, 79767.0]	1602	0.925094	0.005298	1482.0	120.0	0.005466	0.003843	0.352252	6.580096e-03	0.099032	0.022189

In [82]:

```
plot_by_woe(df_temp, 90)
```



In [83]:

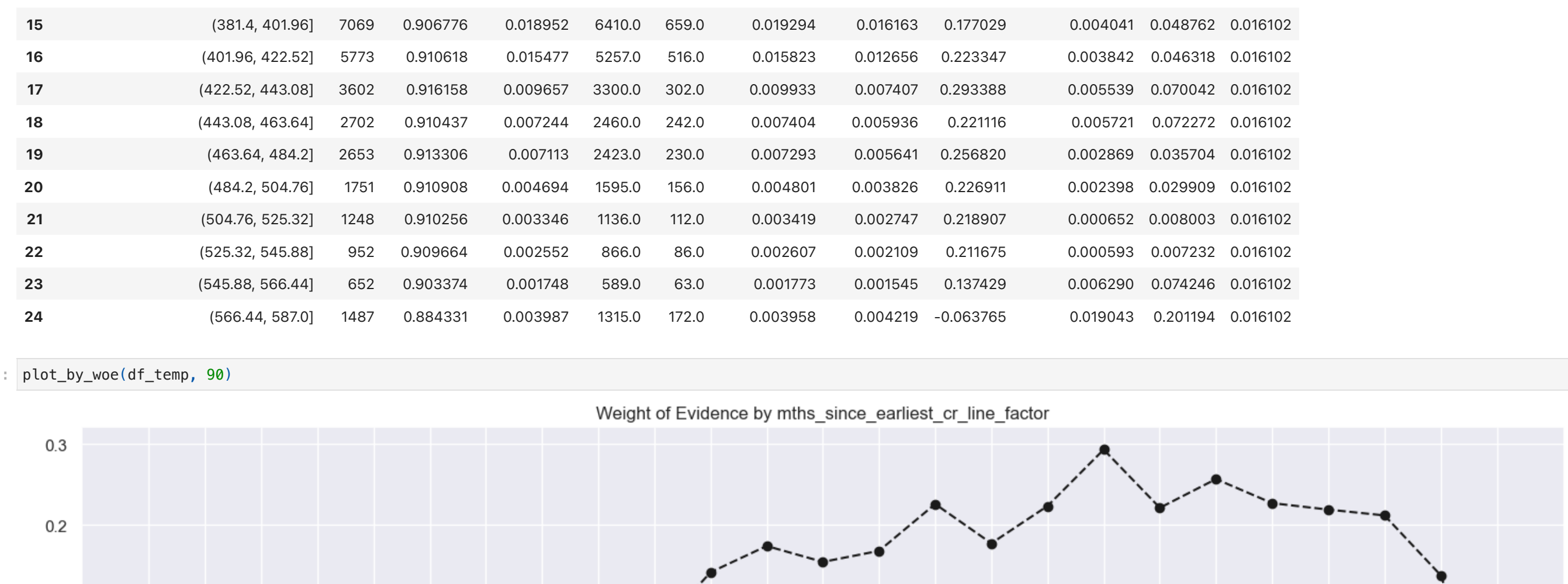
```
# fine-classing
X_train_prepr['mths_since_earliest_cr_line_factor'] = pd.cut(X_train_prepr['mths_since_earliest_cr_line'], 25)
# preprocess
df_temp = woe_ordered_continuous(X_train_prepr, 'mths_since_earliest_cr_line_factor', y_train_prepr)
df_temp
```

Out[83]:

	mths_since_earliest_cr_line_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(72.486, 93.56]	2650	0.895094	0.007105	2372.0	278.0	0.007140	0.006819	0.046005	NaN	NaN	0.016102
1	(93.56, 114.12]	6388	0.865529	0.017126	5529.0	859.0	0.016642	0.021069	-0.235869	0.029565	0.281874	0.016102
2	(114.12, 134.68]	15672	0.862557	0.042016	13518.0	2154.0	0.040688	0.052832	-0.261167	0.002972	0.025298	0.016102
3	(134.68, 155.24]	23403	0.873948	0.062742	20453.0	2950.0	0.061562	0.072355	-0.161538	0.011390	0.099629	0.016102
4	(155.24, 175.8]	30963	0.882860	0.083010	27336.0	3627.0	0.082280	0.088960	-0.078064	0.008912	0.083474	0.016102
5	(175.8, 196.36]	41418	0.886088	0.111039	36700.0	4718.0	0.110465	0.115720	-0.046471	0.003228	0.031593	0.016102
6	(196.36, 216.92]	45873	0.884071	0.122983	40555.0	5318.0	0.122068	0.130436	-0.066301	0.002017	0.019830	0.016102
7	(216.92, 237.48]	43012	0.888403	0.115313	38212.0	4800.0	0.115016	0.117731	-0.023329	0.004332	0.042972	0.016102
8	(237.48, 258.04]	33146	0.891390	0.088863	29546.0	3600.0	0.088932	0.088298	0.007152	0.002986	0.030481	0.016102
9	(258.04, 278.6]	28884	0.896794	0.077436	25903.0	2981.0	0.077967	0.073116	0.064238	0.005404	0.057085	0.016102
10	(278.6, 299.16]	23446	0.903694	0.062857	21188.0	2258.0	0.063775	0.055383	0.141093	0.006900	0.076856	0.016102
11	(299.16, 319.72]	14609	0.906496	0.039166	13243.0	1366.0	0.039861	0.033504	0.173720	0.002802	0.032627	0.016102
12	(319.72, 340.28]	14959	0.904806	0.040104	13535.0	1424.0	0.040740	0.034927	0.153947	0.001690	0.019773	0.016102
13	(340.28, 360.84]	11317	0.905982	0.030340	10253.0	1064.0	0.030861	0.026097	0.167672	0.001176	0.013726	0.016102
14	(360.84, 381.4]	9374	0.910817	0.025131	8538.0	836.0	0.025699	0.020505	0.225791	0.004835	0.058118	0.016102
15	(381.4, 401.96]	7069	0.906776	0.018952	6410.0	659.0	0.019294	0.016163	0.177029	0.004041	0.048762	0.016102
16	(401.96, 422.52]	5773	0.910618	0.015477	5257.0	516.0	0.015823	0.012656	0.223347	0.003842	0.046318	0.016102
17	(422.52, 443.08]	3602	0.916158	0.009657	3300.0	302.0	0.009933	0.007407	0.293388	0.005539	0.070042	0.016102
18	(443.08, 463.64]	2702	0.910437	0.007244	2460.0	242.0	0.007404	0.005936	0.221116	0.005721	0.072272	0.016102
19	(463.64, 484.2]	2653	0.913306	0.007113	2423.0	230.0	0.007293	0.005641	0.256820	0.002869	0.035704	0.016102
20	(484.2, 504.76]	1751	0.910908	0.004694	1595.0	156.0	0.004801	0.003826	0.226911	0.002398	0.029909	0.016102
21	(504.76, 525.32]	1248	0.910256	0.003346	1136.0	112.0	0.003419	0.002747	0.218907	0.000652	0.008003	0.016102
22	(525.32, 545.88]	952	0.909664	0.002552	866.0	86.0	0.002607	0.002109	0.211675	0.000593	0.007232	0.016102
23	(545.88, 566.44]	652	0.903374	0.001748	589.0	63.0	0.001773	0.001545	0.137429	0.006290	0.074246	0.016102
24	(566.44, 587.0]	1487	0.884331	0.003987	1315.0	172.0	0.003958	0.004219	-0.063765	0.019043	0.201194	0.016102

In [84]:

```
plot_by_woe(df_temp, 90)
```



In [85]:

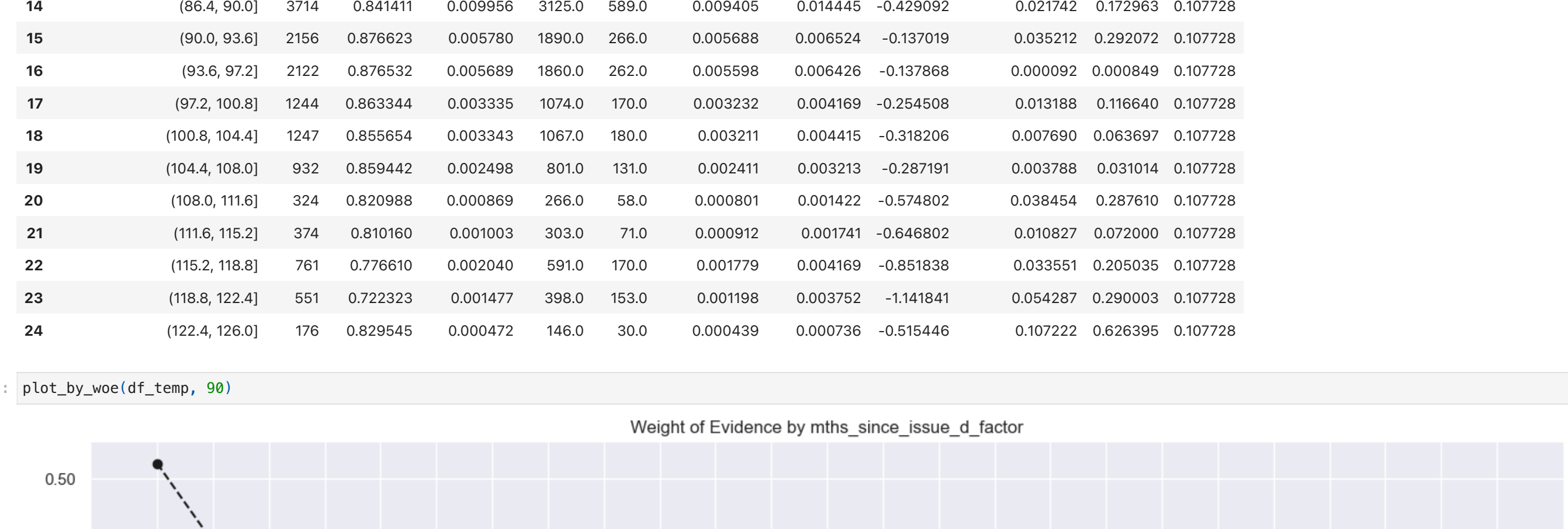
```
# fine-classing
X_train_prepr['mths_since_issue_d_factor'] = pd.cut(X_train_prepr['mths_since_issue_d'], 25)
# preprocess
df_temp = woe_ordered_continuous(X_train_prepr, "mths_since_issue_d_factor", y_train_prepr)
df_temp
```

Out[85]:

	mths_since_issue_d_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(35.91, 39.6]	67865	0.934311	0.181930	63407.0	4458.0	0.190839	0.109334	0.557019	NaN	NaN	0.107728
1	(39.6, 43.2]	67434	0.913145	0.180775	61577.0	5857.0	0.185331	0.143645	0.254796	0.021166	0.302223	0.107728
2	(43.2, 46.8]	40680	0.900959	0.109053	36651.0	4029.0	0.110310	0.098813	0.110067	0.012186	0.144728	0.107728
3	(46.8, 50.4]	47619	0.892837	0.127655	42516.0	5103.0	0.127962	0.125153	0.022197	0.008122	0.087871	0.107728
4	(50.4, 54.0]	38870	0.874968	0.104201	34010.0	4860.0	0.102361	0.119194	-0.152239	0.017869	0.174436	0.107728
5	(54.0, 57.6]	22337	0.862023	0.059880	19255.0	3082.0	0.057953	0.075587	-0.265663	0.012945	0.113424	0.107728
6	(57.6, 61.2]	21530	0.856015	0.057717	18430.0	3100.0	0.055470	0.076029	-0.315277	0.006008	0.049614	0.107728
7	(61.2, 64.8]	14249	0.849954	0.038198	12111.0	2138.0	0.036451	0.052435	-0.363612	0.006060	0.048334	0.107728
8	(64.8, 68.4]	11976	0.832749	0.032105	9973.0	2003.0	0.030016	0.049124	-0.492620	0.017206	0.129008	0.107728
9	(68.4, 72.0]	8304	0.834176	0.022261	6927.0	1377.0	0.020849	0.033772	-0.482336	0.001427	0.010284	0.107728
10	(72.0, 75.6]	5085	0.851524	0.013632	4330.0	755.0	0.013032	0.018517	-0.351250	0.017348	0.131085	0.107728
11	(75.6, 79.2]	5880	0.849660	0.015763	4996.0	884.0	0.015037	0.021680	-0.365919	0.001864	0.014669	0.107728
12	(79.2, 82.8]	3462	0.860774	0.009281	2980.0	482.0	0.008969	0.011821	-0.276121	0.011114	0.089799	0.107728
13	(82.8, 86.4]	4136	0.863153	0.011088	3570.0	566.0	0.010745	0.013881	-0.256128	0.002379	0.019992	0.107728
14	(86.4, 90.0]	3714	0.841411	0.009956	3125.0	589.0	0.009405	0.014445	-0.429092	0.021742	0.172963	0.107728
15	(90.0, 93.6]	2156	0.876623	0.005780	1890.0	266.0	0.005688	0.006524	-0.137019	0.035212	0.292072	0.107728
16	(93.6, 97.2]	2122	0.876532	0.005689	1860.0	262.0	0.005598	0.006426	-0.137868	0.000092	0.000849	0.107728
17	(97.2, 100.8]	1244	0.863344	0.003335	1074.0	170.0	0.003232	0.004169	-0.254508	0.013188	0.116640	0.107728
18	(100.8, 104.4]	1247	0.855654	0.003343	1067.0	180.0	0.003211	0.004415	-0.318206	0.007690	0.063697	0.107728
19	(104.4, 108.0]	932	0.859442	0.002498	801.0	131.0	0.002411	0.003213	-0.287191	0.003788	0.031014	0.107728
20	(108.0, 111.6]	324	0.820988	0.000869	266.0	58.0	0.000801	0.001422	-0.574802	0.038454	0.287610	0.107728
21	(111.6, 115.2]	374	0.810160	0.001003	303.0	71.0	0.000912	0.001741	-0.646802	0.010827	0.072000	0.107728
22	(115.2, 118.8]	761	0.776610	0.002040	591.0	170.0	0.001779	0.004169	-0.851838	0.033551	0.205035	0.107728
23	(118.8, 122.4]	551	0.722323	0.001477	398.0	153.0	0.001198	0.003752	-1.141841	0.054287	0.290003	0.107728
24	(122.4, 126.0]	176	0.829545	0.000472	146.0	30.0	0.000439	0.000736	-0.515446	0.107222	0.626395	0.107728

In [86]:

```
plot_by_woe(df_temp, 90)
```



In [87]:

```
# fine-classing
X_train_prepr['mths_since_last_pymnt_d_factor'] = pd.cut(X_train_prepr['mths_since_last_pymnt_d'], 25)
# preprocess
df_temp = woe_ordered_continuous(X_train_prepr, "mths_since_last_pymnt_d_factor", y_train_prepr)
df_temp
```

Out[87]:

	mths_since_last_pymnt_d_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(22.903, 26.88]	209784	0.977849	0.562851	205137.0	4647.0	0.617410	0.114846	1.681944	NaN	NaN	1.281457
1	(26.88, 30.76]	33582	0.787922	0.090101	26460.0	7122.0	0.079638	0.176013	-0.793066	0.189927	2.475011	1.281457
2	(30.76, 34.64]	30532	0.798637	0.081917	24384.0	6148.0	0.073390	0.151941	-0.727711	0.010715	0.065355	1.281457
3	(34.64, 38.52]	23842	0.781394	0.063968	18630.0	5212.0	0.056072	0.128809	-0.831702	0.017243	0.103991	1.281457
4	(38.52, 42.4]	19634	0.770908	0.052678	15136.0	4498.0	0.045556	0.111163	-0.892069	0.010487	0.060366	1.281457
5	(42.4, 46.28]	13743	0.787164	0.036872	10818.0	2925.0	0.032559	0.072288	-0.797595	0.016257	0.094474	1.281457
6	(46.28, 50.16]	9743	0.754080	0.026140	7347.0	2396.0	0.022113	0.059215	-0.985020	0.033085	0.187426	1.281457
7	(50.16, 54.04]	7548	0.740991	0.020251	5593.0	1955.0	0.016834	0.048316	-1.054386	0.013089	0.069366	1.281457
8	(54.04, 57.92]	4913	0.802565	0.013182	3943.0	970.0	0.011867	0.023973	-0.703111	0.061574	0.351275	1.281457
9	(57.92, 61.8]	4373	0.771553	0.011733	3374.0	999.0	0.010155	0.024689	-0.888412	0.031012	0.185302	1.281457
10	(61.8, 65.68]	3619	0.760155	0.009710	2751.0	868.0	0.008280	0.021452	-0.951984	0.011398	0.063571	1.281457
11	(65.68, 69.56]	2836	0.806417	0.007609	2287.0	549.0	0.006883	0.013568	-0.678614	0.046263	0.273370	1.281457
12	(69.56, 73.44]	1954	0.786592	0.005243	1537.0	417.0	0.004626	0.010306	-0.801010	0.019826	0.122396	1.281457
13	(73.44, 77.32]	1529	0.746239	0.004102	1141.0	388.0	0.003434	0.009589	-1.026857	0.040352	0.225847	1.281457
14	(77.32, 81.2]	1617	0.823748	0.004338	1332.0	285.0	0.004009	0.007043	-0.563564	0.077508	0.463293	1.281457
15	(81.2, 85.08]	1100	0.815455	0.002951	897.0	203.0	0.002700	0.005017	-0.619662	0.008293	0.056098	1.281457
16	(85.08, 88.96]	557	0.716338	0.001494	399.0	158.0	0.001201	0.003905	-1.179145	0.099117	0.559483	1.281457
17	(88.96, 92.84]	549	0.695811	0.001473	382.0	167.0	0.001150	0.004127	-1.278085	0.020527	0.098940	1.281457
18	(92.84, 96.72]	420	0.659524	0.001127	277.0	143.0	0.000834	0.003534	-1.444339	0.036287	0.166254	1.281457
19	(96.72, 100.6]	226	0.588496	0.000606	133.0	93.0	0.000400	0.002298	-1.747762	0.071028	0.303423	1.281457
20	(100.6, 104.48]	219	0.515982	0.000588	113.0	106.0	0.000340	0.002620	-2.041563	0.072514	0.293801	1.281457
21	(104.48, 108.36]	161	0.478261	0.000432	77.0	84.0	0.000232	0.002076	-2.192523	0.037721	0.150960	1.281457
22	(108.36, 112.24]	118	0.423729	0.000317	50.0	68.0	0.000150	0.001681	-2.412996	0.054532	0.220473	1.281457
23	(112.24, 116.12]	87	0.471264	0.000233	41.0	46.0	0.000123	0.001137	-2.220581	0.047536	0.192415	1.281457
24	(116.12, 120.0]	31	0.483871	0.000083	15.0	16.0	0.000045	0.000395	-2.170050	0.012607	0.050531	1.281457

Ignore this given the abnormally high IV value

In [88]:

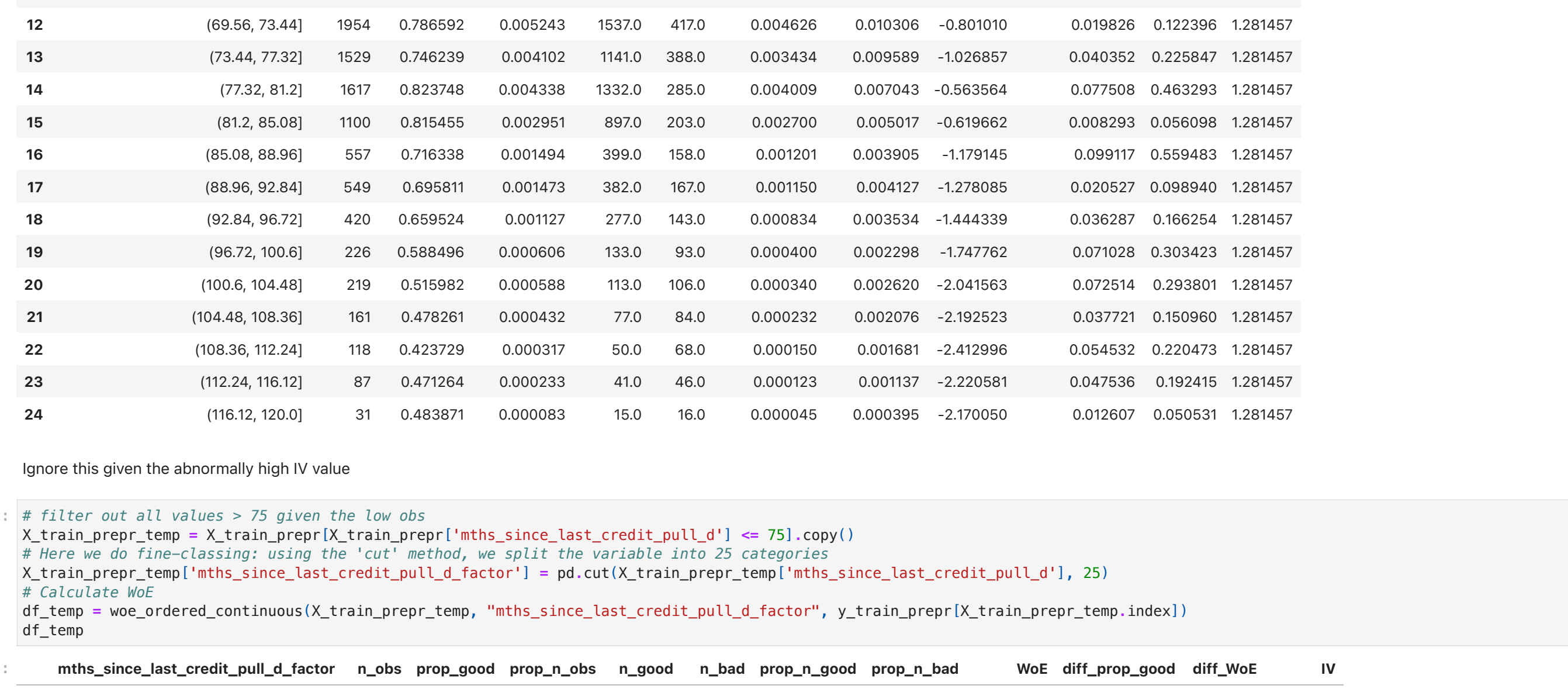
```
# filter out all values > 75 given the low obs
X_train_prepr_temp = X_train_prepr[X_train_prepr['mths_since_last_credit_pull_d'] <= 75].copy()
# Here we do fine-classing: using the 'cut' method, we split the variable into 25 categories
X_train_prepr_temp['mths_since_last_credit_pull_d_factor'] = pd.cut(X_train_prepr_temp['mths_since_last_credit_pull_d'], 25)
# Calculate WoE
df_temp = woe_ordered_continuous(X_train_prepr_temp, "mths_since_last_credit_pull_d_factor", y_train_prepr[X_train_prepr_temp.index])
df_temp
```

Out[88]:

	mths_since_last_credit_pull_d_factor	n_obs	prop_good	prop_n_obs	n_good	n_bad	prop_n_good	prop_n_bad	WoE	diff_prop_good	diff_WoE	IV
0	(22.948, 25.08]	280106	0.916607	0.756600	256747.0	23359.0	0.778123	0.580204	0.293505	NaN	NaN	0.243681
1	(25.08, 27.16]	12807	0.931600	0.034593	11931.0	876.0	0.036159	0.021759	0.507925	0.014993	0.214420	0.243681
2	(27.16, 29.24]	11183	0.847000	0.030207	9472.0	1711.0	0.028707	0.042499	-0.392342	0.084600	0.900267	0.243681
3	(29.24, 31.32]	9219	0.835991	0.024902	7707.0	1512.0	0.023358	0.037556	-0.474908	0.011009	0.082566	0.243681
4	(31.32, 33.4]	8887	0.809722	0.024005	7196.0	1691.0	0.021809	0.042002	-0.655399	0.026269	0.180490	0.243681
5	(33.4, 35.48]	8002	0.782304	0.021614	6260.0	1742.0	0.018972	0.043269	-0.824458	0.027418	0.169059	0.243681
6	(35.48, 37.56]	5476	0.747991	0.014791	4096.0	1380.0	0.012414	0.034277	-1.015677	0.034313	0.191219	0.243681
7	(37.56, 39.64]	5098	0.789525	0.013770	4025.0	1073.0	0.012199	0.026652	-0.781537	0.041534	0.234139	0.243681
8	(39.64, 41.72]	4491	0.790024	0.012131	3548.0	943.0	0.010753	0.023423	-0.778531	0.000499	0.003007	0.243681
9	(41.72, 43.8]	3604	0.769700	0.009735	2774.0	830.0	0.008407	0.020616	-0.896984	0.020324	0.118453	0.243681
10	(43.8, 45.88]	3512	0.750569	0.009486	2636.0	876.0	0.007989	0.021759	-1.001952	0.019131	0.104968	0.243681
11	(45.88, 47.96]	2955	0.719120	0.007982	2125.0	830.0	0.006440	0.020616	-1.163503	0.031449	0.161550	0.243681
12	(47.96, 50.04]	3256	0.720209	0.008795	2345.0	911.0	0.007107	0.022628	-1.158106	0.001089	0.005396	0.243681
13	(50.04, 52.12]	1694	0.772727	0.004576	1309.0	385.0	0.003967	0.009563	-0.879828	0.052518	0.278278	0.243681
14	(52.12, 54.2]	1485	0.783165	0.004011	1163.0	322.0	0.003525	0.007998	-0.819397	0.010438	0.060431	0.243681
15	(54.2, 56.28]	1452	0.764463	0.003922	1110.0	342.0	0.003364	0.008495	-0.926299	0.018702	0.106902	0.243681
16	(56.28, 58.36]	1972	0.762170	0.005327	1503.0	469.0	0.004555	0.011649	-0.938988	0.002292	0.012689	0.243681
17	(58.36, 60.44]	524	0.755725	0.001415	396.0	128.0	0.001200	0.003179	-0.974220	0.006445	0.035232	0.243681
18	(60.44, 62.52]	871	0.768083	0.002353	669.0	202.0	0.002028	0.005017	-0.906088	0.012357	0.068132	0.243681
19	(62.52, 64.6]	826	0.811138	0.002231	670.0	156.0	0.002031	0.003875	-0.646182	0.043055	0.259905	0.243681
20	(64.6, 66.68]	603	0.840796	0.001629	507.0	96.0	0.001537	0.002385	-0.439441	0.029658	0.206741	0.243681
21	(66.68, 68.76]	592	0.854730	0.001599	506.0	86.0	0.001534	0.002136	-0.331415	0.013934	0.108027	0.243681
22	(68.76, 70.84]	684	0.820175	0.001848	561.0	123.0	0.001700	0.003055	-0.586067	0.034554	0.254653	0.243681
23	(70.84, 72.92]	435	0.744828	0.001175	324.0	111.0	0.000982	0.002757	-1.032391	0.075348	0.446323	0.243681
24	(72.92, 75.0]	483	0.780538	0.001305	377.0	106.0	0.001143	0.002633	-0.834798	0.035711	0.197593	0.243681

In [89]:

```
plot_by_woe(df_temp, 90)
```



Define Custom Class for WoE Binning/Reengineering

In [90]:

```
# create a list of all the reference categories, i.e. one category from each of the global features
ref_categories = ['mths_since_last_credit_pull_d:>75', 'mths_since_issue_d:>122', 'mths_since_earliest_cr_line:>434', 'total_rev_hi_lim:>79,780',
                  'total_rec_int:>7,260', 'total_pymnt:>25,000', 'out_prncp:>15,437', 'revol_util:>1.0', 'inq_last_6mths:>4', 'dti:>35.191',
                  'annual_inc:>150K', 'int_rate:>20.281', 'term:60', 'purpose:major_purch__car__home_impr', 'verification_status:Not Verified',
                  'home_ownership:MORTGAGE', 'grade:G']
```

In [91]:

```
# This custom class will create new categorical dummy features based on the cut-off points that we manually identified
# based on the WoE plots and IV above.
# Given the way it is structured, this class also allows a fit_transform method to be implemented on it, thereby allowing
# us to use it as part of a scikit-learn Pipeline
class WoE_Binning(BaseEstimator, TransformerMixin):
    def __init__(self, X): # no *args or *kargs
        self.X = X
    def fit(self, X, y = None):
        return self #nothing else to do
    def transform(self, X):
        X_new = X.loc[:, 'grade:A': 'grade:G']
        X_new['home_ownership:OWN'] = X.loc[:,'home_ownership:OWN']
        X_new['home_ownership:MORTGAGE'] = X.loc[:,'home_ownership:MORTGAGE']
        X_new['home_ownership:OTHER_NONE_RENT'] = sum([X['home_ownership:OTHER'], X['home_ownership:NONE'], X['home_ownership:RENT']])
        X_new = pd.concat([X_new, X.loc[:, 'verification_status:Not Verified':'verification_status:Verified']], axis = 1)
        # For the purpose of this column, we keep debt_consolidation (due to volume) and credit_card (due to unique characteristics) as separate cateogories
        # These categories have very few observations: educational, renewable_energy, vacation, house, wedding, car
        # car is the least risky so we will combine it with the other 2 least risky categories: home_improvement and major_purchase
        # educational, renewable_energy (both low observations) will be combined with small_business and moving
        # vacation, house and wedding (remaining 3 with low observations) will be combined with medical and other
        X_new['purpose:debt_consolidation'] = X.loc[:,'purpose:debt_consolidation']
        X_new['purpose:credit_card'] = X.loc[:,'purpose:credit_card']
        X_new['purpose:major_purch__car__home_impr'] = sum([X['purpose:major_purchase'], X['purpose:car'], X['purpose:home_improvement']])
        X_new['purpose:educ__ren_en__sm_b__mov'] = sum([X['purpose:educational'], X['purpose:renewable_energy'], X['purpose:small_business'],
                                                        X['purpose:moving']])
        X_new['purpose:vacation__house__wedding__med__oth'] = sum([X['purpose:vacation'], X['purpose:house'], X['purpose:wedding'],
                                                                   X['purpose:medical'], X['purpose:other']])
        X_new['term:36'] = np.where((X['term'] == 36), 1, 0)
        X_new['term:60'] = np.where((X['term'] == 60), 1, 0)
        X_new['int_rate:<7.071'] = np.where((X['int_rate'] <= 7.071), 1, 0)
        X_new['int_rate:7.071-10.374'] = np.where((X['int_rate'] > 7.071) & (X['int_rate'] <= 10.374), 1, 0)
        X_new['int_rate:10.374-13.676'] = np.where((X['int_rate'] > 10.374) & (X['int_rate'] <= 13.676), 1, 0)
        X_new['int_rate:13.676-15.74'] = np.where((X['int_rate'] > 13.676) & (X['int_rate'] <= 15.74), 1, 0)
        X_new['int_rate:15.74-20.281'] = np.where((X['int_rate'] > 15.74) & (X['int_rate'] <= 20.281), 1, 0)
        X_new['int_rate:>20.281'] = np.where((X['int_rate'] > 20.281), 1, 0)
        X_new['annual_inc:missing'] = np.where(X['annual_inc'].isnull(), 1, 0)
        X_new['annual_inc:<28,555'] = np.where((X['annual_inc'] <= 28555), 1, 0)
        X_new['annual_inc:28,555-37,440'] = np.where((X['annual_inc'] > 28555) & (X['annual_inc'] <= 37440), 1, 0)
        X_new['annual_inc:37,440-61,137'] = np.where((X['annual_inc'] > 37440) & (X['annual_inc'] <= 61137), 1, 0)
        X_new['annual_inc:61,137-81,872'] = np.where((X['annual_inc'] > 61137) & (X['annual_inc'] <= 81872), 1, 0)
        X_new['annual_inc:81,872-102,606'] = np.where((X['annual_inc'] > 81872) & (X['annual_inc'] <= 102606), 1, 0)
        X_new['annual_inc:102,606-120,379'] = np.where((X['annual_inc'] > 102606) & (X['annual_inc'] <= 120379), 1, 0)
        X_new['annual_inc:120,379-150,000'] = np.where((X['annual_inc'] > 120379) & (X['annual_inc'] <= 150000), 1, 0)
        X_new['annual_inc:>150K'] = np.where((X['annual_inc'] > 150000), 1, 0)
        X_new['dti:<=1.6'] = np.where((X['dti'] <= 1.6), 1, 0)
        X_new['dti:1.6-5.599'] = np.where((X['dti'] > 1.6) & (X['dti'] <= 5.599), 1, 0)
        X_new['dti:5.599-10.397'] = np.where((X['dti'] > 5.599) & (X['dti'] <= 10.397), 1, 0)
        X_new['dti:10.397-15.196'] = np.where((X['dti'] > 10.397) & (X['dti'] <= 15.196), 1, 0)
        X_new['dti:15.196-19.195'] = np.where((X['dti'] > 15.196) & (X['dti'] <= 19.195), 1, 0)
        X_new['dti:19.195-24.794'] = np.where((X['dti'] > 19.195) & (X['dti'] <= 24.794), 1, 0)
        X_new['dti:24.794-35.191'] = np.where((X['dti'] > 24.794) & (X['dti'] <= 35.191), 1, 0)
        X_new['dti:>35.191'] = np.where((X['dti'] > 35.191), 1, 0)
        X_new['inq_last_6mths:missing'] = np.where(X['inq_last_6mths'].isnull(), 1, 0)
        X_new['inq_last_6mths:0'] = np.where((X['inq_last_6mths'] == 0), 1, 0)
        X_new['inq_last_6mths:1-2'] = np.where((X['inq_last_6mths'] >= 1) & (X['inq_last_6mths'] <= 2), 1, 0)
        X_new['inq_last_6mths:3-4'] = np.where((X['inq_last_6mths'] >= 3) & (X['inq_last_6mths'] <= 4), 1, 0)
        X_new['inq_last_6mths:>4'] = np.where((X['inq_last_6mths'] > 4), 1, 0)
        # We will discretize on the deciles for revol_util
        X_new['revol_util:missing'] = np.where(X['revol_util'].isnull(), 1, 0)
        X_new['revol_util:<0.1'] = np.where((X['revol_util'] <= 0.1), 1, 0)
        X_new['revol_util:0.1-0.2'] = np.where((X['revol_util'] > 0.1) & (X['revol_util'] <= 0.2), 1, 0)
        X_new['revol_util:0.2-0.3'] = np.where((X['revol_util'] > 0.2) & (X['revol_util'] <= 0.3), 1, 0)
        X_new['revol_util:0.3-0.4'] = np.where((X['revol_util'] > 0.3) & (X['revol_util'] <= 0.4), 1, 0)
        X_new['revol_util:0.4-0.5'] = np.where((X['revol_util'] > 0.4) & (X['revol_util'] <= 0.5), 1, 0)
        X_new['revol_util:0.5-0.6'] = np.where((X['revol_util'] > 0.5) & (X['revol_util'] <= 0.6), 1, 0)
        X_new['revol_util:0.6-0.7'] = np.where((X['revol_util'] > 0.6) & (X['revol_util'] <= 0.7), 1, 0)
        X_new['revol_util:0.7-0.8'] = np.where((X['revol_util'] > 0.7) & (X['revol_util'] <= 0.8), 1, 0)
        X_new['revol_util:0.8-0.9'] = np.where((X['revol_util'] > 0.8) & (X['revol_util'] <= 0.9), 1, 0)
        X_new['revol_util:0.9-1.0'] = np.where((X['revol_util'] > 0.9) & (X['revol_util'] <= 1.0), 1, 0)
        X_new['revol_util:>1.0'] = np.where((X['revol_util'] > 1.0), 1, 0)
        X_new['out_prncp:<1,286'] = np.where((X['out_prncp'] <= 1286), 1, 0)
        X_new['out_prncp:1,286-6,432'] = np.where((X['out_prncp'] > 1286) & (X['out_prncp'] <= 6432), 1, 0)
        X_new['out_prncp:6,432-9,005'] = np.where((X['out_prncp'] > 6432) & (X['out_prncp'] <= 9005), 1, 0)
        X_new['out_prncp:9,005-10,291'] = np.where((X['out_prncp'] > 9005) & (X['out_prncp'] <= 10291), 1, 0)
        X_new['out_prncp:10,291-15,437'] = np.where((X['out_prncp'] > 10291) & (X['out_prncp'] <= 15437), 1, 0)
        X_new['out_prncp:>15,437'] = np.where((X['out_prncp'] > 15437), 1, 0)
        X_new['total_pymnt:<10,000'] = np.where((X['total_pymnt'] <= 10000), 1, 0)
        X_new['total_pymnt:10,000-15,000'] = np.where((X['total_pymnt'] > 10000) & (X['total_pymnt'] <= 15000), 1, 0)
        X_new['total_pymnt:15,000-20,000'] = np.where((X['total_pymnt'] > 15000) & (X['total_pymnt'] <= 20000), 1, 0)
        X_new['total_pymnt:20,000-25,000'] = np.where((X['total_pymnt'] > 20000) & (X['total_pymnt'] <= 25000), 1, 0)
        X_new['total_pymnt:>25,000'] = np.where((X['total_pymnt'] > 25000), 1, 0)
        X_new['total_rec_int:<1,089'] = np.where((X['total_rec_int'] <= 1089), 1, 0)
        X_new['total_rec_int:1,089-2,541'] = np.where((X['total_rec_int'] > 1089) & (X['total_rec_int'] <= 2541), 1, 0)
        X_new['total_rec_int:2,541-4,719'] = np.where((X['total_rec_int'] > 2541) & (X['total_rec_int'] <= 4719), 1, 0)
        X_new['total_rec_int:4,719-7,260'] = np.where((X['total_rec_int'] > 4719) & (X['total_rec_int'] <= 7260), 1, 0)
        X_new['total_rec_int:>7,260'] = np.where((X['total_rec_int'] > 7260), 1, 0)
        X_new['total_rev_hi_lim:missing'] = np.where(X['total_rev_hi_lim'].isnull(), 1, 0)
        X_new['total_rev_hi_lim:<6,381'] = np.where((X['total_rev_hi_lim'] <= 6381), 1, 0)
        X_new['total_rev_hi_lim:6,381-19,144'] = np.where((X['total_rev_hi_lim'] > 6381) & (X['total_rev_hi_lim'] <= 19144), 1, 0)
        X_new['total_rev_hi_lim:19,144-25,525'] = np.where((X['total_rev_hi_lim'] > 19144) & (X['total_rev_hi_lim'] <= 25525), 1, 0)
        X_new['total_rev_hi_lim:25,525-35,097'] = np.where((X['total_rev_hi_lim'] > 25525) & (X['total_rev_hi_lim'] <= 35097), 1, 0)
        X_new['total_rev_hi_lim:35,097-54,241'] = np.where((X['total_rev_hi_lim'] > 35097) & (X['total_rev_hi_lim'] <= 54241), 1, 0)
        X_new['total_rev_hi_lim:54,241-79,780'] = np.where((X['total_rev_hi_lim'] > 54241) & (X['total_rev_hi_lim'] <= 79780), 1, 0)
        X_new['total_rev_hi_lim:>79,780'] = np.where((X['total_rev_hi_lim'] > 79780), 1, 0)
        X_new['mths_since_earliest_cr_line:missing'] = np.where(X['mths_since_earliest_cr_line'].isnull(), 1, 0)
        X_new['mths_since_earliest_cr_line:<125'] = np.where((X['mths_since_earliest_cr_line'] <= 125), 1, 0)
        X_new['mths_since_earliest_cr_line:125-167'] = np.where((X['mths_since_earliest_cr_line'] > 125) & (X['mths_since_earliest_cr_line'] <= 167), 1, 0)
        X_new['mths_since_earliest_cr_line:167-249'] = np.where((X['mths_since_earliest_cr_line'] > 167) & (X['mths_since_earliest_cr_line'] <= 249), 1, 0)
        X_new['mths_since_earliest_cr_line:249-331'] = np.where((X['mths_since_earliest_cr_line'] > 249) & (X['mths_since_earliest_cr_line'] <= 331), 1, 0)
        X_new['mths_since_earliest_cr_line:331-434'] = np.where((X['mths_since_earliest_cr_line'] > 331) & (X['mths_since_earliest_cr_line'] <= 434), 1, 0)
        X_new['mths_since_earliest_cr_line:>434'] = np.where((X['mths_since_earliest_cr_line'] > 434), 1, 0)
        X_new['mths_since_issue_d:<79'] = np.where((X['mths_since_issue_d'] <= 79), 1, 0)
        X_new['mths_since_issue_d:79-89'] = np.where((X['mths_since_issue_d'] > 79) & (X['mths_since_issue_d'] <= 89), 1, 0)
        X_new['mths_since_issue_d:89-100'] = np.where((X['mths_since_issue_d'] > 89) & (X['mths_since_issue_d'] <= 100), 1, 0)
        X_new['mths_since_issue_d:100-122'] = np.where((X['mths_since_issue_d'] > 100) & (X['mths_since_issue_d'] <= 122), 1, 0)
        X_new['mths_since_issue_d:>122'] = np.where((X['mths_since_issue_d'] > 122), 1, 0)
        X_new['mths_since_last_credit_pull_d:missing'] = np.where(X['mths_since_last_credit_pull_d'].isnull(), 1, 0)
        X_new['mths_since_last_credit_pull_d:<56'] = np.where((X['mths_since_last_credit_pull_d'] <= 56), 1, 0)
        X_new['mths_since_last_credit_pull_d:56-61'] = np.where((X['mths_since_last_credit_pull_d'] > 56) & (X['mths_since_last_credit_pull_d'] <= 61), 1, 0)
        X_new['mths_since_last_credit_pull_d:61-75'] = np.where((X['mths_since_last_credit_pull_d'] > 61) & (X['mths_since_last_credit_pull_d'] <= 75), 1, 0)
        X_new['mths_since_last_credit_pull_d:>75'] = np.where((X['mths_since_last_credit_pull_d'] > 75), 1, 0)
        X_new.drop(columns = ref_categories, inplace = True)
        return X_new
# we could have also structured this class without the last drop statement and without creating categories out of the
# feature categories. But doing the way we have done here allows us to keep a proper track of the categories, if required
```

In [92]:
```
# reconfirm shape of the 4 datasets
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(373028, 73)
(373028,)
(93257, 73)
(93257,)
```

Logistic Regression

In [93]:
```
# define modeling pipeline
reg = LogisticRegression(max_iter=1000, class_weight = 'balanced')
woe_transform = WoE_Binning(X)
pipeline = Pipeline(steps=[('woe', woe_transform), ('model', reg)])

# define cross-validation criteria. RepeatedStratifiedKFold automatially takes care of the class imbalance while splitting
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=1)

# fit and evaluate the logistic regression pipeline with cross-validation as defined in cv
scores = cross_val_score(pipeline, X_train, y_train, scoring = 'roc_auc', cv = cv)
AUROC = np.mean(scores)
GINI = AUROC * 2 - 1

# print the mean AUROC score and Gini
print('Mean AUROC: %.4f' % (AUROC))
print('Gini: %.4f' % (GINI))
```

```
Mean AUROC: 0.8630
Gini: 0.7259
```

In [94]:
```
# pretty impressive scores for the first time, now let's fit the pipeline on the whole training set
pipeline.fit(X_train, y_train)
```

Out[94]:
```
▸ Pipeline
  ▸ WoE_Binning
  ▸ LogisticRegression
```

In [95]:
```
# first create a transformed training set through our WoE_Binning custom class
X_train_woe_transformed = woe_transform.fit_transform(X_train)
# Store the column names in X_train as a list
feature_name = X_train_woe_transformed.columns.values
# Create a summary table of our logistic regression model
summary_table = pd.DataFrame(columns = ['Feature name'], data = feature_name)
# Create a new column in the dataframe, called 'Coefficients', with row values the transposed coefficients from the 'LogisticRegression' model
summary_table['Coefficients'] = np.transpose(pipeline['model'].coef_)
# Increase the index of every row of the dataframe with 1 to store our model intercept in 1st row
summary_table.index = summary_table.index + 1
# Assign our model intercept to this new row
summary_table.loc[0] = ['Intercept', pipeline['model'].intercept_[0]]
# Sort the dataframe by index
summary_table.sort_index(inplace = True)
summary_table
```

Out[95]:

	Feature name	Coefficients
0	Intercept	2.464030
1	grade:A	0.878816
2	grade:B	0.681447
3	grade:C	0.529299
4	grade:D	0.429916
...	...	...
80	mths_since_issue_d:100-122	0.517805
81	mths_since_last_credit_pull_d:missing	0.040285
82	mths_since_last_credit_pull_d:<56	0.394185
83	mths_since_last_credit_pull_d:56-61	-0.155123
84	mths_since_last_credit_pull_d:61-75	-0.009916

85 rows × 2 columns

In [96]:
```
with pd.option_context('display.max_rows', None,
                       'display.max_columns', None):
    display(summary_table)
```

	Feature name	Coefficients
0	Intercept	2.464030
1	grade:A	0.878816
2	grade:B	0.681447
3	grade:C	0.529299
4	grade:D	0.429916
5	grade:E	0.302952
6	grade:F	0.152838
7	home_ownership:OWN	-0.057182
8	home_ownership:OTHER_NONE_RENT	-0.107103
9	verification_status:Source Verified	-0.277094
10	verification_status:Verified	-0.474672
11	purpose:debt_consolidation	-0.313398
12	purpose:credit_card	-0.219243
13	purpose:educ__ren_en__sm_b__mov	-0.499964
14	purpose:vacation__house__wedding__med__oth	-0.001884
15	term:36	-0.074683
16	int_rate:<7.071	1.083360
17	int_rate:7.071-10.374	0.426055
18	int_rate:10.374-13.676	0.188460
19	int_rate:13.676-15.74	0.133012
20	int_rate:15.74-20.281	0.038217
21	annual_inc:missing	0.028251
22	annual_inc:<28,555	0.536420
23	annual_inc:28,555-37,440	0.461520
24	annual_inc:37,440-61,137	0.436298
25	annual_inc:61,137-81,872	0.398064
26	annual_inc:81,872-102,606	0.346064
27	annual_inc:102,606-120,379	0.208119
28	annual_inc:120,379-150,000	0.173450
29	dti:<=1.6	0.516757
30	dti:1.6-5.599	0.502709
31	dti:5.599-10.397	0.353762
32	dti:10.397-15.196	0.260120
33	dti:15.196-19.195	0.142084
34	dti:19.195-24.794	0.068490
35	dti:24.794-35.191	0.062238
36	inq_last_6mths:missing	0.083746
37	inq_last_6mths:0	0.407593
38	inq_last_6mths:1-2	0.367752
39	inq_last_6mths:3-4	0.267488
40	revol_util:missing	0.039821
41	revol_util:<0.1	-0.187724
42	revol_util:0.1-0.2	-0.201178
43	revol_util:0.2-0.3	0.136019
44	revol_util:0.3-0.4	-0.021381
45	revol_util:0.4-0.5	0.068239
46	revol_util:0.5-0.6	0.109897
47	revol_util:0.6-0.7	-0.054774
48	revol_util:0.7-0.8	0.013668
49	revol_util:0.8-0.9	-0.022090
50	revol_util:0.9-1.0	0.131379
51	out_prncp:<1,286	-3.477300
52	out_prncp:1,286-6,432	-1.011833
53	out_prncp:6,432-9,005	-0.603413
54	out_prncp:9,005-10,291	-0.372086
55	out_prncp:10,291-15,437	-0.258860
56	total_pymnt:<10,000	-6.168987
57	total_pymnt:10,000-15,000	-3.988031
58	total_pymnt:15,000-20,000	-2.876217
59	total_pymnt:20,000-25,000	-1.821992
60	total_rec_int:<1,089	4.154453
61	total_rec_int:1,089-2,541	3.625060
62	total_rec_int:2,541-4,719	2.531826
63	total_rec_int:4,719-7,260	1.281351
64	total_rev_hi_lim:missing	0.161101
65	total_rev_hi_lim:<6,381	0.437075
66	total_rev_hi_lim:6,381-19,144	0.365133
67	total_rev_hi_lim:19,144-25,525	0.256432
68	total_rev_hi_lim:25,525-35,097	0.136156
69	total_rev_hi_lim:35,097-54,241	0.087641
70	total_rev_hi_lim:54,241-79,780	0.027509
71	mths_since_earliest_cr_line:missing	0.083746
72	mths_since_earliest_cr_line:<125	-0.080289
73	mths_since_earliest_cr_line:125-167	-0.003298
74	mths_since_earliest_cr_line:167-249	-0.002011
75	mths_since_earliest_cr_line:249-331	0.034296
76	mths_since_earliest_cr_line:331-434	0.048567
77	mths_since_issue_d:<79	0.262352
78	mths_since_issue_d:79-89	0.652052
79	mths_since_issue_d:89-100	0.612261
80	mths_since_issue_d:100-122	0.517805
81	mths_since_last_credit_pull_d:missing	0.040285
82	mths_since_last_credit_pull_d:<56	0.394185
83	mths_since_last_credit_pull_d:56-61	-0.155123
84	mths_since_last_credit_pull_d:61-75	-0.009916

In [97]:
```
k = 10  # number of most important variables

top_k = (
    summary_table
    .loc[summary_table['Feature name'] != 'Intercept']          # drop intercept
    .assign(abs_coef=lambda d: d['Coefficients'].abs())
    .sort_values('abs_coef', ascending=False)
    .head(k)
)

top_k[['Feature name', 'Coefficients']]
```

Out[97]:

	Feature name	Coefficients
56	total_pymnt:<10,000	-6.168987
60	total_rec_int:<1,089	4.154453
57	total_pymnt:10,000-15,000	-3.988031
61	total_rec_int:1,089-2,541	3.625060
51	out_prncp:<1,286	-3.477300
58	total_pymnt:15,000-20,000	-2.876217
62	total_rec_int:2,541-4,719	2.531826
59	total_pymnt:20,000-25,000	-1.821992
63	total_rec_int:4,719-7,260	1.281351
16	int_rate:<7.071	1.083360

Prediction Time

In [98]:
```
# make preditions on our test set
y_hat_test = pipeline.predict(X_test)
# get the predicted probabilities
y_hat_test_proba = pipeline.predict_proba(X_test)
# select the probabilities of only the positive class (class 1 - default)
y_hat_test_proba = y_hat_test_proba[:][: , 1]
```

In [99]:
```
# create the table confusion matrix
cm = confusion_matrix(y_test, y_hat_test)
cm_df = pd.DataFrame(cm, index = ['Actual Negative', 'Actual Positive'], columns = ['Predicted Negative', 'Predicted Positive'])
cm_df
```

Out[99]:

	Predicted Negative	Predicted Positive
Actual Negative	8266	1928
Actual Positive	17751	65312

In [100…
```
# we will now create a new DF with actual classes and the predicted probabilities
# create a temp y_test DF to reset its index to allow proper concaternation with y_hat_test_proba
y_test_temp = y_test.copy()
y_test_temp.reset_index(drop = True, inplace = True)
y_test_proba = pd.concat([y_test_temp, pd.DataFrame(y_hat_test_proba)], axis = 1)
# check the shape to make sure the number of rows is same as that in y_test
y_test_proba.shape
```

Out[100…
```
(93257, 2)
```

In [101…
```
# Rename the columns
y_test_proba.columns = ['y_test_class_actual', 'y_hat_test_proba']
# Makes the index of one dataframe equal to the index of another dataframe.
y_test_proba.index = X_test.index
y_test_proba.head()
```

Out[101…

	y_test_class_actual	y_hat_test_proba
395346	1	0.309881
376583	1	0.900020
297790	1	0.595728
47347	1	0.811203
446772	0	0.115406

In [102…
```
#auc in the test set
auc_test = roc_auc_score(y_test_proba['y_test_class_actual'], y_test_proba['y_hat_test_proba'])
print('AUROC in the test set: %.4f' % (auc_test))
```

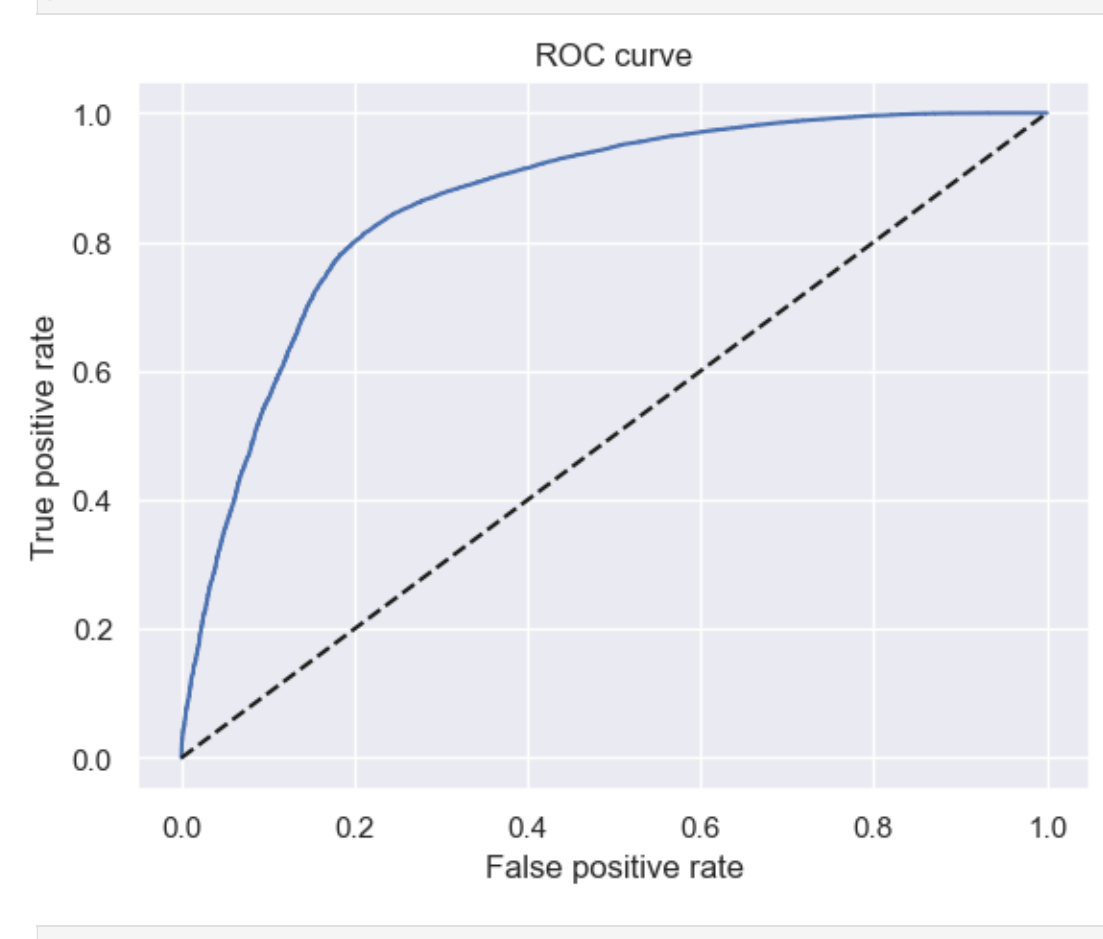
```
AUROC in the test set: 0.8629
```

In [103…
```
# assign a threshold value to differentiate good with bad
tr = 0.5
# crate a new column for the predicted class based on predicted probabilities and threshold
# We will determine this optimat threshold later in this project
y_test_proba['y_test_class_predicted'] = np.where(y_test_proba['y_hat_test_proba'] > tr, 1, 0)
# create the confusion matrix
confusion_matrix(y_test_proba['y_test_class_actual'], y_test_proba['y_test_class_predicted'], normalize = 'all')
```

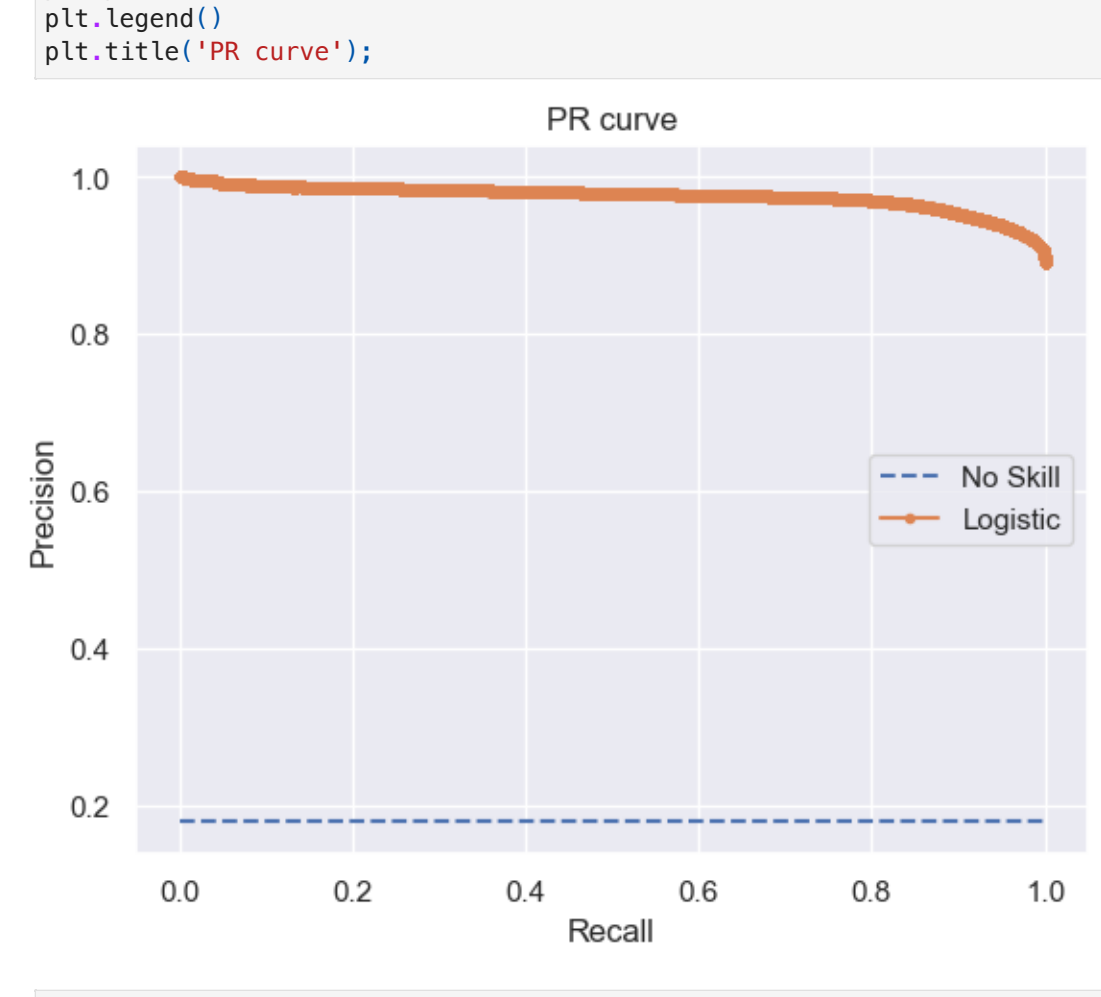
Out[103…
```
array([[0.08863678, 0.02067405],
       [0.19034496, 0.70034421]])
```

In [ ]:

In [104…
```
# get the values required to plot a ROC curve
fpr, tpr, thresholds = roc_curve(y_test_proba['y_test_class_actual'], y_test_proba['y_hat_test_proba'])
# plot the ROC curve
plt.plot(fpr, tpr)
# plot a secondary diagonal line, with dashed line style and black color to represent a no-skill classifier
plt.plot(fpr, fpr, linestyle = '--', color = 'k')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve');
```



In [105…
```
# draw a PR curve
# calculate the no skill line as the proportion of the positive class
no_skill = len(y_test[y_test == 1]) / len(y)
# plot the no skill precision-recall curve
plt.plot([0, 1], [no_skill, no_skill], linestyle='--', label='No Skill')

# calculate inputs for the PR curve
precision, recall, thresholds = precision_recall_curve(y_test_proba['y_test_class_actual'], y_test_proba['y_hat_test_proba'])
# plot PR curve
plt.plot(recall, precision, marker='.', label='Logistic')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.legend()
plt.title('PR curve');
```



In [106…
```
loan_data["issue_d_date"] = pd.to_datetime(loan_data["issue_d"], format="%b-%y", errors="coerce")
# z.B. "Dec-15" -> 2015-12-01 [web:128][web:134]

# 2) Stichtag (Dezember 2017)
present = pd.to_datetime("2017-12-01")

# 3) Monate seit Ausgabe berechnen
loan_data["mths_since_issue_d"] = (
    (present.year - loan_data["issue_d_date"].dt.year) * 12
    + (present.month - loan_data["issue_d_date"].dt.month)
)
# Optional runden / in int casten:
loan_data["mths_since_issue_d"] = loan_data["mths_since_issue_d"].astype("Int64")

# 4) Kontrolle
loan_data["mths_since_issue_d"].describe()
```

Out[106…
```
count    466285.0
mean      51.255187
std       14.340154
min            36.0
25%            41.0
50%            47.0
75%            57.0
max           126.0
Name: mths_since_issue_d, dtype: Float64
```

Score Card Creation

In [107…
```
summary_table
```

Out[107…

	Feature name	Coefficients
0	Intercept	2.464030
1	grade:A	0.878816
2	grade:B	0.681447
3	grade:C	0.529299
4	grade:D	0.429916
...	...	...
80	mths_since_issue_d:100-122	0.517805
81	mths_since_last_credit_pull_d:missing	0.040285
82	mths_since_last_credit_pull_d:<56	0.394185
83	mths_since_last_credit_pull_d:56-61	-0.155123
84	mths_since_last_credit_pull_d:61-75	-0.009916

85 rows × 2 columns

In [108…
```
ref_categories
```

Out[108…
```
['mths_since_last_credit_pull_d:>75',
 'mths_since_issue_d:>122',
 'mths_since_earliest_cr_line:>434',
 'total_rev_hi_lim:>79,780',
 'total_rec_int:>7,260',
 'total_pymnt:>25,000',
 'out_prncp:>15,437',
 'revol_util:>1.0',
 'inq_last_6mths:>4',
 'dti:>35.191',
 'annual_inc:>150K',
 'int_rate:>20.281',
 'term:60',
 'purpose:major_purch__car__home_impr',
 'verification_status:Not Verified',
 'home_ownership:MORTGAGE',
 'grade:G']
```

In [109…
```
# We create a new dataframe with one column. Its values are the values from the 'reference_categories' list. We name it 'Feature name'.
df_ref_categories = pd.DataFrame(ref_categories, columns = ['Feature name'])
# We create a second column, called 'Coefficients', which contains only 0 values.
df_ref_categories['Coefficients'] = 0
df_ref_categories
# Concatenates two dataframes.
df_scorecard = pd.concat([summary_table, df_ref_categories])
# We reset the index of a dataframe.
df_scorecard.reset_index(inplace = True)
df_scorecard
```

Out[109…

	index	Feature name	Coefficients
0	0	Intercept	2.464030
1	1	grade:A	0.878816
2	2	grade:B	0.681447
3	3	grade:C	0.529299
4	4	grade:D	0.429916
...	...	...	...
97	12	term:60	0.000000
98	13	purpose:major_purch__car__home_impr	0.000000
99	14	verification_status:Not Verified	0.000000
100	15	home_ownership:MORTGAGE	0.000000
101	16	grade:G	0.000000

102 rows × 3 columns

In [110…
```
# create a new column, called 'Original feature name', which contains the value of the 'Feature name' column, up to the column symbol.
df_scorecard['Original feature name'] = df_scorecard['Feature name'].str.split(':').str[0]
df_scorecard
```

Out[110…

	index	Feature name	Coefficients	Original feature name
0	0	Intercept	2.464030	Intercept
1	1	grade:A	0.878816	grade
2	2	grade:B	0.681447	grade
3	3	grade:C	0.529299	grade
4	4	grade:D	0.429916	grade
...	...	...	...	...
97	12	term:60	0.000000	term
98	13	purpose:major_purch__car__home_impr	0.000000	purpose
99	14	verification_status:Not Verified	0.000000	verification_status
100	15	home_ownership:MORTGAGE	0.000000	home_ownership
101	16	grade:G	0.000000	grade

102 rows × 4 columns

In [111…
```
# Define the min and max threshholds for our scorecard
min_score = 300
max_score = 850
```

In [112…
```
df_scorecard.groupby('Original feature name')['Coefficients'].min()
```

Out[112…
```
Original feature name
Intercept                      2.464030
annual_inc                     0.000000
dti                            0.000000
grade                          0.000000
home_ownership                -0.107103
inq_last_6mths                 0.000000
int_rate                       0.000000
mths_since_earliest_cr_line   -0.080289
mths_since_issue_d             0.000000
mths_since_last_credit_pull_d -0.155123
out_prncp                     -3.477300
purpose                       -0.499964
revol_util                    -0.201178
term                          -0.074683
total_pymnt                   -6.168987
total_rec_int                  0.000000
total_rev_hi_lim               0.000000
verification_status           -0.474672
Name: Coefficients, dtype: float64
```

In [113…
```
df_scorecard.groupby('Original feature name')['Coefficients'].max()
```

Out[113…
```
Original feature name
Intercept                      2.464030
annual_inc                     0.536420
dti                            0.516757
grade                          0.878816
home_ownership                 0.000000
inq_last_6mths                 0.407593
int_rate                       1.083360
mths_since_earliest_cr_line    0.083746
mths_since_issue_d             0.652052
mths_since_last_credit_pull_d  0.394185
out_prncp                      0.000000
purpose                        0.000000
revol_util                     0.136019
term                           0.000000
total_pymnt                    0.000000
total_rec_int                  4.154453
total_rev_hi_lim               0.437075
verification_status            0.000000
Name: Coefficients, dtype: float64
```

In [114…
```
#
min_sum_coef = df_scorecard.groupby('Original feature name')['Coefficients'].min().sum()
min_sum_coef
```

Out[114…
```
-8.775268513027768
```

In [115…
```
# calculate the sum of the minimum coefficients of each category within the original feature name
min_sum_coef = df_scorecard.groupby('Original feature name')['Coefficients'].min().sum()
# calculate the sum of the maximum coefficients of each category within the original feature name
max_sum_coef = df_scorecard.groupby('Original feature name')['Coefficients'].max().sum()
# create a new columns that has the imputed calculated Score based on the multiplication of the coefficient by the ratio of the differences between
# maximum & minimum score and maximum & minimum sum of cefficients.
df_scorecard['Score - Calculation'] = df_scorecard['Coefficients'] * (max_score - min_score) / (max_sum_coef - min_sum_coef)
# update the calculated score of the Intercept (i.e. the default score for each loan)
df_scorecard.loc[0, 'Score - Calculation'] = ((df_scorecard.loc[0,'Coefficients'] - min_sum_coef) / (max_sum_coef - min_sum_coef)) * (max_score - min_score) + min_score
# round the values of the 'Score - Calculation' column and store them in a new column
df_scorecard['Score - Preliminary'] = df_scorecard['Score - Calculation'].round()
df_scorecard
```

Out[115…

	index	Feature name	Coefficients	Original feature name	Score - Calculation	Score - Preliminary
0	0	Intercept	2.464030	Intercept	601.251544	601.0
1	1	grade:A	0.878816	grade	23.555270	24.0
2	2	grade:B	0.681447	grade	18.265114	18.0
3	3	grade:C	0.529299	grade	14.187006	14.0
4	4	grade:D	0.429916	grade	11.523224	12.0
...	...	...	...	...	...	...
97	12	term:60	0.000000	term	0.000000	0.0
98	13	purpose:major_purch__car__home_impr	0.000000	purpose	0.000000	0.0
99	14	verification_status:Not Verified	0.000000	verification_status	0.000000	0.0
100	15	home_ownership:MORTGAGE	0.000000	home_ownership	0.000000	0.0
101	16	grade:G	0.000000	grade	0.000000	0.0

102 rows × 6 columns

In [116…
```
# check the min and max possible scores of our scorecard
min_sum_score_prel = df_scorecard.groupby('Original feature name')['Score - Preliminary'].min().sum()
max_sum_score_prel = df_scorecard.groupby('Original feature name')['Score - Preliminary'].max().sum()
print(min_sum_score_prel)
print(max_sum_score_prel)
```

```
301.0
850.0
```

In [117…
```
pd.options.display.max_rows = 102
df_scorecard['Difference'] = df_scorecard['Score - Preliminary'] - df_scorecard['Score - Calculation']
df_scorecard
```

Out[117…

	index	Feature name	Coefficients	Original feature name	Score - Calculation	Score - Preliminary	Difference
0	0	Intercept	2.464030	Intercept	601.251544	601.0	-0.251544
1	1	grade:A	0.878816	grade	23.555270	24.0	0.444730
2	2	grade:B	0.681447	grade	18.265114	18.0	-0.265114
3	3	grade:C	0.529299	grade	14.187006	14.0	-0.187006
4	4	grade:D	0.429916	grade	11.523224	12.0	0.476776
5	5	grade:E	0.302952	grade	8.120155	8.0	-0.120155
6	6	grade:F	0.152838	grade	4.096580	4.0	-0.096580
7	7	home_ownership:OWN	-0.057182	home_ownership	-1.532682	-2.0	-0.467318
8	8	home_ownership:OTHER_NONE_RENT	-0.107103	home_ownership	-2.870714	-3.0	-0.129286
9	9	verification_status:Source Verified	-0.277094	verification_status	-7.427068	-7.0	0.427068
10	10	verification_status:Verified	-0.474672	verification_status	-12.722830	-13.0	-0.277170
11	11	purpose:debt_consolidation	-0.313398	purpose	-8.400143	-8.0	0.400143
12	12	purpose:credit_card	-0.219243	purpose	-5.876463	-6.0	-0.123537
13	13	purpose:educ__ren_en__sm_b__mov	-0.499964	purpose	-13.400729	-13.0	0.400729
14	14	purpose:vacation__house__wedding__med__oth	-0.001884	purpose	-0.050487	-0.0	0.050487
15	15	term:36	-0.074683	term	-2.001769	-2.0	0.001769
16	16	int_rate:<7.071	1.083360	int_rate	29.037736	29.0	-0.037736
17	17	int_rate:7.071-10.374	0.426055	int_rate	11.419720	11.0	-0.419720
18	18	int_rate:10.374-13.676	0.188460	int_rate	5.051378	5.0	-0.051378
19	19	int_rate:13.676-15.74	0.133012	int_rate	3.565164	4.0	0.434836
20	20	int_rate:15.74-20.281	0.038217	int_rate	1.024339	1.0	-0.024339
21	21	annual_inc:missing	0.028251	annual_inc	0.757234	1.0	0.242766
22	22	annual_inc:<28,555	0.536420	annual_inc	14.377886	14.0	-0.377886
23	23	annual_inc:28,555-37,440	0.461520	annual_inc	12.370323	12.0	-0.370323
24	24	annual_inc:37,440-61,137	0.436298	annual_inc	11.694272	12.0	0.305728
25	25	annual_inc:61,137-81,872	0.398064	annual_inc	10.669471	11.0	0.330529
26	26	annual_inc:81,872-102,606	0.346064	annual_inc	9.275699	9.0	-0.275699
27	27	annual_inc:102,606-120,379	0.208119	annual_inc	5.578296	6.0	0.421704
28	28	annual_inc:120,379-150,000	0.173450	annual_inc	4.649046	5.0	0.350954
29	29	dti:<=1.6	0.516757	dti	13.850860	14.0	0.149140
30	30	dti:1.6-5.599	0.502709	dti	13.474308	13.0	-0.474308
31	31	dti:5.599-10.397	0.353762	dti	9.482028	9.0	-0.482028
32	32	dti:10.397-15.196	0.260120	dti	6.972114	7.0	0.027886
33	33	dti:15.196-19.195	0.142084	dti	3.808325	4.0	0.191675
34	34	dti:19.195-24.794	0.068490	dti	1.835763	2.0	0.164237
35	35	dti:24.794-35.191	0.062238	dti	1.668189	2.0	0.331811
36	36	inq_last_6mths:missing	0.083746	inq_last_6mths	2.244682	2.0	-0.244682
37	37	inq_last_6mths:0	0.407593	inq_last_6mths	10.924894	11.0	0.075106
38	38	inq_last_6mths:1-2	0.367752	inq_last_6mths	9.857002	10.0	0.142998
39	39	inq_last_6mths:3-4	0.267488	inq_last_6mths	7.169590	7.0	-0.169590
40	40	revol_util:missing	0.039821	revol_util	1.067340	1.0	-0.067340
41	41	revol_util:<0.1	-0.187724	revol_util	-5.031655	-5.0	0.031655
42	42	revol_util:0.1-0.2	-0.201178	revol_util	-5.392260	-5.0	0.392260
43	43	revol_util:0.2-0.3	0.136019	revol_util	3.645783	4.0	0.354217
44	44	revol_util:0.3-0.4	-0.021381	revol_util	-0.573082	-1.0	-0.426918
45	45	revol_util:0.4-0.5	0.068239	revol_util	1.829049	2.0	0.170951
46	46	revol_util:0.5-0.6	0.109897	revol_util	2.945620	3.0	0.054380
47	47	revol_util:0.6-0.7	-0.054774	revol_util	-1.468132	-1.0	0.468132
48	48	revol_util:0.7-0.8	0.013668	revol_util	0.366336	0.0	-0.366336
49	49	revol_util:0.8-0.9	-0.022090	revol_util	-0.592083	-1.0	-0.407917
50	50	revol_util:0.9-1.0	0.131379	revol_util	3.521410	4.0	0.478590
51	51	out_prncp:<1,286	-3.477300	out_prncp	-93.203508	-93.0	0.203508
52	52	out_prncp:1,286-6,432	-1.011833	out_prncp	-27.120569	-27.0	0.120569
53	53	out_prncp:6,432-9,005	-0.603413	out_prncp	-16.173518	-16.0	0.173518
54	54	out_prncp:9,005-10,291	-0.372086	out_prncp	-9.973177	-10.0	-0.026823
55	55	out_prncp:10,291-15,437	-0.258860	out_prncp	-6.938318	-7.0	-0.061682
56	56	total_pymnt:<10,000	-6.168987	total_pymnt	-165.349908	-165.0	0.349908
57	57	total_pymnt:10,000-15,000	-3.988031	total_pymnt	-106.892844	-107.0	-0.107156
58	58	total_pymnt:15,000-20,000	-2.876217	total_pymnt	-77.092438	-77.0	0.092438
59	59	total_pymnt:20,000-25,000	-1.821992	total_pymnt	-48.835589	-49.0	-0.164411
60	60	total_rec_int:<1,089	4.154453	total_rec_int	111.353524	111.0	-0.353524
61	61	total_rec_int:1,089-2,541	3.625060	total_rec_int	97.163983	97.0	-0.163983
62	62	total_rec_int:2,541-4,719	2.531826	total_rec_int	67.861581	68.0	0.138419
63	63	total_rec_int:4,719-7,260	1.281351	total_rec_int	34.344582	34.0	-0.344582
64	64	total_rev_hi_lim:missing	0.161101	total_rev_hi_lim	4.318048	4.0	-0.318048
65	65	total_rev_hi_lim:<6,381	0.437075	total_rev_hi_lim	11.715092	12.0	0.284908
66	66	total_rev_hi_lim:6,381-19,144	0.365133	total_rev_hi_lim	9.786804	10.0	0.213196
67	67	total_rev_hi_lim:19,144-25,525	0.256432	total_rev_hi_lim	6.873246	7.0	0.126754
68	68	total_rev_hi_lim:25,525-35,097	0.136156	total_rev_hi_lim	3.649437	4.0	0.350563
69	69	total_rev_hi_lim:35,097-54,241	0.087641	total_rev_hi_lim	2.349080	2.0	-0.349080
70	70	total_rev_hi_lim:54,241-79,780	0.027509	total_rev_hi_lim	0.737337	1.0	0.262663
71	71	mths_since_earliest_cr_line:missing	0.083746	mths_since_earliest_cr_line	2.244682	2.0	-0.244682
72	72	mths_since_earliest_cr_line:<125	-0.080289	mths_since_earliest_cr_line	-2.152006	-2.0	0.152006
73	73	mths_since_earliest_cr_line:125-167	-0.003298	mths_since_earliest_cr_line	-0.088402	-0.0	0.088402
74	74	mths_since_earliest_cr_line:167-249	-0.002011	mths_since_earliest_cr_line	-0.053914	-0.0	0.053914
75	75	mths_since_earliest_cr_line:249-331	0.034296	mths_since_earliest_cr_line	0.919239	1.0	0.080761
76	76	mths_since_earliest_cr_line:331-434	0.048567	mths_since_earliest_cr_line	1.301774	1.0	-0.301774
77	77	mths_since_issue_d:<79	0.262352	mths_since_issue_d	7.031937	7.0	-0.031937
78	78	mths_since_issue_d:79-89	0.652052	mths_since_issue_d	17.477214	17.0	-0.477214
79	79	mths_since_issue_d:89-100	0.612261	mths_since_issue_d	16.410675	16.0	-0.410675
80	80	mths_since_issue_d:100-122	0.517805	mths_since_issue_d	13.878936	14.0	0.121064
81	81	mths_since_last_credit_pull_d:missing	0.040285	mths_since_last_credit_pull_d	1.079775	1.0	-0.079775
82	82	mths_since_last_credit_pull_d:<56	0.394185	mths_since_last_credit_pull_d	10.565514	11.0	0.434486
83	83	mths_since_last_credit_pull_d:56-61	-0.155123	mths_since_last_credit_pull_d	-4.157821	-4.0	0.157821
84	84	mths_since_last_credit_pull_d:61-75	-0.009916	mths_since_last_credit_pull_d	-0.265776	-0.0	0.265776
85	0	mths_since_last_credit_pull_d:>75	0.000000	mths_since_last_credit_pull_d	0.000000	0.0	0.000000
86	1	mths_since_issue_d:>122	0.000000	mths_since_issue_d	0.000000	0.0	0.000000
87	2	mths_since_earliest_cr_line:>434	0.000000	mths_since_earliest_cr_line	0.000000	0.0	0.000000
88	3	total_rev_hi_lim:>79,780	0.000000	total_rev_hi_lim	0.000000	0.0	0.000000
89	4	total_rec_int:>7,260	0.000000	total_rec_int	0.000000	0.0	0.000000
90	5	total_pymnt:>25,000	0.000000	total_pymnt	0.000000	0.0	0.000000
91	6	out_prncp:>15,437	0.000000	out_prncp	0.000000	0.0	0.000000
92	7	revol_util:>1.0	0.000000	revol_util	0.000000	0.0	0.000000
93	8	inq_last_6mths:>4	0.000000	inq_last_6mths	0.000000	0.0	0.000000
94	9	dti:>35.191	0.000000	dti	0.000000	0.0	0.000000
95	10	annual_inc:>150K	0.000000	annual_inc	0.000000	0.0	0.000000
96	11	int_rate:>20.281	0.000000	int_rate	0.000000	0.0	0.000000
97	12	term:60	0.000000	term	0.000000	0.0	0.000000
98	13	purpose:major_purch__car__home_impr	0.000000	purpose	0.000000	0.0	0.000000
99	14	verification_status:Not Verified	0.000000	verification_status	0.000000	0.0	0.000000
100	15	home_ownership:MORTGAGE	0.000000	home_ownership	0.000000	0.0	0.000000
101	16	grade:G	0.000000	grade	0.000000	0.0	0.000000

In [118…
```
df_scorecard.loc[18, 'Score - Final'] = 4
df_scorecard.loc[2, 'Score - Final'] = 18
```

In [119…
```
df_scorecard['Score - Final'] = df_scorecard['Score - Preliminary']
```

In [119…

```
df_scorecard['Score - Final'] = df_scorecard['Score - Preliminary']
df_scorecard
```

Out[119…

	index	Feature name	Coefficients	Original feature name	Score - Calculation	Score - Preliminary	Difference	Score - Final
0	0	Intercept	2.464030	Intercept	601.251544	601.0	-0.251544	601.0
1	1	grade:A	0.878816	grade	23.555270	24.0	0.444730	24.0
2	2	grade:B	0.681447	grade	18.265114	18.0	-0.265114	18.0
3	3	grade:C	0.529299	grade	14.187006	14.0	-0.187006	14.0
4	4	grade:D	0.429916	grade	11.523224	12.0	0.476776	12.0
5	5	grade:E	0.302952	grade	8.120155	8.0	-0.120155	8.0
6	6	grade:F	0.152838	grade	4.096580	4.0	-0.096580	4.0
7	7	home_ownership:OWN	-0.057182	home_ownership	-1.532682	-2.0	-0.467318	-2.0
8	8	home_ownership:OTHER_NONE_RENT	-0.107103	home_ownership	-2.870714	-3.0	-0.129286	-3.0
9	9	verification_status:Source Verified	-0.277094	verification_status	-7.427068	-7.0	0.427068	-7.0
10	10	verification_status:Verified	-0.474672	verification_status	-12.722830	-13.0	-0.277170	-13.0
11	11	purpose:debt_consolidation	-0.313398	purpose	-8.400143	-8.0	0.400143	-8.0
12	12	purpose:credit_card	-0.219243	purpose	-5.876463	-6.0	-0.123537	-6.0
13	13	purpose:educ__ren_en__sm_b__mov	-0.499964	purpose	-13.400729	-13.0	0.400729	-13.0
14	14	purpose:vacation__house__wedding__med__oth	-0.001884	purpose	-0.050487	-0.0	0.050487	-0.0
15	15	term:36	-0.074683	term	-2.001769	-2.0	0.001769	-2.0
16	16	int_rate:<7.071	1.083360	int_rate	29.037736	29.0	-0.037736	29.0
17	17	int_rate:7.071-10.374	0.426055	int_rate	11.419720	11.0	-0.419720	11.0
18	18	int_rate:10.374-13.676	0.188460	int_rate	5.051378	5.0	-0.051378	5.0
19	19	int_rate:13.676-15.74	0.133012	int_rate	3.565164	4.0	0.434836	4.0
20	20	int_rate:15.74-20.281	0.038217	int_rate	1.024339	1.0	-0.024339	1.0
21	21	annual_inc:missing	0.028251	annual_inc	0.757234	1.0	0.242766	1.0
22	22	annual_inc:<28,555	0.536420	annual_inc	14.377886	14.0	-0.377886	14.0
23	23	annual_inc:28,555-37,440	0.461520	annual_inc	12.370323	12.0	-0.370323	12.0
24	24	annual_inc:37,440-61,137	0.436298	annual_inc	11.694272	12.0	0.305728	12.0
25	25	annual_inc:61,137-81,872	0.398064	annual_inc	10.669471	11.0	0.330529	11.0
26	26	annual_inc:81,872-102,606	0.346064	annual_inc	9.275699	9.0	-0.275699	9.0
27	27	annual_inc:102,606-120,379	0.208119	annual_inc	5.578296	6.0	0.421704	6.0
28	28	annual_inc:120,379-150,000	0.173450	annual_inc	4.649046	5.0	0.350954	5.0
29	29	dti:<=1.6	0.516757	dti	13.850860	14.0	0.149140	14.0
30	30	dti:1.6-5.599	0.502709	dti	13.474308	13.0	-0.474308	13.0
31	31	dti:5.599-10.397	0.353762	dti	9.482028	9.0	-0.482028	9.0
32	32	dti:10.397-15.196	0.260120	dti	6.972114	7.0	0.027886	7.0
33	33	dti:15.196-19.195	0.142084	dti	3.808325	4.0	0.191675	4.0
34	34	dti:19.195-24.794	0.068490	dti	1.835763	2.0	0.164237	2.0
35	35	dti:24.794-35.191	0.062238	dti	1.668189	2.0	0.331811	2.0
36	36	inq_last_6mths:missing	0.083746	inq_last_6mths	2.244682	2.0	-0.244682	2.0
37	37	inq_last_6mths:0	0.407593	inq_last_6mths	10.924894	11.0	0.075106	11.0
38	38	inq_last_6mths:1-2	0.367752	inq_last_6mths	9.857002	10.0	0.142998	10.0
39	39	inq_last_6mths:3-4	0.267488	inq_last_6mths	7.169590	7.0	-0.169590	7.0
40	40	revol_util:missing	0.039821	revol_util	1.067340	1.0	-0.067340	1.0
41	41	revol_util:<0.1	-0.187724	revol_util	-5.031655	-5.0	0.031655	-5.0
42	42	revol_util:0.1-0.2	-0.201178	revol_util	-5.392260	-5.0	0.392260	-5.0
43	43	revol_util:0.2-0.3	0.136019	revol_util	3.645783	4.0	0.354217	4.0
44	44	revol_util:0.3-0.4	-0.021381	revol_util	-0.573082	-1.0	-0.426918	-1.0
45	45	revol_util:0.4-0.5	0.068239	revol_util	1.829049	2.0	0.170951	2.0
46	46	revol_util:0.5-0.6	0.109897	revol_util	2.945620	3.0	0.054380	3.0
47	47	revol_util:0.6-0.7	-0.054774	revol_util	-1.468132	-1.0	0.468132	-1.0
48	48	revol_util:0.7-0.8	0.013668	revol_util	0.366336	0.0	-0.366336	0.0
49	49	revol_util:0.8-0.9	-0.022090	revol_util	-0.592083	-1.0	-0.407917	-1.0
50	50	revol_util:0.9-1.0	0.131379	revol_util	3.521410	4.0	0.478590	4.0
51	51	out_prncp:<1,286	-3.477300	out_prncp	-93.203508	-93.0	0.203508	-93.0
52	52	out_prncp:1,286-6,432	-1.011833	out_prncp	-27.120569	-27.0	0.120569	-27.0
53	53	out_prncp:6,432-9,005	-0.603413	out_prncp	-16.173518	-16.0	0.173518	-16.0
54	54	out_prncp:9,005-10,291	-0.372086	out_prncp	-9.973177	-10.0	-0.026823	-10.0
55	55	out_prncp:10,291-15,437	-0.258860	out_prncp	-6.938318	-7.0	-0.061682	-7.0
56	56	total_pymnt:<10,000	-6.168987	total_pymnt	-165.349908	-165.0	0.349908	-165.0
57	57	total_pymnt:10,000-15,000	-3.988031	total_pymnt	-106.892844	-107.0	-0.107156	-107.0
58	58	total_pymnt:15,000-20,000	-2.876217	total_pymnt	-77.092438	-77.0	0.092438	-77.0
59	59	total_pymnt:20,000-25,000	-1.821992	total_pymnt	-48.835589	-49.0	-0.164411	-49.0
60	60	total_rec_int:<1,089	4.154453	total_rec_int	111.353524	111.0	-0.353524	111.0
61	61	total_rec_int:1,089-2,541	3.625060	total_rec_int	97.163983	97.0	-0.163983	97.0
62	62	total_rec_int:2,541-4,719	2.531826	total_rec_int	67.861581	68.0	0.138419	68.0
63	63	total_rec_int:4,719-7,260	1.281351	total_rec_int	34.344582	34.0	-0.344582	34.0
64	64	total_rev_hi_lim:missing	0.161101	total_rev_hi_lim	4.318048	4.0	-0.318048	4.0
65	65	total_rev_hi_lim:<6,381	0.437075	total_rev_hi_lim	11.715092	12.0	0.284908	12.0
66	66	total_rev_hi_lim:6,381-19,144	0.365133	total_rev_hi_lim	9.786804	10.0	0.213196	10.0
67	67	total_rev_hi_lim:19,144-25,525	0.256432	total_rev_hi_lim	6.873246	7.0	0.126754	7.0
68	68	total_rev_hi_lim:25,525-35,097	0.136156	total_rev_hi_lim	3.649437	4.0	0.350563	4.0
69	69	total_rev_hi_lim:35,097-54,241	0.087641	total_rev_hi_lim	2.349080	2.0	-0.349080	2.0
70	70	total_rev_hi_lim:54,241-79,780	0.027509	total_rev_hi_lim	0.737337	1.0	0.262663	1.0
71	71	mths_since_earliest_cr_line:missing	0.083746	mths_since_earliest_cr_line	2.244682	2.0	-0.244682	2.0
72	72	mths_since_earliest_cr_line:<125	-0.080289	mths_since_earliest_cr_line	-2.152006	-2.0	0.152006	-2.0
73	73	mths_since_earliest_cr_line:125-167	-0.003298	mths_since_earliest_cr_line	-0.088402	-0.0	0.088402	-0.0
74	74	mths_since_earliest_cr_line:167-249	-0.002011	mths_since_earliest_cr_line	-0.053914	-0.0	0.053914	-0.0
75	75	mths_since_earliest_cr_line:249-331	0.034296	mths_since_earliest_cr_line	0.919239	1.0	0.080761	1.0
76	76	mths_since_earliest_cr_line:331-434	0.048567	mths_since_earliest_cr_line	1.301774	1.0	-0.301774	1.0
77	77	mths_since_issue_d:<79	0.262352	mths_since_issue_d	7.031937	7.0	-0.031937	7.0
78	78	mths_since_issue_d:79-89	0.652052	mths_since_issue_d	17.477214	17.0	-0.477214	17.0
79	79	mths_since_issue_d:89-100	0.612261	mths_since_issue_d	16.410675	16.0	-0.410675	16.0
80	80	mths_since_issue_d:100-122	0.517805	mths_since_issue_d	13.878936	14.0	0.121064	14.0
81	81	mths_since_last_credit_pull_d:missing	0.040285	mths_since_last_credit_pull_d	1.079775	1.0	-0.079775	1.0
82	82	mths_since_last_credit_pull_d:<56	0.394185	mths_since_last_credit_pull_d	10.565514	11.0	0.434486	11.0
83	83	mths_since_last_credit_pull_d:56-61	-0.155123	mths_since_last_credit_pull_d	-4.157821	-4.0	0.157821	-4.0
84	84	mths_since_last_credit_pull_d:61-75	-0.009916	mths_since_last_credit_pull_d	-0.265776	-0.0	0.265776	-0.0
85	0	mths_since_last_credit_pull_d:>75	0.000000	mths_since_last_credit_pull_d	0.000000	0.0	0.000000	0.0
86	1	mths_since_issue_d:>122	0.000000	mths_since_issue_d	0.000000	0.0	0.000000	0.0
87	2	mths_since_earliest_cr_line:>434	0.000000	mths_since_earliest_cr_line	0.000000	0.0	0.000000	0.0
88	3	total_rev_hi_lim:>79,780	0.000000	total_rev_hi_lim	0.000000	0.0	0.000000	0.0
89	4	total_rec_int:>7,260	0.000000	total_rec_int	0.000000	0.0	0.000000	0.0
90	5	total_pymnt:>25,000	0.000000	total_pymnt	0.000000	0.0	0.000000	0.0
91	6	out_prncp:>15,437	0.000000	out_prncp	0.000000	0.0	0.000000	0.0
92	7	revol_util:>1.0	0.000000	revol_util	0.000000	0.0	0.000000	0.0
93	8	inq_last_6mths:>4	0.000000	inq_last_6mths	0.000000	0.0	0.000000	0.0
94	9	dti:>35.191	0.000000	dti	0.000000	0.0	0.000000	0.0
95	10	annual_inc:>150K	0.000000	annual_inc	0.000000	0.0	0.000000	0.0
96	11	int_rate:>20.281	0.000000	int_rate	0.000000	0.0	0.000000	0.0
97	12	term:60	0.000000	term	0.000000	0.0	0.000000	0.0
98	13	purpose:major_purch__car__home_impr	0.000000	purpose	0.000000	0.0	0.000000	0.0
99	14	verification_status:Not Verified	0.000000	verification_status	0.000000	0.0	0.000000	0.0
100	15	home_ownership:MORTGAGE	0.000000	home_ownership	0.000000	0.0	0.000000	0.0
101	16	grade:G	0.000000	grade	0.000000	0.0	0.000000	0.0

In [120…

```
print(df_scorecard.groupby('Original feature name')['Score - Final'].min().sum())
print(df_scorecard.groupby('Original feature name')['Score - Final'].max().sum())
```

```
301.0
850.0
```

To calculate the credit score of a borrower, we only have to sum the credit scores corresponding to the respective dummy variables categories

In [121…

```
# first create a transformed test set through our WoE_Binning custom class
X_test_woe_transformed = woe_transform.fit_transform(X_test)
# insert an Intercept column in its beginning to align with the # of rows in scorecard
X_test_woe_transformed.insert(0, 'Intercept', 1)
X_test_woe_transformed.head()
```

Out[121…

	Intercept	grade:A	grade:B	grade:C	grade:D	grade:E	grade:F	home_ownership:OWN	home_ownership:OTHER_NONE_RENT	verification_status:Source Verified	verification_status:Verified	purpose:d
395346	1	False	False	True	False	False	False	True	0	True	False	
376583	1	True	False	False	False	False	False	False	1	False	False	
297790	1	False	False	True	False	False	False	False	1	True	False	
47347	1	False	True	False	False	False	False	False	1	False	True	
446772	1	False	False	False	True	False	False	False	0	True	False	

In [122…

```
# get the list of our final scorecard scores
scorecard_scores = df_scorecard['Score - Final']
# check the shapes of test set and scorecard before doing matrix dot multiplication
print(X_test_woe_transformed.shape)
print(scorecard_scores.shape)
```

```
(93257, 85)
(102,)
```

In [123…

```
# we can see that the test set has 17 less columns than the rows in scorecard due to the reference categories
# since the reference categories will always be scored as 0 based on the scorecard, it is safe to add these categories to the end of test set with 0 values
X_test_woe_transformed = pd.concat([X_test_woe_transformed, pd.DataFrame(dict.fromkeys(ref_categories, [0] * len(X_test_woe_transformed)),
                                                                         index = X_test_woe_transformed.index)], axis = 1)
# Need to reshape scorecard_scores so that it is (102,1) to allow for matrix dot multiplication
scorecard_scores = scorecard_scores.values.reshape(102, 1)
print(X_test_woe_transformed.shape)
print(scorecard_scores.shape)
```

```
(93257, 102)
(102, 1)
```

In [124…

```
scorecard_scores
```

Out[124…

```
array([[ 601.],
       [  24.],
       [  18.],
       [  14.],
       [  12.],
       [   8.],
       [   4.],
       [  -2.],
       [  -3.],
       [  -7.],
       [ -13.],
       [  -8.],
       [  -6.],
       [ -13.],
       [  -0.],
       [  -2.],
       [  29.],
       [  11.],
       [   5.],
       [   4.],
       [   1.],
       [   1.],
       [  14.],
       [  12.],
       [  12.],
       [  11.],
       [   9.],
       [   6.],
       [   5.],
       [  14.],
       [  13.],
       [   9.],
       [   7.],
       [   4.],
       [   2.],
       [   2.],
       [   2.],
       [  11.],
       [  10.],
       [   7.],
       [   1.],
       [  -5.],
       [  -5.],
       [   4.],
       [  -1.],
       [   2.],
       [   3.],
       [  -1.],
       [   0.],
       [  -1.],
       [   4.],
       [ -93.],
       [ -27.],
       [ -16.],
       [ -10.],
       [  -7.],
       [-165.],
       [-107.],
       [ -77.],
       [ -49.],
       [ 111.],
       [  97.],
       [  68.],
       [  34.],
       [   4.],
       [  12.],
       [  10.],
       [   7.],
       [   4.],
       [   2.],
       [   1.],
       [   2.],
       [  -2.],
       [  -0.],
       [  -0.],
       [   1.],
       [   1.],
       [   7.],
       [  17.],
       [  16.],
       [  14.],
       [   1.],
       [  11.],
       [  -4.],
       [  -0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.],
       [   0.]])
```

In [125…

```
X_test_woe_transformed.head()
```

Out[125…

	Intercept	grade:A	grade:B	grade:C	grade:D	grade:E	grade:F	home_ownership:OWN	home_ownership:OTHER_NONE_RENT	verification_status:Source Verified	verification_status:Verified	purpose:d
395346	1	False	False	True	False	False	False	True	0	True	False	
376583	1	True	False	False	False	False	False	False	1	False	False	
297790	1	False	False	True	False	False	False	False	1	True	False	
47347	1	False	True	False	False	False	False	False	1	False	True	
446772	1	False	False	False	True	False	False	False	0	True	False	

In [126…

```
# matrix dot multiplication of test set with scorecard scores
y_scores = X_test_woe_transformed.dot(scorecard_scores)
y_scores.head()
```

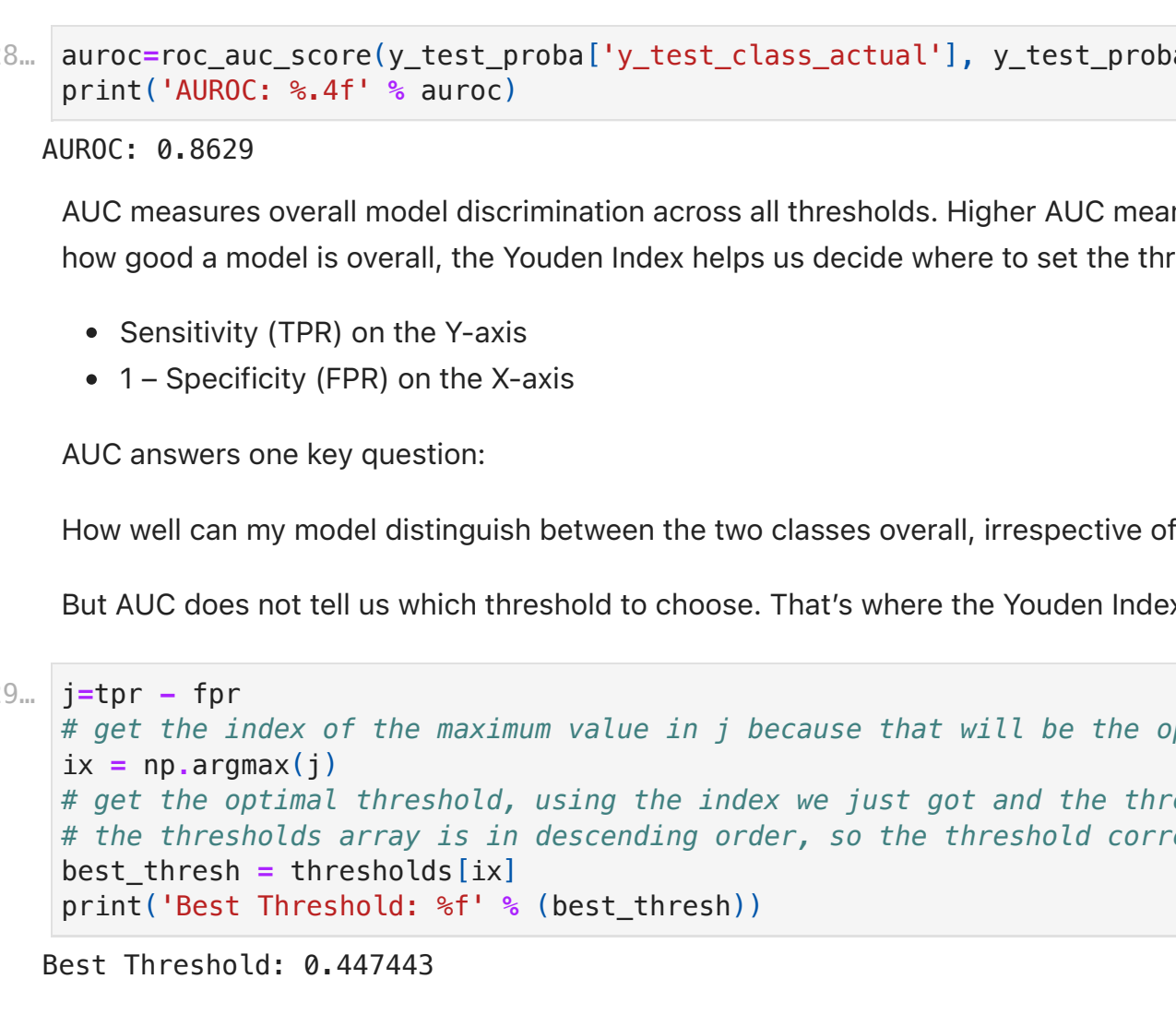
Out[126…

	0
395346	515.0
376583	595.0
297790	547.0
47347	575.0
446772	483.0

Threshold

In [127…

```
# get the values required to plot a ROC curve
fpr, tpr, thresholds = roc_curve(y_test_proba['y_test_class_actual'], y_test_proba['y_hat_test_proba'])
# plot the ROC curve
plt.plot(fpr, tpr)
# plot a secondary diagonal line, with dashed line style and black color to represent a no-skill classifier
plt.plot(fpr, fpr, linestyle = '--', color = 'k')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve');
```



In [128…

```
auroc=roc_auc_score(y_test_proba['y_test_class_actual'], y_test_proba['y_hat_test_proba'])
print('AUROC: %.4f' % auroc)
```

```
AUROC: 0.8629
```

AUC measures overall model discrimination across all thresholds. Higher AUC means the ROC curve bows more toward the top-left, capturing higher TPR at low FPR across thresholds.While AUC–ROC tells us how good a model is overall, the Youden Index helps us decide where to set the threshold. Together, they form a complete and practical framework for evaluating classifiers. The ROC curve plots:

- Sensitivity (TPR) on the Y-axis
- 1 – Specificity (FPR) on the X-axis

AUC answers one key question:

How well can my model distinguish between the two classes overall, irrespective of threshold?

But AUC does not tell us which threshold to choose. That's where the Youden Index (J) steps in.

In [129…

```
j=tpr - fpr
# get the index of the maximum value in j because that will be the optimal threshold
ix = np.argmax(j)
# get the optimal threshold, using the index we just got and the thresholds array from the roc_curve function, which contains the thresholds corresponding to each point in the R
# the thresholds array is in descending order, so the threshold corresponding to the maximum value of j will be the optimal threshold that gives us the best balance between true
best_thresh = thresholds[ix]
print('Best Threshold: %f' % (best_thresh))
```

```
Best Threshold: 0.447443
```

This means that based on the Youden's J statistic, the ideal probability threshold is 0.448989 which minimizes the FPR and maximimizes the TPR - all samples with a predicted probability higher than this should be classified as in Default and vice versa. At first, this ideal threshold appears to be counterintuitive compared to the default probability threshold of 0.5. But remember that we used the class_weight parameter when fitting our logistic regression model that would have helped us.

With class_weight='balanced' on imbalanced credit data (few defaults), the model learns to output lower probabilities for the positive class to compensate for its high training weight. Standard >0.5 would miss many defaults; Youden's optimal 0.449 corrects this by classifying more conservatively at inference

Update and apply the new threshold

In [130…

```
Threshold =0.5
# assign a threshold value to differentiate good with bad
tr = 0.5
# crate a new column for the predicted class based on predicted probabilities and threshold
# We will determine this optimat threshold later in this project
y_test_proba['y_test_class_predicted'] = np.where(y_test_proba['y_hat_test_proba'] > tr, 1, 0)
# create the confusion matrix
confusion_matrix(y_test_proba['y_test_class_actual'], y_test_proba['y_test_class_predicted'], normalize = 'all')
```

Out[130…

```
array([[0.08863678, 0.02067405],
       [0.19034496, 0.70034421]])
```

In [131…

```
# update the threshold value
tr = best_thresh
# crate a new column for the predicted class based on predicted probabilities and threshold
y_test_proba['y_test_class_predicted'] = np.where(y_test_proba['y_hat_test_proba'] > tr, 1, 0)
# create the confusion matrix
confusion_matrix(y_test_proba['y_test_class_actual'], y_test_proba['y_test_class_predicted'], normalize = 'all')
```

Out[131…

```
array([[0.08625626, 0.02305457],
       [0.16610013, 0.72458904]])
```

The updated confusion matrix shows a marked improvement in the TPR at marginal cost of lower TNR. But at the same time, FNR has improved drastically with a corresponding marginal increase in FPR rewrite it

In [132…

```
# create a new DF comprising of the thresholds from the ROC output
df_cutoffs = pd.DataFrame(thresholds, columns = ['thresholds'])
# calculate Score corresponding to each threshold
df_cutoffs['Score'] = ((np.log(df_cutoffs['thresholds'] / (1 - df_cutoffs['thresholds'])) - min_sum_coef) *
                       ((max_score - min_score) / (max_sum_coef - min_sum_coef)) + min_score).round()
df_cutoffs.head()
```

Out[132…

	thresholds	Score
0	inf	NaN
1	0.999550	742.0
2	0.996290	685.0
3	0.996288	685.0
4	0.996039	683.0

In [133…

```
# We define a function called 'n_approved' which assigns a value of 1 if a predicted probability
# is greater than the parameter p, which is a threshold, and a value of 0, if it is not.
# Then it sums the column.
# Thus, for given any percentage values the function will return
# the number of rows wih estimated probabilites greater than the threshold.
def n_approved(p):
    return np.where(y_test_proba['y_hat_test_proba'] >= p, 1, 0).sum()
```

In [134…

```
# Assuming that all credit applications above a given probability of being 'good' will be approved,
# when we apply the 'n_approved' function to a threshold, it will return the number of approved applications.
# Thus, here we calculate the number of approved appliations for al thresholds.
df_cutoffs['N Approved'] = df_cutoffs['thresholds'].apply(n_approved)
# Then, we calculate the number of rejected applications for each threshold.
# It is the difference between the total number of applications and the approved applications for that threshold.
df_cutoffs['N Rejected'] = y_test_proba['y_hat_test_proba'].shape[0] - df_cutoffs['N Approved']
# Approval rate equalts the ratio of the approved applications and all applications.
df_cutoffs['Approval Rate'] = df_cutoffs['N Approved'] / y_test_proba['y_hat_test_proba'].shape[0]
# Rejection rate equals one minus approval rate.
df_cutoffs['Rejection Rate'] = 1 - df_cutoffs['Approval Rate']
df_cutoffs.head()
```

Out[134…

	thresholds	Score	N Approved	N Rejected	Approval Rate	Rejection Rate
0	inf	NaN	0	93257	0.000000	1.000000
1	0.999550	742.0	1	93256	0.000011	0.999989
2	0.996290	685.0	261	92996	0.002799	0.997201
3	0.996288	685.0	263	92994	0.002820	0.997180
4	0.996039	683.0	293	92964	0.003142	0.996858

In [135…

```
#remove the thresholds that give us an approval rate of 0% and 100% because they are not useful for decision making
df_cutoffs = df_cutoffs[(df_cutoffs['Approval Rate'] > 0) & (df_cutoffs['Approval Rate'] < 1)]
df_cutoffs.head()
```

Out[135…

	thresholds	Score	N Approved	N Rejected	Approval Rate	Rejection Rate
1	0.999550	742.0	1	93256	0.000011	0.999989
2	0.996290	685.0	261	92996	0.002799	0.997201
3	0.996288	685.0	263	92994	0.002820	0.997180
4	0.996039	683.0	293	92964	0.003142	0.996858
5	0.996037	683.0	295	92962	0.003163	0.996837

In [136…

```
# We define a function called 'n_approved' which assigns a value of 1 if a predicted probability
# is greater than the parameter p, which is a threshold, and a value of 0, if it is not.
# Then it sums the column.
# Thus, for given any percentage values, the function will return
# the number of rows wih estimated probabilites greater than the threshold.
def n_approved(p):
    return np.where(y_test_proba['y_hat_test_proba'] >= p, 1, 0).sum()
```

In [137…

```
# Assuming that all credit applications above a given probability of being 'good' will be approved,
# when we apply the 'n_approved' function to a threshold, it will return the number of approved applications.
# Thus, here we calculate the number of approved appliations for al thresholds.
df_cutoffs['N Approved'] = df_cutoffs['thresholds'].apply(n_approved)
# Then, we calculate the number of rejected applications for each threshold.
# It is the difference between the total number of applications and the approved applications for that threshold.
df_cutoffs['N Rejected'] = y_test_proba['y_hat_test_proba'].shape[0] - df_cutoffs['N Approved']
# Approval rate equalts the ratio of the approved applications and all applications.
df_cutoffs['Approval Rate'] = df_cutoffs['N Approved'] / y_test_proba['y_hat_test_proba'].shape[0]
# Rejection rate equals one minus approval rate.
df_cutoffs['Rejection Rate'] = 1 - df_cutoffs['Approval Rate']
df_cutoffs.head()
```

Out[137…

	thresholds	Score	N Approved	N Rejected	Approval Rate	Rejection Rate
1	0.999550	742.0	1	93256	0.000011	0.999989
2	0.996290	685.0	261	92996	0.002799	0.997201
3	0.996288	685.0	263	92994	0.002820	0.997180
4	0.996039	683.0	293	92964	0.003142	0.996858
5	0.996037	683.0	295	92962	0.003163	0.996837

In [138…

```
# let's have a look at the approval and rejection rates at our ideal threshold
df_cutoffs[df_cutoffs['thresholds'].between(0.448987, 0.448990)]# let's have a look at the approval and rejection rates at our ideal threshold
```

Out[138…

thresholds	Score	N Approved	N Rejected	Approval Rate	Rejection Rate

In [139…

```
# compare the above rates with the case of the default 0.5 threshold
df_cutoffs[df_cutoffs['thresholds'].between(0.50, 0.5009)]
```

Out[139…

	thresholds	Score	N Approved	N Rejected	Approval Rate	Rejection Rate
7132	0.500738	535.0	67202	26055	0.720611	0.279389
7133	0.500675	535.0	67204	26053	0.720632	0.279368
7134	0.500642	535.0	67205	26052	0.720643	0.279357
7135	0.500628	535.0	67206	26051	0.720654	0.279346
7136	0.500579	535.0	67207	26050	0.720664	0.279336
7137	0.500536	535.0	67211	26046	0.720707	0.279293
7138	0.500503	535.0	67213	26044	0.720729	0.279271
7139	0.500358	535.0	67221	26036	0.720815	0.279185
7140	0.500353	535.0	67222	26035	0.720825	0.279175
7141	0.500113	535.0	67234	26023	0.720954	0.279046
7142	0.500113	535.0	67236	26021	0.720975	0.279025

0.5 threshold would result in a very high rejection rate with a corresponding loss of business.

Accordingly, we will stick with our ideal threshold and the corresponding Credit Score of 535, and will monitor the model's performance in production